

КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА
Факультет інформаційних технологій
Кафедра інтелектуальних технологій

КВАЛІФІКАЦІЙНА РОБОТА
на здобуття освітнього ступеня «магістр»
НА ТЕМУ:

«Розробка та дослідження інтелектуальної технології
визначення привабливості тварин із притулку»

Галузь знань: 12 «Інформаційні технології»

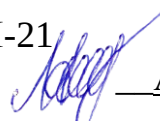
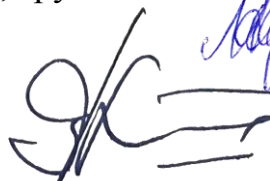
Спеціальність: 122 «Комп'ютерні науки»

Освітньо-наукова програма «Технології штучного інтелекту»

Виконала:

студентка 2 курсу магістратури, групи ТШП-21

Науковий керівник:

 Антоневич М. М.
(ПІБ)
 Снитюк В. Є.
(ПІБ)
Д. Т. Н., професор
(науковий ступінь, вчене звання)

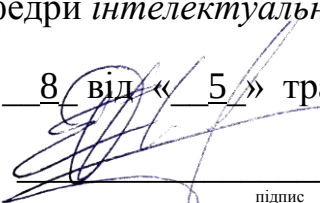
Засвідчую, що в цій кваліфікаційній роботі
немає запозичень з праць інших авторів без
відповідних посилань

Студентка


підпис

Кваліфікаційна робота допущена до захисту
рішенням кафедри *інтелектуальних технологій*

Протокол № 8 від «5» травня 2022 р.

Зав. кафедри  доц. Іларіонов О.Є.

підпис

Київ 2022

РЕФЕРАТ

Кваліфікаційна робота складається зі вступу, 3 розділів, висновків, списку використаної літератури із 93 джерел та 8 додатків. Загальний обсяг роботи 161 сторінка. Робота містить 12 таблиць та 113 рисунків.

Актуальність теми. Процеси у сучасному світі супроводжуються безліччю викликів. Один із них - наявність безпритульних тварин у населених пунктах. Щодня на вулицях зазнають травм багато бездомних тварин. Такі тварини, як правило, покинуті людьми [1]. Відомо, що тварини з якісними фотографіями на сайті притулку викликають набагато більший інтерес і вони, як правило, отримують свій дім швидше. Такі сайти, як правило, не оснащені інтелектуальними технологіями, які можуть прогнозувати оцінку привабливості тварини за фото. Через це, створення саме такої технології та подальше її використання допоможе пришвидшити процес пошуку та підбору нових власників тваринам із притулку, адже керівники притулку зможуть аналізувати фотографії, що будуть завантажуватися у профілі тварин та покращувати їх саме за допомогою використання даної інтелектуальної технології. Така інтелектуальна технологія дозволяє об'єктивізувати вибір тих людей, які б могли взяти ту чи іншу тварину. Навіть ті люди, що не планують брати тварину, можуть подивитися на якісну фотографію та змінити свій вибір і взяти тварину із притулку.

Мета роботи: розробка моделей структурної та параметричної оптимізації нейромережної технології визначення привабливості тварин із притулку.

Об'єкт дослідження: процеси оцінювання привабливості тварин із притулку з використанням технологій штучного інтелекту.

Предмет дослідження: інтелектуальна технологія визначення привабливості тварин із притулку з використанням нейронних мереж.

Результати роботи. Кінцевим результатом є система підтримки прийняття рішень (СППР) прогнозування привабливості тварин із притулку за фото. У

результаті аналізу проаналізовані та досліджені архітектури нейронних мереж для даної задачі. Також у результаті аналізу проаналізовано та досліджено значення гіперпараметрів обраних нейронних мереж. За результатами досліджень обрано найкращу модель нейронної мережі, а саме Swin Transformer. Взято участь у конкурсі PetFinder.my - Pawpularity Contest на сайті kaggle.com (23.IX.2021 - 14.I.2022). Спроековано та розроблено архітектуру програмного модуля (СППР) прогнозування привабливості тварин із притулку за фото у вигляді веб-додатку (HTML/CSS/JS, Python Flask) із використанням бази даних MySQL та нейронних мереж (Python fastai).

Апробація роботи:

- довідка про використання результатів випускної кваліфікаційної роботи від Громадської Організації Черкаське міське товариство захисту тварин "Друг" (Додаток А);
- тези доповіді “Development and Research of the Intelligent Technology for Predicting the Popularity of Animals From the Shelter” та виступ на VIII Міжнародній науково-практичній конференції Information Technology and Implementation (Satellite) 2021 (Додаток Б).

Ключові слова: тварини; притулок; штучний інтелект; розпізнавання образів; штучна нейронна мережа; Swin Transformer; Python; MySQL; веб-додаток.

ABSTRACT

Qualifying work consists of an introduction, 3 chapters, conclusions, a list of references from 93 sources and 8 appendices. The total amount of work is 161 pages. The work contains 12 Tables and 113 Figures.

Relevance of the topic of the work. Processes in the modern world are accompanied by many challenges. One of them is the presence of homeless animals in shelters. Many stray animals are injured on the streets every day. Such animals are usually abandoned by humans [1]. It is obvious, that animals with high-quality photos on the shelter website have greater interest from potential pet owners and such animals tend to get their home faster. Such websites are usually not equipped with intelligent technologies that can predict the attractiveness of the shelter animal by the photo. Because of this, creation of such a technology and its further use will help speed up the process of finding and selecting new owners for animals from the shelter, as shelter managers will be able to analyze photos uploaded to animal profiles and improve them using this intelligent technology. Such intelligent technology allows to objectify the choice of those people who could take this or that animal. Even those people who do not plan to take an shelter animal, can look at a high quality photo and change their choice and take the animal from the shelter.

The goal of the work: development of models of structural and parametric optimization of neural network technology to determine the attractiveness of animals from the shelter.

Object of study: the processes of evaluating the attractiveness of animals from the shelter using artificial intelligence technologies.

Subject of study: the intelligent technology for determining the attractiveness of animals from the shelter using neural networks.

Results of the work. The end result is a decision support system (DSS) for predicting the attractiveness of animals from the shelter by their photo. The architectures of neural networks for this task are analyzed and investigated. The

selected neural networks values of hyperparameters were analyzed and investigated. According to the results of the research, the best model of the neural network was chosen, Swin Transformer. Took part in the PetFinder.my - Pawpularity Contest competition on kaggle.com (23.IX.2021 - 14.I.2022). The given software module (DSS) for predicting the attractiveness of animals from the shelter by their photos is represented as a web application (HTML / CSS / JS, Python Flask), using MySQL database and neural networks (Python fastai).

Approbation of work:

- certificate on the use of the results of the final qualification work from the Public Organization Cherkasy City Society for the Protection of Animals "Friend" (Appendix A);
- the thesis "Development and Research of the Intelligent Technology for Predicting the Popularity of Animals From the Shelter" and speech at the VIII International Scientific and Practical Conference Information Technology and Implementation (Satellite) 2021 (Appendix B).

Key words: animals; shelter; artificial intelligence; pattern recognition; artificial neural network; Swin Transformer; Python; MySQL; web application.

ЗМІСТ

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ ТА СКОРОЧЕНЬ.....	8
ВСТУП	9
РОЗДІЛ 1 ПРОБЛЕМА ІНФОРМАЦІЙНО-АНАЛІТИЧНОГО СУПРОВОДУ У ПРОЦЕСІ ПРИЙНЯТТЯ РІШЕНЬ ЩОДО ВИЗНАЧЕННЯ ПРИВАБЛИВОСТІ ТВАРИН ІЗ ПРИТУЛКУ	11
1.1 Аспекти вирішення проблем привабливості тварин із притулку	11
1.2 Аналітичний огляд технологій для визначення привабливості тварин із притулку	14
1.3 Структурні елементи дослідження прогнозування привабливості тварин із притулку	22
1.4 Висновки до розділу 1	30
РОЗДІЛ 2 ПРОЕКТУВАННЯ ІНТЕЛЕКТУАЛЬНОЇ ТЕХНОЛОГІЇ ПРОГНОЗУВАННЯ ПРИВАБЛИВОСТІ ТВАРИН ІЗ ПРИТУЛКУ	31
2.1 Проектування архітектури технології інтелектуальної технології прогнозування привабливості тварин із притулку	31
2.2 Нейромережні моделі для прогнозування привабливості тварин із притулку	36
2.3 Особливості застосування бібліотек Python для реалізації нейронних мереж	59
2.4 Логіка бази даних	64
2.5 Логіка інтерфейсу.....	68
2.5. Висновки до розділу 2	72
РОЗДІЛ 3 РЕАЛІЗАЦІЯ ІНТЕЛЕКТУАЛЬНОЇ ТЕХНОЛОГІЇ. РЕЗУЛЬТАТИ ДОСЛІДЖЕНЬ ТА ЕКСПЕРИМЕНТІВ.....	74
3.1 Вхідні обмеження для проведення експериментів	74
3.2 Реалізація та вибір оптимальної мережі	74
3.3 Веб-додаток.....	118
3.4 Висновки до розділу 3	120
ВИСНОВКИ.....	122
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	124

ДОДАТОК А Довідка про використання результатів випускної кваліфікаційної роботи	133
ДОДАТОК Б Матеріали конференції IT&I-2021 (Satellite)	134
ДОДАТОК В Лістинг програми навчання Swin Transformer (Модель-34)	141
ДОДАТОК Г Лістинг програми “app.py”	149
ДОДАТОК Ґ Лістинг програми “database_func.py”	152
ДОДАТОК Д Лістинг програми “neural_net_func.py”	153
ДОДАТОК Е Знімки екрану веб-сайту	156
ДОДАТОК Є Знімки екрану таблиць бази даних MySQL	161

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ ТА СКОРОЧЕНЬ

БД - база даних

НМ - нейронна мережа

ІС - інформаційна система

ПрО - предметна область

ВСТУП

У першому розділі розглянуто аспекти вирішення проблем привабливості тварин із притулку. Розглянуто проблему визначення привабливості тварин, базуючись на основних принципах і аспектах інформаційних технологій, а саме - штучного інтелекту та розпізнавання образів. Відомо, що у першу чергу, коли потенційні власники планують взяти тварину із притулку, вони заходять на сайт і роздивляються фотографії собак чи котів. Очевидно, що тварини з якісними фотографіями на сайті притулку викликають набагато більший інтерес і вони, як правило, отримують свій дім швидше. Такі сайти, як правило, не оснащені інтелектуальними технологіями, які можуть давати оцінку привабливості тварини за фото.

Через це, створення саме такої технології та подальше її використання допоможе пришвидшити процес пошуку та підбору нових власників тваринам із притулку, адже керівники притулку зможуть аналізувати фотографії, що будуть завантажуватися у профілі тварин та покращувати їх саме за допомогою використання даної інтелектуальної технології.

Така інтелектуальна технологія дозволяє об'єктивізувати вибір тих людей, які б могли взяти ту чи іншу тварину. Ті люди, що не планують брати тварину, можуть подивитися на якісну фотографію та змінити свій вибір і взяти тварину із притулку.

Також, було здійснено огляд існуючих засобів та технологій прогнозування привабливості тварин із притулку. Проаналізовано та досліджено рішення із відкритих джерел.

У другому розділі було спроектовано елементи інтелектуальної технології. Побудовано DFD та структурну діаграми. Виконано аналіз нейромережних засобів для виконання випускної кваліфікаційної роботи. Визначено переваги та недоліки нейронних мереж. Вибрано як основні засоби для дослідження згорткові нейронні мережі (Xception) та трансформери (Swin Transformer).

У третьому розділі проведені експерименти. Всього було проведено більше 30 вдалих досліджень. Експерименти проводилися із використанням згорткових нейронних мереж і трансформерів. Аналізуючи отримані результати, бачимо, що використання Swin Transformer покращило фінальні результати у порівнянні із згортковими нейронними мережами. Створений інтелектуальний додаток за допомогою веб-складових HTML/CSS/JavaScript, серверної сторони Python Flask та бази даних MySQL, із використанням MySQL Workbench та MySQL Server.

Аналізуючи отримані результати, можливо запропонувати майбутні покращення та вдосконалення. Як відомо, нейронні мережі можна вчити не лише за відомими правилами (наприклад, back propagation), а і з використанням методики підбору випадкових коефіцієнтів. До таких методик можна віднести ті, що використовують еволюційні оптимізаційні методи. Такі методи при виконанні певних умов можуть давати не лише локальний оптимум, а й глобальний оптимум. До таких можна віднести наступні методи: генетичні алгоритми, еволюційні стратегії, метод деформованих зірок [92-93] тощо.

Запропоновані у магістерській випускній кваліфікаційній роботі результати знайшли своє практичне відображення у розробці системи підтримки прийняття рішень, результати якої використані та впроваджені. Це підтверджується довідкою про використання результатів випускної кваліфікаційної роботи від Громадської Організації Черкаське міське товариство захисту тварин "Друг" (Додаток А), опублікованими тезами доповіді "Development and Research of the Intelligent Technology for Predicting the Popularity of Animals From the Shelter" та виступом на VIII Міжнародній науково-практичній конференції Information Technology and Implementation (Satellite) 2021 (Додаток Б).

РОЗДІЛ 1 ПРОБЛЕМА ІНФОРМАЦІЙНО-АНАЛІТИЧНОГО СУПРОВОДУ У ПРОЦЕСІ ПРИЙНЯТТЯ РІШЕНЬ ЩОДО ВИЗНАЧЕННЯ ПРИВАБЛИВОСТІ ТВАРИН ІЗ ПРИТУЛКУ

1.1 Аспекти вирішення проблем привабливості тварин із притулку

Процеси у сучасному світі супроводжуються безліччю викликів. Один із них - наявність безпритульних тварин у населених пунктах. Щодня на їх вулицях зазнають травм мільйони бездомних тварин. Такі тварини, як правило, покинуті людьми [1]. Проблема безпритульних покинутих тварин також має місце і в Україні. За статистикою, в українських притулках для безпритульних тварин проживають десятки тисяч котів та собак. Прихистку не мають багато покинутих тварин, які блукають вулицями міст та сіл. Відомо, що підрахунок безпритульних та домашніх тварин ведеться лише у деяких містах, але у 2017 році, за даними Washington Post, в Україні налічувалося близько 7,5 млн бездомних собак і котів. Наша країна у 10-ці лідерів серед країн світу за найбільшою кількістю тварин, зокрема і безпритульних [2]. Проблема бездомних котів та собак створена людьми. Адже всі безпритульні тварини, які зараз є у містах, потрапляють туди, як правило, не з власної волі. Раніше це могли бути чийсь домашні улюбленці, популяція яких неконтрольовано збільшувалася впродовж десятків років. Є багато українців, які готові допомагати бездомним тваринам, як стверджує волонтерка [2]. Більше того, іноді, допомога людини може бути такою: виявивши безпритульну тварину, її намагаються терміново відправити до притулку, вважаючи це рішення єдиноправильним. Однак, якими би не були за розмірами притулки, вони є лише частиною загального ресурсу, тому такі притулки можуть бути не розраховані на таку кількість тварин, як потрібно. До того ж, більшістю наявних притулків, на сьогодні, опікуються волонтери, в яких не достатньо ні сил, ні коштів, ні їжі для тварин, що знаходяться в цих притулках. Багато тварин не можуть знайти свої сім'ї, і це є великою проблемою, не тільки для міста, а й для всієї планети у цілому [2].

Крім того, ці тварини (рис. 1.1) зіштовхуються з багатьма проблемами навіть під час життя у притулках. Життя тварин у притулках супроводжується певною кількістю проблем, до яких можна відносити такі, про які описано нижче.

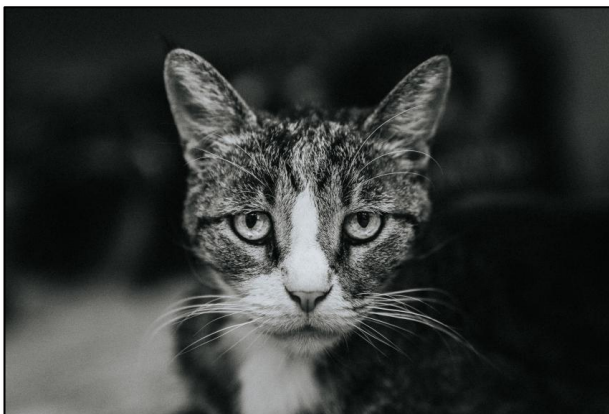


Рисунок 1.1 Кіт у притулку (unsplash.com)

Для більшості собак проживання у такому притулку супроводжується нервовими ситуаціями. Наслідком такого стресу є погіршення фізичного здоров'я.

У роботі [2] зазначено причини, чому необхідно взяти тварину з притулку. Розглянемо деякі з них:

- Порятунком не однієї, а декількох тварин. Взявши одного собаку чи kota, ви автоматично полегшуєте життя ще одній тварині, адже більшість притулків переповнені і мають проблеми із фінансовими ресурсами. Тому власники притулків все частіше відмовляються брати до себе нових тварин. А коли із притулку забирають когось, то це означає, що туди тепер може потрапити інша тварина, замість того, щоб жити на вулиці [2].
- Взяти тварину з притулку - безкоштовно. Зазвичай, такі притулки не відмовляються від пожертвувань щедрих людей, але і платити за тварину ніхто не змушує. Розмір добродійного внеску визначається також на ваш розсуд. Швидше за все, така жертва виявиться меншою, ніж якби ви купували собі улюбленця. Зрозуміло, це не означає, що подальше утримання вихованця для вас

теж буде безкоштовним, адже витрати на них завжди є, неважливо, з притулку вони або із розплідника [2].

- Оразу відомо про стан здоров'я тварини. Волонтери притулків досить зацікавлені в тому, щоб тварини знайшли собі дім і не поверталися до притулку, тому відразу попереджають про різні захворювання чи про проблеми поведінки. А ось про особливості новонароджених тварин з розплідника заводчики можуть приховати інформацію заради фінансової вигоди [2].
- Собака - найвірніший друг. Такий вихованець буде вам відданим і вдячним якого ви заберете зі притулку. Тварина подарує вам всю свою любов, адже для неї ви будете найкращою людиною, яка подарувала їй дім і сім'ю [2].

Відомо, що у 2021 році посилили відповідальність за жорстоке поводження із тваринами, у тому числі і безпритульними (Закон України “Про захист тварин від жорстокого поводження”. Редакція від 08.08.2021).

Зараз, у зв'язку з війною, багатьом людям довелося покинути свої населені пункти, що зазнають атак з боку ворожих військ. Багато мешканців України під час евакуації вивозять своїх домашніх тварин з собою у більш безпечні місця, але трапляються такі ситуації, коли люди покидають улюбленців. Тому притулки продовжують працювати, щоб рятувати тварин, але в той же час ці притулки потребують підтримки — фінансової, матеріальної, інформаційної та волонтерської. Деякі благодійні організації допомагають розшукувати загублених тварин. Волонтери у своїх соціальних мережах регулярно оновлюють актуальну інформацію щодо ситуації з різними притулками та пропонують всім охочим забрати покинутих тварин до себе додому [3]. Тут виникає питання: як зробити так, щоб люди швидше забирали додому ту чи іншу тварину із притулку? Як пришвидшити та покращити цей процес? Які фактори на це впливають? Отже, що можна зробити, щоб допомогти цим тваринам знайти новий будинок? Розглянемо проблему визначення привабливості тварин, базуючись на основних принципах і аспектах інформаційних технологій, а саме - штучного інтелекту та розпізнавання образів.

Відомо, що у першу чергу, коли потенційні власники планують взяти тварину із притулку, вони заходять на сайт і роздивляються фотографії собак чи котів. Очевидно, що тварини з якісними фотографіями на сайті притулку викликають набагато більший інтерес і вони, як правило, отримують свій дім швидше. Такі сайти, як правило, не оснащені інтелектуальними технологіями, які можуть давати оцінку привабливості тварини за фото. Через це, створення саме такої технології та подальше її використання допоможе пришвидшити процес пошуку та підбору нових власників тваринам із притулку, адже керівники притулку зможуть аналізувати фотографії, що будуть завантажуватися у профілі тварин та покращувати їх саме за допомогою використання даної інтелектуальної технології. Така інтелектуальна технологія дозволяє об'єктивізувати вибір тих людей, які б могли взяти ту чи іншу тварину. Ті люди, що не планують брати тварину, можуть подивитися на якісну фотографію та змінити свій вибір і взяти тварину із притулку. Використання саме такої інтелектуальної технології позитивно вплине на процеси пошуку тваринам із притулку нових власників.

1.2 Аналітичний огляд технологій для визначення привабливості тварин із притулку

Задача прогнозування взаємодії користувача з домашньою твариною на основі фотографії була запропонована PetFinder.my [4] (рис. 1.2-1.3) у вересні 2021 року. Це провідна платформа, створена для захисту тварин у Малайзії, на якій представлено понад 180'000 тварин, серед яких 54'000 щасливо прийнятих у нові сім'ї. PetFinder тісно співпрацює з любителями тварин, ЗМІ, корпораціями та великими організаціями, щоб покращити добробут тварин. А щоб поліпшити саме якість фотографій бездомних тварин, що завантажуються на сайт притулку, автори зазначають, що використовують засоби штучного інтелекту для розробки ексклюзивного вимірювача привабливості Cuteness Meter, який аналізує привабливість такої фотографії, що завантажується на сайт. Цей алгоритм

аналізує композицію зображення та інші фактори порівняно з продуктивністю тисяч профілів домашніх тварин. Оцінювання відбуваються за такими критеріями, як поза та фон тварини, якість фотографії, композиція зображення, на основі продуктивності тисяч профілів домашніх тварин у системі PetFinder.

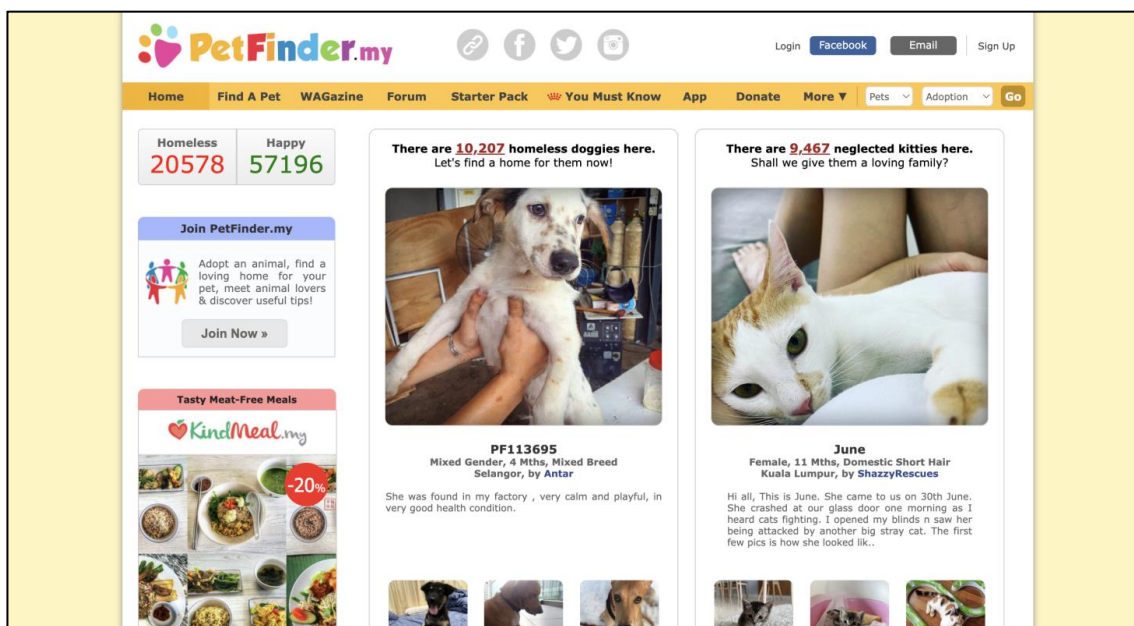


Рисунок 1.2 Сайт PetFinder.my [4]

Цей вимірювач привабливості все ще на експериментальній стадії, як зазначають власники і він, на даний момент, лише для собак і котів, тому вони постійно працюють над удосконаленням алгоритму.



Рисунок 1.3 Емблема PetFinder.my [4]

Тепер розглянемо детальніше, як же працює цей вимірювач привабливості Cuteness Meter. Для цього виконуються такі дії:

1. Увійдіть у свій обліковий запис PetFinder або зареєструйтеся.
2. Створіть профіль для тварини та додайте фотографії.
3. Поруч із кожною фотографією на сторінці «Edit Pet» автоматично з'явиться індикатор Cuteness Meter.
4. Завантажте більше привабливих фотографій і виберіть гарне фото як фотографію профілю для тварини.

Хоча цей основний інструмент є корисним, як зазначають автори, але він все ще знаходиться на стадії експерименту, і алгоритм можна вдосконалити у майбутньому. Через це і виникла проблема покращення і створення нового методу за допомогою штучного інтелекту, який буде більш точно та краще аналізувати зображення тварин і прогнозувати їх привабливість. Що, у свою чергу, зможе підвищити якість та швидкість всиновлення тварин із притулку.

Таким чином, люди із PetFinder.my запустили змагання “PetFinder.my - Pawpularity Contest. Predict the popularity of shelter pet photos” [5] на сайті Kaggle.com (рис. 1.4-1.5) задля того, щоб звичайні люди, науковці і всі бажаючі взяли участь у них і спробували покращити технологію для визначення привабливості тварин. Взагалі, змагання від Kaggle, що називаються Kaggle Competitions, це різні задачі, наприклад, у таких сферах як: прогнозування, класифікація, аналітика даних тощо. Вони запускаються Kaggle або компаніями, такими як Google чи іншими. Якщо ви займаєте призові місця, то ви можете отримати не лише знання, чи фірмові речі Kaggle, а навіть реальні грошові призи; все залежить від умов і правил авторів окремих змагань. Ці поединки різноманітні за типами задач і складністю. Можна взяти участь у будь-якому, навіть якщо ви новачок. Все залежить від самої задачі і Ваших знань. Наприклад, більш складніші змагання набагато цікавіші, а топове місце в конкурсі це чудове доповнення до вашого резюме, наприклад, того ж самого інженера з машинного навчання. Конкурс із прогнозування привабливості тварин “PetFinder.my - Pawpularity Contest. Predict the popularity of shelter pet photos” проходив у такі дати [5]:

1. 23 вересня 2021 р. – дата початку конкурсу.
2. 6 січня 2022 року – крайній термін подачі заявок на конкурс. Учасники повинні прийняти правила змагань до цієї дати, щоб взяти участь у змаганнях.
3. 6 січня 2022 р. – дедлайн для створення команд. Це останній день, коли учасники можуть приєднуватися до команд або об'єднуватися із іншими гравцями.
4. 14 січня 2022 року – кінцевий термін подання коду на конкурс.

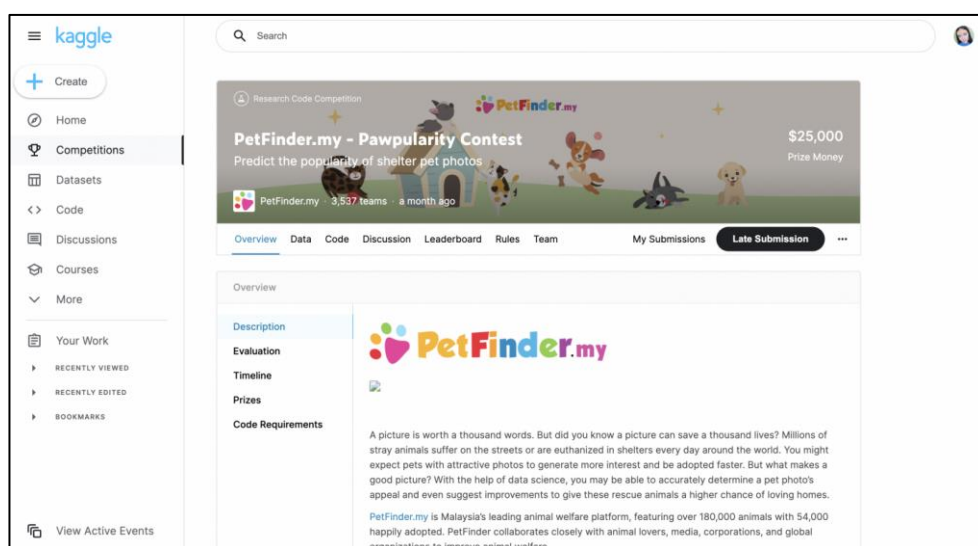


Рисунок 1.4 Сторінка змагань [5]

Усі кінцеві терміни рахуються о 23:59 UTC відповідного дня завершення конкурсу, якщо не зазначено інше. Організатори конкурсу могли залишити за собою право оновлювати графік конкурсу, якщо заважали би це за необхідне.



Рисунок 1.5 Емблема сайту kaggle.com

Так як ця задача виявилася досить актуальною, було вирішено взяти участь у цьому конкурсі із прогнозування привабливості тварин по фото. Для виконання

конкурсної роботи, що лягла в основу випускної кваліфікаційної роботи, були розглянуті роботи конкурсантів із відкритих джерел. Такі роботи проаналізовано та вирішено, як їх можна покращити заради того, щоб отримати більш кращий результат для змагань. Всього участь взяло 3537 команд. Частина таких робіт базувалися на таких Python-бібліотеках, як:

- `timm` [6]. Це бібліотека глибинного навчання, що є набором моделей для комп'ютерного зору, шарів, утиліт, оптимізаторів, планувальників, завантажувачів даних, доповнень, а також сценаріїв навчання/перевірки з можливістю відтворення результатів навчання ImageNet.
- `fastai` [7]. Ця бібліотека спрощує навчання швидких і точних нейронних мереж, використовуючи сучасні передові методи
- `sklearn` [8]. Ця бібліотека містить прості та ефективні інструменти для прогнозного аналізу даних. Доступна кожному. Побудована на NumPy [9], SciPy [10] і matplotlib [11]. З відкритим кодом, має ліцензію BSD;
- тощо.

Також хотілося б зазначити як формувалася рейтингова таблиця учасників. Конкурсанти пишуть код, відправляють його на конкурс і отримують два результати - Public Score та Private Score. Перший з них, Public Score, розраховувався на основі приблизно 25% прихованих тестових даних. Рейтингова таблиця із цим результатом використовувалася лише для того, щоб конкурсанти розуміли, який у них буде приблизно фінальний бал. Другий з них, Private Score, розраховувався на основі приблизно 75% прихованих тестових даних. Рейтингова таблиця із цим результатом відображає фінальні результати. Private Score не відображався до кінця конкурсу. Це прихована оцінка. Також повторно зазначалося, що загальнодоступний бал (Public Score) є лише приблизним показником фінального результату (Private Score). Щодня конкурсанти могли подавати дві роботи на конкурс. Більше того, до закінчення змагань, науковці могли вибрати до двох робіт, які могли бути використані для зарахування результатів до остаточної рейтингової таблиці (та, що формувалася за

прихованим балом Private Score). Якщо конкурсант не обрав дві бажані роботи, які будуть враховані до фінальної рейтингової таблиці, то ці роботи автоматично вибиралися на основі найкращих результатів із загальнодоступної рейтингової таблиці (та, що формувалася за Public Score). У випадку, якщо автоматичний вибір робіт не був здійснений, то інструкції для ручного вибору двох обраних робіт надавалися в правилах конкурсу або в офіційному оголошенні на форумі. Таким чином, ви повинні вибрати матеріали, які, швидше за все, будуть найкращими в цілому, а не обов'язково в загальнодоступній підмножині. Ці результати (Public Score та Private Score) формуються за формулою RMSE, квадратний корінь середньоквадратичної помилки (формула 1.1):

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}, \quad (1.1)$$

де $\hat{y}_i$ - прогнозоване значення, а y_i - реальне значення для кожної сутності i .

Частина робіт, що були у відкритих джерелах під час проведення конкурсу (23.10.2021 - 14.01.2022), використовували за базову модель Swin Transformer чи Hybrid Swin Transformer [12-13]. Адже ця модель була використана в попередньому конкурсі Google Landmark Recognition 2021 [14], автор застосування якої посів перше місце. Наприклад, один із авторів використав таку модель і отримав результат Private Score = 17.31907 та Public Score = 18.32707.

Конкурсанти використовували EfficientNet [15], поєднуючи її із моделями типу Transformer. У роботі [16] автор, що використав саме дану комбінацію моделей, Transformer (архітектура під назвою swin_large_patch4_window12_384) та EfficientNet (архітектура EfficientB0), отримав Private Score = 17.95148 та Public Score = 18.40130.

Дослідник у [17] використав лише Transformer (архітектура swin_large_patch4_window12_384) і отримав такі результати: Private Score = 17.26332 та Public Score = 18.04929.

Автор [18] посилається на роботу [17] та зазначає, що у своїх дослідженнях вони додають попереду вже навчених раніше моделей нові елементи, що називаються RAPIDS SVR. Автор вказує, що техніка, яку він описує, може бути застосована до будь-якої моделі, навченої на картинках чи навіть моделі, що використовується для NLP (Natural language processing, обробки природної мови). Науковець отримав такі результати: Private Score = 17.19873 та Public Score = 17.98651.

Автор [19] також посилається на роботу [17] та додає RAPIDS SVR. Він отримав такі результати: Private Score = 17.19991 та Public Score = 18.01906.

Науковець [20] також розглядав роботу [17] та додавав RAPIDS SVR. Його результати: Private Score = 17.17538 та Public Score = 18.03389.

Також, були розглянуті рішення, що використовують всім відому згорткову модель VGG16 (Keras) [21].

Також використовувалися моделі MobileNetV2 [22] або EfficientNetB2 [15]. Автор [23] використовує саме MobileNetV2 та отримує такі результати: Private Score = 20.53353 та Public Score = 20.53071.

Деякі варіанти розв'язку задачі були комбінаціями мультишарового перцептрону із згортковою нейронною мережею. Дослідник [24] використовує саме поєднання таких двох нейронних мереж та отримує такі результати: Private Score = 20.48686 та Public Score = 20.48295.

Також були розглянуті варіанти, коли конкурсанти створювали просто свої користувацькі моделі нейронних мереж, без використання всім відомих архітектур нейронних мереж.

Хотілося б зазначити, що для перевірки результатів роботи моделі багато хто із дослідників використав метод K-FOLDS [25]. Як відомо, K-Fold Cross Validation – це такий метод, де набір даних розбивається на K секцій, кожна частина якого у певний момент використовується як набір для тестування моделі. Для візуалізації (рис. 1.6), візьмемо 4-кратну перевірку (K=4). Тобто, тут набір даних розділений на 4 частин.

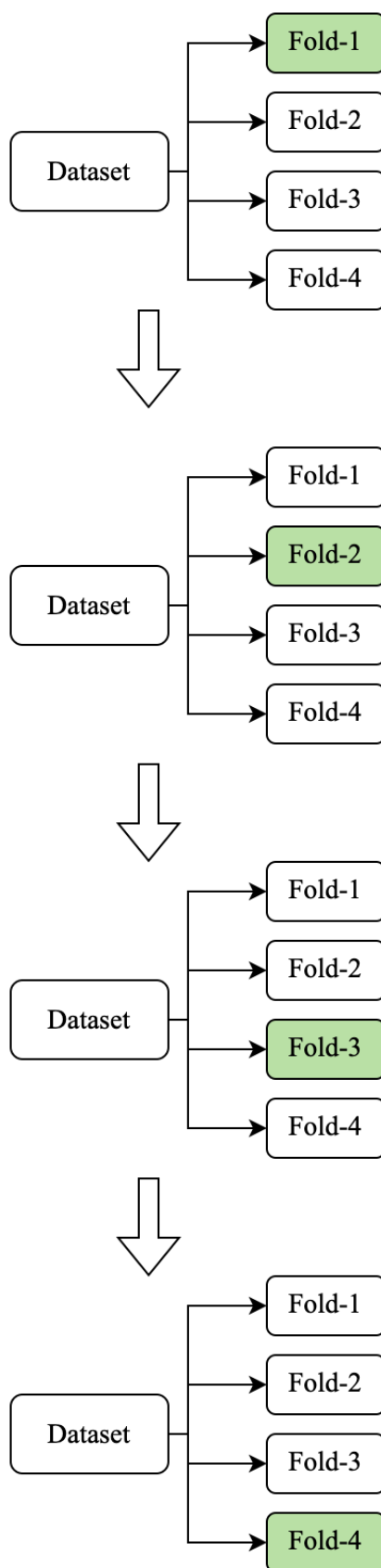


Рисунок 1.6 Метод K-FOLDS

На 1-й ітерації перша секція використовується для перевірки моделі, а решта використовуються для навчання моделі. У 2-й ітерації 2-га секція використовується як набір для тестування, а решта використовується як навчальний набір. Цей процес повторюється до тих пір, поки кожна секція із 4 можливих не буде використана як набір для тестування. Підсумовуючи, хотілося б зазначити, що так як ця задача є новою, то і рішень з відкритих джерел після початку запуску конкурсу існувало не так багато, що свідчить про новизну і про те, що можна працювати над розробкою нових технологій для розв'язання цієї задачі. у цьому напрямку. Проаналізувавши дані відкриті джерела, вирішено провести декілька різних досліджень, щоб провести власні дослідження та покращити існуючі результати.

1.3 Структурні елементи дослідження прогнозування привабливості тварин із притулку

1.3.1 Постановка задачі

Метою даної випускної кваліфікаційної роботи є розробка моделей структурної та параметричної оптимізації нейромережної технології визначення привабливості тварин із притулку.

Мета досягається у розв'язанні таких завдань:

- аналіз релевантної предметної області;
- дослідження принципів технології прогнозування привабливості тварин із притулку;
- розробка архітектури нейромережної моделі, аналіз результатів та вдосконалення отриманих рішень (структурна оптимізація);
- розробка та вдосконалення значень гіперпараметрів моделі, аналіз результатів та вдосконалення отриманих рішень (параметрична оптимізація);
- експериментальна верифікація одержаних результатів;
- розробка взаємодії модулів користувацького інтерфейсу;

- тощо.

Об'єктом дослідження є процеси оцінювання привабливості тварин із притулку з використанням технологій штучного інтелекту.

Предметом дослідження є інтелектуальна технологія визначення привабливості тварин із притулку з використанням нейронних мереж.

Наукова новизна роботи полягає:

- у вдосконаленні методів розпізнавання образів, які лежать в основі створення технологій штучного інтелекту, що використовуються для прогнозування привабливості тварини із притулку;
- у використанні сучасних моделей згорткових нейронних мереж та трансформерів, що раніше не застосовувалися для прогнозування привабливості тварин із притулку;
- у параметричній та структурній оптимізації існуючих моделей нейронних мереж (згорткових та трансформерів), що були у відкритому доступі на час проведення конкурсу PetFinder.my - Pawpularity Contest (23.10.2021-14.01.2022).

І, взагалі, задача прогнозування привабливості тварини за її фотографією була запропонована PetFinder.my лише у вересні 2021 року.

1.3.2 Структурно-логічна схема проведення дослідження

Перейдемо до опису структурно-логічної схеми проведення дослідження (рис. 1.7). На ній зображено основні етапи розробки інтелектуальної технології для визначення привабливості тварин із притулку.

Спочатку, на кроці (1) проводиться аналіз існуючої проблеми прогнозування привабливості тварин із притулку, цей пункт вже був описаний вище. На цьому етапі розглядається актуальність проблеми та чому важливо вирішити це питання. На кроці (2) проводиться огляд попередніх досліджень, цей пункт також вже був описаний вище. Тут розглядаються основні методи та способи, за допомогою яких можливо робити прогнозування і визначення привабливості тварин із притулку.

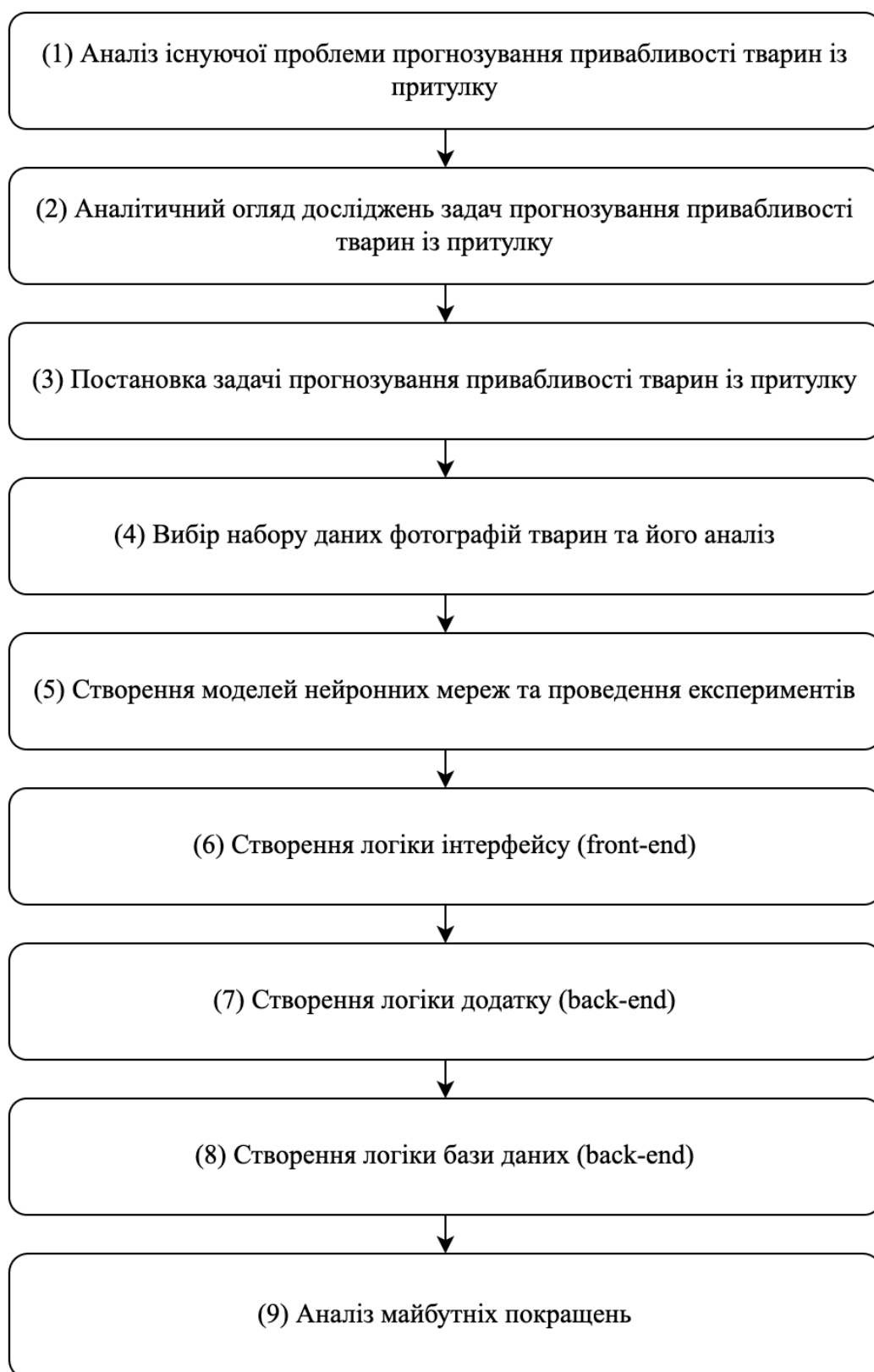


Рисунок 1.7 Структурно-логічна схема дослідження

Наступний етап - постановка задачі (3) прогнозування привабливості тварин із притулку, що також була описана вище. Визначаються цілі для випускної кваліфікаційної роботи, основні завдання, описується мета, об'єкт, предмет дослідження тощо. Далі обирається набір даних (4), на яких буде проведено навчання штучної нейронної мережі (НМ). Очевидно, що для даного завдання набір даних буде складатися із фотографій тварин із притулку. Після цього виконується власне навчання і тестування результатів НМ (5). На цьому етапі відбувається також вибір найкращої структури нейромережі, адже необхідно спробувати різні архітектури, щоб досягнути найкращих результатів. Тобто, проводяться експерименти. Наступний (6) етап - це створення логіки інтерфейсу, front-end частина. Ця частина необхідна для взаємодії із користувачами, для візуального оформлення та кращого сприйняття інформації. Після цього створюється логіка додатку (7), або ж ця частина ще називається back-end. Цей блок включає в себе серверну частину взаємодії користувача із системою. Також, тут підключається модель нейронної мережі, що була навчена на етапі (5). Також, back-end включає в себе і створення логіки бази даних (8). Тут створюється взаємодія користувача із базою даних, що містить додаткову інформацію. І, фінальний етап - це аналіз майбутніх покращень (9): тут описується те, що можна додатково зробити і вдосконалити. Тобто, бачимо, що на кожному етапі відбувається важливий крок, без якого не можливо зробити наступний.

1.3.3 Набір даних

Як вже і описано вище, навчання нейромережі не можливе без набору даних. Тут використовується набір зображень, що був запропонований авторами конкурсу PetFinder. Розглянемо його структуру детальніше. Даний набір складається із двох частин: зображень (рис. 1.8) та метаданих (рис. 1.9). Він містить 9912 фотографій різних тварин, переважно котів та собак. А метадані містять додаткову інформацію про кожне зображення. Всі зображення різного

розміру та різної орієнтації: як пейзажної, так і портретної.

Дані, що створені авторами змагань для навчання моделей штучного інтелекту:

- train/ – папка, що містить фотографії навчального набору у формі {id}.jpg, де {id} – унікальний ідентифікатор профілю домашньої тварини.
- train.csv – метадані (детальніше описані нижче) для кожної фотографії в навчальному наборі, а також цільове значення, Rawpularity Score фотографії. У стовпці "Ідентифікатор" вказано унікальний ідентифікатор профілю домашньої тварини, що відповідає назві файлу фотографії.

На додаток до даних для навчання, автори змагання включають деякі випадково згенеровані зашумлені приклади тестових даних (рис. 1.10), щоб допомогти учасникам створити правильний код подання результатів навчання мережі.

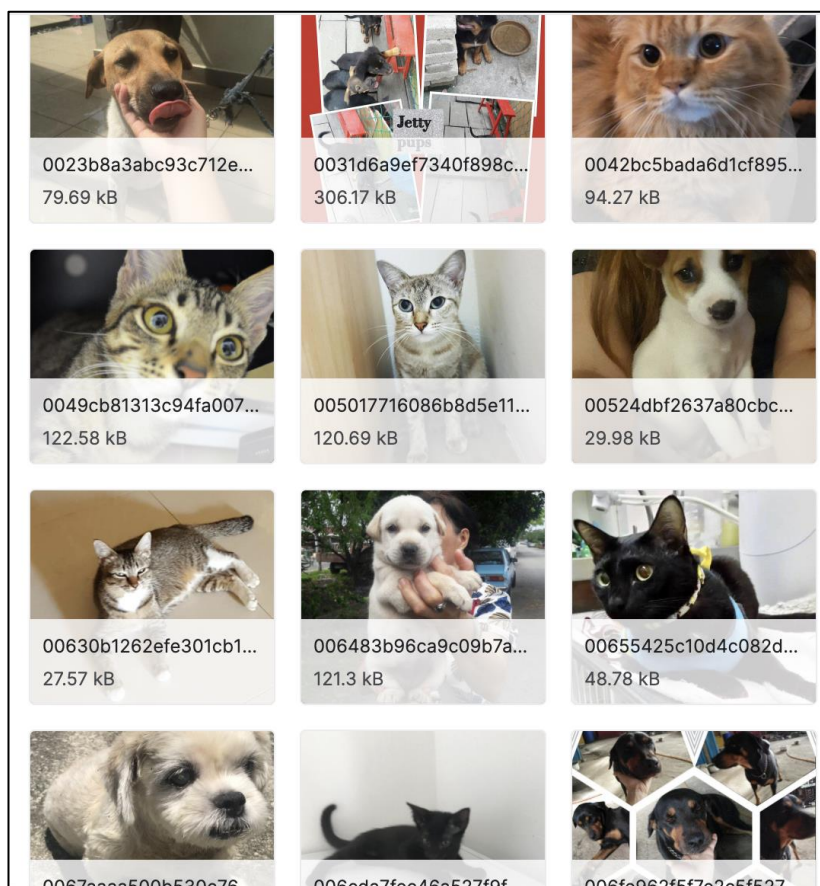


Рисунок 1.8 Різні зображення із набору даних

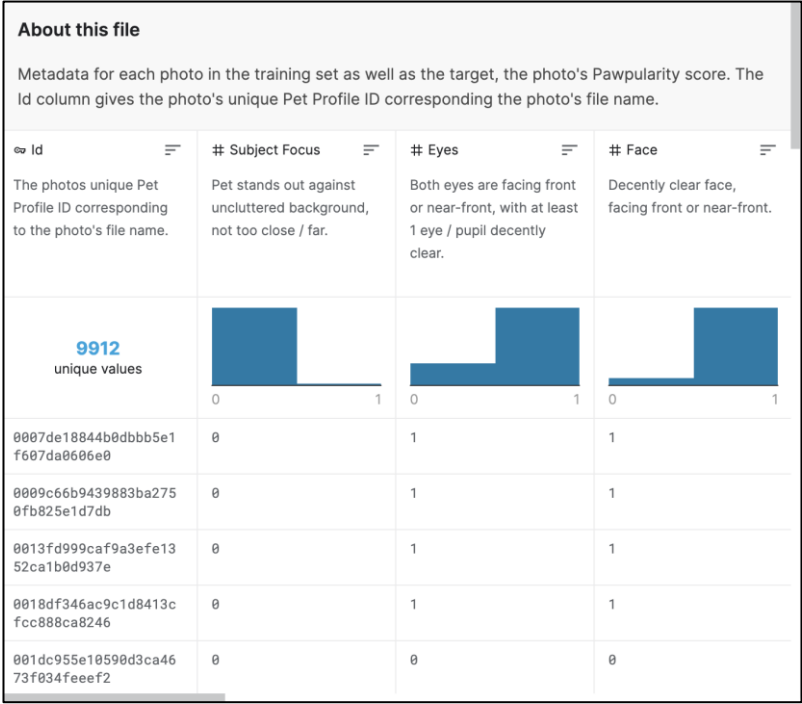


Рисунок 1.9 Метадані для кожного зображення

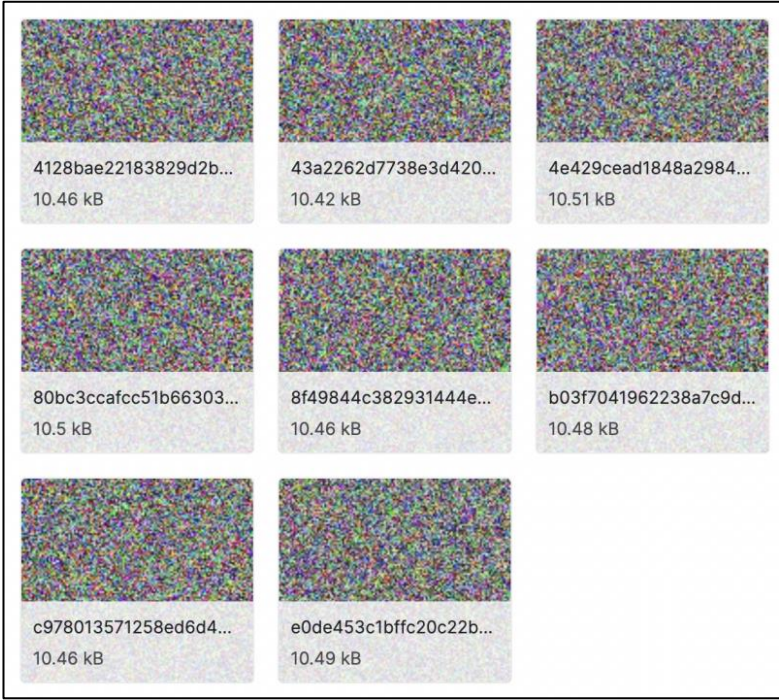


Рисунок 1.10 Випадково згенеровані зашумлені приклади тестових даних

А вже коли ваш надісланий код буде оцінено, ці дані прикладу буде замінено фактичними даними тесту (включаючи поданий зразок).

- test/ - папка, що містить випадково згенеровані зображення у форматі, схожому на фотографії навчального набору. А вже фактичні дані для тестування результатів роботи нейромережі містять близько 6800 фотографій домашніх тварин, подібних до фотографій тренувального набору. Необхідно зауважити, що ці фактичні дані є прихованими, їх бачать лише творці.
- test.csv – випадково згенеровані метадані, схожі на метадані навчального набору.
- sample_submission.csv – Зразок файлу для подання у правильному форматі (рис. 1.11).

Ці метадані були зібрані вручну авторами змагань, вони не використовуються для визначення значення Pawpularity, але вони можуть бути корисними для кращого розуміння вмісту та співвідношення їх із привабливістю фотографії. Також, метадані містять власне Pawpularity Score (оцінку привабливості тварини), яке є цільовим значенням.

Id	Pawpularity
Pet IDs for test set photos	The predicted Pawpularity value
8 unique values	8 total values
4128bae22183829d2b5f ea10effdb0c3	67.75
43a2262d7738e3d420d4 53815151079e	59.15
4e429cead1848a298432 a0acad014c9d	20.02
80bc3ccafcc51b66303c 2c263aa38486	94.53
8f49844c382931444e68 dffbe20228f4	89.82
b03f7041962238a7c9d6 537e22f9b017	65.50
c978013571258ed6d463 7f6e8cc9d6a3	71.42
e0de453c1bffc20c22b0 72b34b54e50f	5.85

Рисунок 1.11 Зразок подання файлу із результатами у правильному форматі

Зазначимо, що Pawpularity Score у наборі даних подається як ціле число. Але у прикладі подання файлу із результатами у правильному форматі (рис. 1.11) бачимо, що Pawpularity Score може також бути і дійсним числом.

Що ж входить до метаданих, які значення і поля туди включені авторами змагань? Розглянемо детальніше. Як вже описано вище, файли `train.csv` і `test.csv` містять метадані для фотографій у навчальному наборі та тестовому наборі відповідно. Кожна фотографія домашньої тварини позначена значенням 1 (Так) або 0 (Ні) для кожної з наступних ознак:

- Focus - домашня тварина виділяється на незаміщеному тлі, не дуже близько/далеко.
- Eyes – обидва ока звернені вперед або ближче до передньої частини, принаймні 1 око/зіниця досить чисті.
- Face - пристойно чітке обличчя, звернене спереду або близько спереду.
- Near – один домашня тварина займає значну частину фотографії (приблизно понад 50% ширини або висоти фотографії).
- Action – тварина в середині дії (наприклад, стрибки).
- Accessory – супровідний фізичний або цифровий аксесуар/реквізит (тобто іграшка, цифрова наклейка), за винятком нашійника та повідця.
- Group - на фото більше 1 вихованця.
- Collage – фотографія, відретушована в цифровому вигляді (тобто з цифровою фоторамкою, комбінація кількох фотографій).
- Human - людина на фото.
- Occlusion – специфічні небажані предмети, що блокують частину тварини (наприклад, людину, клітку або паркан). Зауважте, що не всі блокуючі об'єкти вважаються оклюзією.
- Info – спеціально доданий текст або мітки (наприклад, ім'я тварини, опис).
- Blur - помітно розфокусовані або шумні, особливо для очей і обличчя вихованця. Для записів розмиття стовпець «Eyes» завжди має значення 0.

То як же ж вираховано Pawpularity Score? Показник Pawpularity визначається зі статистики переглядів сторінок кожного профілю домашньої тварини на сторінках зі списками, за допомогою алгоритму, який нормалізує дані про трафік на різних сторінках, платформах (веб- та мобільних) і базується на різних показниках. Подвійні кліки, звернення до ботів і спонсоруючі профілі вилучаються із аналізу.

1.4 Висновки до розділу 1

У першому розділі розглянуто аспекти вирішення проблем привабливості тварин із притулку. Взагалі, задача прогнозування взаємодію користувача з профілем домашньої тварини на основі фотографії для цього профілю була запропонована PetFinder.my у вересні 2021 року. Наразі PetFinder.my використовує базовий вимірювач Cuteness Meter для оцінювання фотографій домашніх тварин. Він аналізує композицію зображення та інші фактори порівняно з продуктивністю тисяч профілів домашніх тварин. Були проаналізовані та описані вище роботи конкурсантів із відкритих джерел. Розглянуті рішення дозволяють зробити висновки, що ці роботи можна покращити, наприклад, змінивши гіперпараметри нейронних мереж чи навіть обрати іншу архітектуру мережі. Із досліджених робіт та їх результатів Public Score та Private Score видно, що всім відомі архітектури нейронних мереж, наприклад, такі як MobileNetV2 чи комбінації багатошарового перцептронну із користувацькою згортковою нейронною мережею, дають гірші результати у порівнянні із нейронними мережами типу Transformer. Проаналізовані результати із відкритих джерел свідчать про те, що задачу прогнозування привабливості тварини за фото краще вирішувати використовуючи нейронну мережу типу Transformer. Поставлена та описана мета дослідження, задачі, предмет, об'єкт дослідження. Набір даних складається із двох частин: зображень та метаданих. Він містить 9912 фотографій різних тварин, переважно котів та собак. А метадані містять додаткову інформацію про кожне зображення.

РОЗДІЛ 2 ПРОЕКТУВАННЯ ІНТЕЛЕКТУАЛЬНОЇ ТЕХНОЛОГІЇ ПРОГНОЗУВАННЯ ПРИВАБЛИВОСТІ ТВАРИН ІЗ ПРИТУЛКУ

2.1 Проектування архітектури технології інтелектуальної технології прогнозування привабливості тварин із притулку

2.1.1 Діаграми потоків даних

Проблеми автоматизації проектування інформаційних систем (ІС) спонукали дослідників розробити програмно-технологічні засоби спеціального типу – CASE-засоби (повна назва - Computer Aided Software Engineering). Така CASE-технологія репрезентує деяку множину методів проектування допомагають моделювати дану предметну область (ПрО), аналізувати модель під час стадій розробки та супроводження ПЗ відповідно до інформаційних потреб користувачів [26].

Методологія CASE-технологій базується на так званому спадному підході і дає змогу контролювати всі етапи життєвого циклу ІС. Ідея спадного підходу зводиться до того, що під час побудови системи її складові деталізуються все глибше та глибше [26].

Сьогочасні CASE-системи належать до структурних чи об'єктно-орієнтованих [26]. Розглянемо саме структурні CASE-системи.

Відомо, що саме у структурному підході для проектування та аналізу системи використовуються такі різновиди моделей як [26]:

- Перший тип з них - це DFD, діаграми потоків даних;
- Наступні - SADT, метод структурного аналізу і проектування. До них належать моделі і відповідні функціональні діаграми;
- Також є ERD, діаграми "суть–зв'язок".

Відомо, що DFD є одним із основних засобів моделювання та побудови функціональних вимог для системи. Такі вимоги зображуються як ієрархія функціональних компонентів, що зв'язані різними користувацькими потоками даних (рис. 2.1) [26].

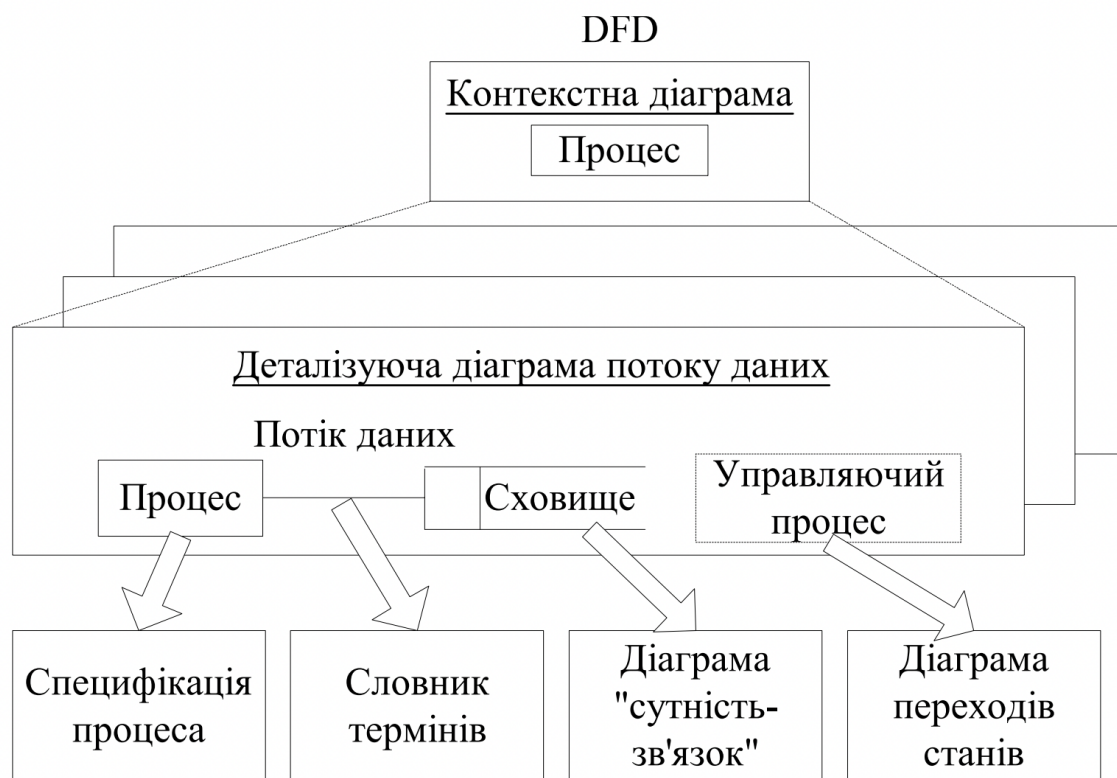


Рисунок 2.1 Елементи DFD [26]

У програмній інженерії можна побудувати DFD для представлення різних рівнів абстракції. DFD вищого рівня можуть бути декомпозовані на нижчі рівні, розкриваючи більше інформації та функціональних елементів [27]. DFD 0 рівня, що також відома як контекстна діаграма, розроблена як абстрактне представлення, показуючи систему як єдиний процес з її зв'язком із зовнішніми сутностями. Вона представляє всю систему у вигляді єдиного цілого з вхідними та вихідними даними, зазначеними вхідними та вихідними стрілками [27]. В DFD 1 рівня контекстна діаграма розбивається на декілька підпроцесів. На цьому рівні виділяються основні функції системи та розбиваємо високорівневий процес DFD 0 рівня на підпроцеси [27].

Традиційно, окрім DFD, також застосовують такі методики як IDEF0 і IDEF3 для того, щоб виконати функціональне структурне і потокове моделювання [27].

Побудуємо контекстну діаграму потоків даних (рис. 2.2). Діаграми потоків даних виконано у середовищі ERwin Process Modeler, із використанням нотації Гейна–Сарсона (Gane-Sarson).

Ця нотація була введена Крісом Гейном і Тріш Сарсоном [28]. Нотація Гейна-Сарсона використовується для діаграм потоків даних. Виділимо такі компоненти для контекстної діаграми потоків даних:

- Зовнішня сутність: користувач;
- Процес: прогнозування привабливості тварини із притулку;
- Потоки даних:
 - зображення тварини;
 - додаткова інформація про тварину;
 - результат прогнозування - оцінка привабливості тварини за фото;
 - історія випадків прогнозування привабливості тварин із притулку за фото;
 - інформація про наявних тварин у притулку.

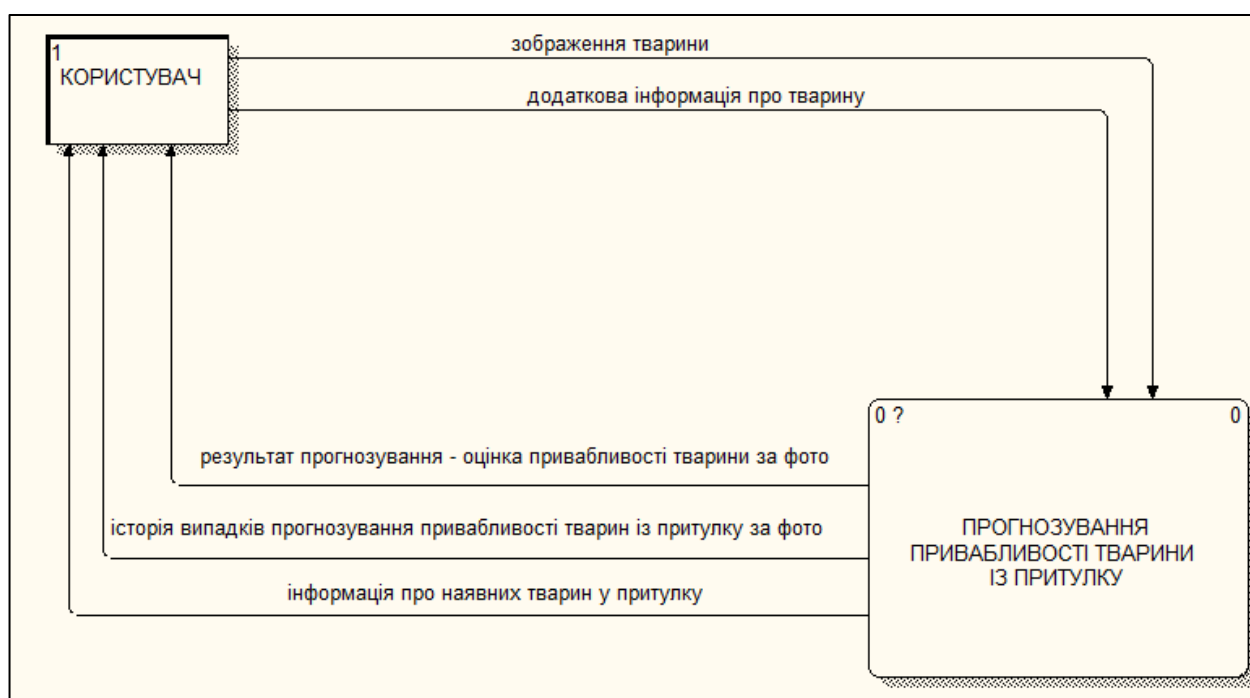


Рисунок 2.2 Діаграма потоків даних

Для того, щоб побудувати діаграму потоків даних 1 рівня (рисунок 2.3), потрібно її декомпонувати на менші частини. Тоді отримаємо такі складові:

- Підпроцеси:
 - опрацювання вхідної інформації;
 - прогнозування привабливості тварин із притулку;
 - формування звітності.
- Накопичувачі:
 - модель НМ, що складається із моделі нейронної мережі для прогнозування привабливості тварини із притулку за фото;
 - БД тварин із притулку, що містить інформацію про наявних тварин у притулку;
 - БД історії випадків прогнозування привабливості тварин із притулку, що містить відповідні записи.
- Потоки даних:
 - зображення тварини;
 - додаткова інформація про тварину;
 - запит про модель НМ;
 - дані про модель НМ;
 - запит про тварин із притулку;
 - звіт про тварин із притулку;
 - запис випадку прогнозування привабливості тварини із притулку;
 - результат прогнозування привабливості тварини із притулку за фото;
 - запит про історію прогнозування привабливості тварин із притулку;
 - звіт про історію випадків прогнозування привабливості тварин із притулку;
 - результат прогнозування - оцінка привабливості тварини за фото;
 - історія випадків прогнозування привабливості тварин із притулку за фото;
 - інформація про наявних тварин у притулку.

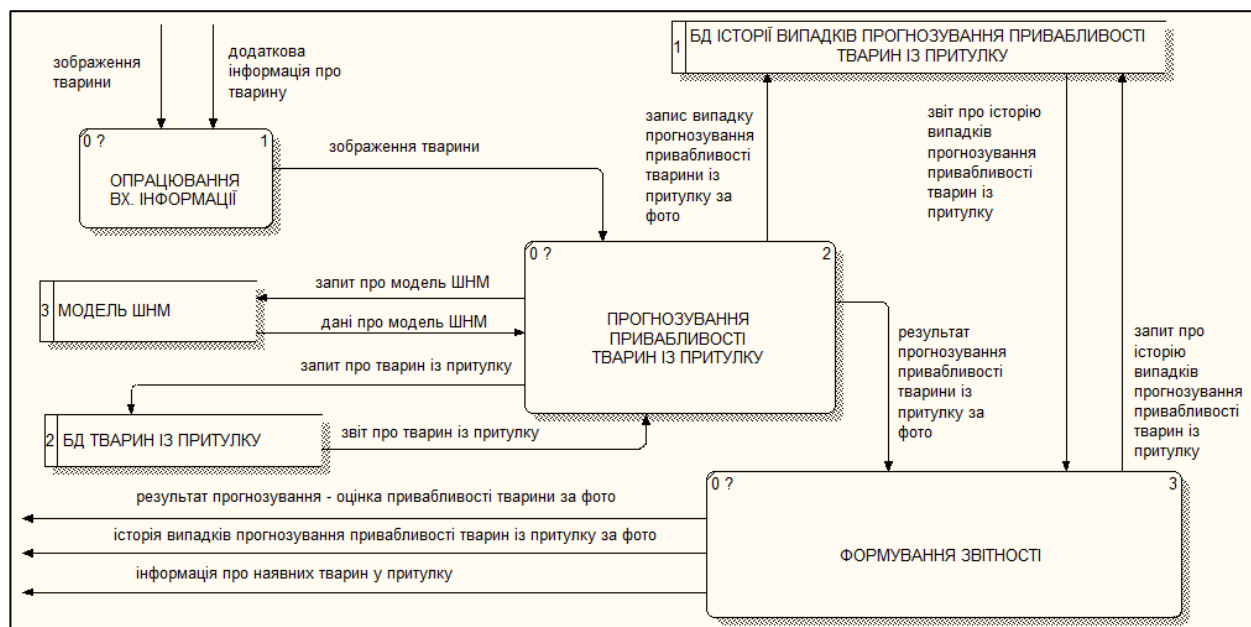


Рисунок 2.3 Діаграма потоків даних першого рівня

2.1.2 Дерево функцій інформаційної технології

Структурна діаграма (рис. 2.4), або, як ще вона називається, дерево функцій інформаційної технології, репрезентує певну ієрархічну структуру технології. Ця діаграма показує основні функції технології, та складові цих функцій. Виділимо основні функції для технології прогнозування привабливості тварини із притулку за фото:

- Опрацювання вхідної інформації
 - отримання зображення;
 - збереження зображення;
- Прогнозування привабливості тварини із притулку за фото:
 - обробка зображення;
 - прогнозування оцінки привабливості;
 - видалення зображення;
 - ведення БД історії випадків.
- Формування звітності:
 - формування звіту про прогнозування привабливості тварини;

- формування звіту про наявних тварин із притулку;
- формування звіту про історію випадків;



Рисунок 2.4 Дерево функцій інформаційної технології

2.2 Нейромережні моделі для прогнозування привабливості тварин із притулку

2.2.1 Нейронні мережі для задач розпізнавання образів

Розпізнавання образів може здійснюватися різними технологіями. Все залежить від того, які самі образи. Для прогнозування привабливості тварини із притулку за фото було застосовано розпізнавання образів із використанням методів штучного інтелекту.

Взагалі, розпізнавання образів (РО) – це автоматичне розпізнавання образів і шаблонів у даних. РО застосовується, наприклад, в статистичному аналізі даних, обробці сигналів, аналізі зображень, пошуку інформації, біоінформатиці, стисканні даних, комп'ютерній графіці та у машинному навчанні. Розпізнавання образів бере початок у статистиці та інженерії; деякі сучасні підходи до РО включають використання машинного навчання через збільшення доступності великих даних і нового великого обсягу обчислювальної потужності. Ці види діяльності можна розглядати як два аспекти однієї сфери застосування, і вони зазнали значного розвитку за останні кілька десятиліть [29]. До прикладів задач розпізнавання образів можна віднести наступні:

- розпізнавання зображень природи;
- розпізнавання букв та цифр;
- розпізнавання автомобільних номерів та автомобільних знаків на дорогах;
- розпізнавання штрих-кодів на товарах в магазині;
- розпізнавання людей за обличчям чи скануванням сітківки ока;
- розпізнавання мови: української, англійської тощо.

На рис. 2.5 наведено класифікацію методів розпізнавання образів. На ній представлено різновиди систем розпізнавання, їх таксономічний поділ.

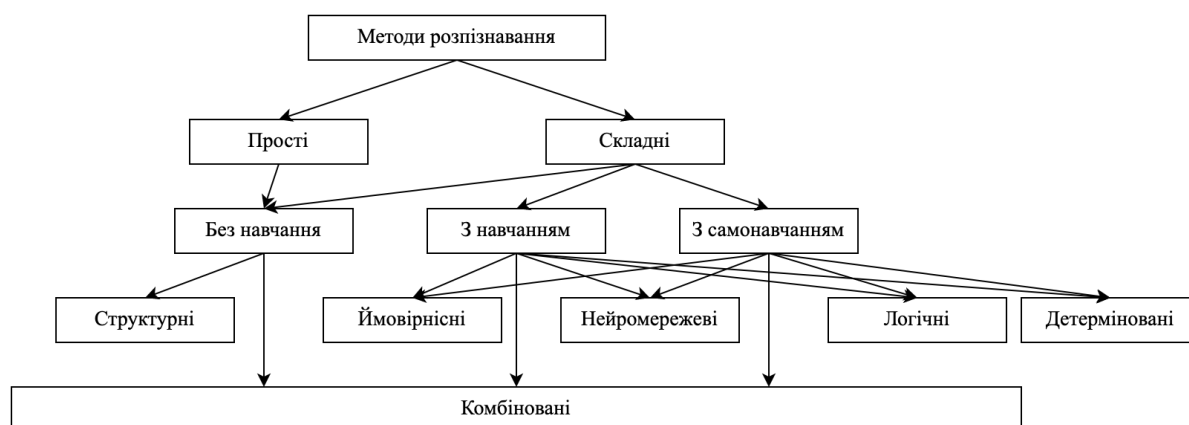


Рисунок 2.5 Класифікація методів розпізнавання образів

Також, для прикладу, наведемо декілька напрямків у розпізнаванні образів. До них можна віднести наступні:

- зіставлення зображення із деяким еталоном;
- розпізнавання зображення, за характерними точками. При цьому спосіб отримання цих точок може відрізнятися.
- розпізнавання за допомогою штучних нейронних мереж.

Для розв'язання даної задачі було обрано саме нейронні мережі як апарат для розпізнавання образів та прогнозування привабливості тварини із притулку за фото. Розглянемо їх детальніше.

Штучна нейронна мережа (НМ) – це деяка математична модель, а також її апаратна та програмна розробка, яка базується на ідеї функціонування справжніх нейронних мереж, тобто мереж нервових клітин, що є в живому організмі.

Штучні нейронні мережі, які ще називають штучними нейронними системами, чи нейрокомп'ютерами, або паралельно розподіленими процесорами, або моделями зв'язків, є спробою хоча б частково змодельовати структуру і функції мозку та нейронних систем живих створінь.

НМ є інформаційною, або такою, що обробляє сигнали, системою, яка складається із великою кількістю простих обробляючих елементів. Такі елементи називаються штучними нейронами, або просто вузлами. Вони взаємно з'єднуються за допомогою ланок або зв'язків і кооперуються для виконання паралельно розподіленого оброблення сигналів для розв'язання певної обчислювальної задачі. Нейронні мережі здатні адаптуватися до умов середовища зміною сили зв'язків або своєї структури [30].

Нейронні мережі, як відомо, гарно вирішують важкоформалізовані та слабко структуровані задачі. Також, НМ використовуються при вирішенні проблем, алгоритми вирішення яких не можуть бути точно сформульовані.

Далі розглянемо, чому нейронні мережі так часто застосовуються для задач розпізнавання образів.

- Навчання. Штучні нейронні мережі можуть змінювати вихідні результати в залежності від вхідного набору даних. НМ самоналаштовуються для забезпечення необхідної реакції. Як наслідок - властивість універсальності.
- Узагальнення. Після того, як мережа навчилася, вона може не відчувати невеликі зміни нових вхідних сигналів. Ця внутрішня властивість бачити даний образ крізь деякий шум і спотворення необхідний для розпізнавання образів в реальному світі.
- Абстрагування. Деякі з штучних нейронних мереж мають здатність аналізувати та визначати ознаки у вхідних сигналах. Наприклад, мережа може бути навчена розпізнавати зашумлені букви "М". Після відповідного навчання,

коли користувач подасть на вхід нейронної мережі цю зашумлену букву, то це приведе до того, що мережа зможе розпізнати і у результаті користувач отримає букву досконалої форми. У деякому розумінні вона вчиться продукувати нові образи.

- Паралельність обробки інформації та розподілені обчислення. Кожен вихід нейрона формується тільки на основі своїх входів і власного внутрішнього стану.
- Висока надійність. Вихід із ладу одного нейрону не впливає на систему так, як у послідовних ЕОМ, що базуються на принципах теорії Неймана, де збій однієї комірки пам'яті чи одного вузла в апаратурі призводить до краху системи.

Нейронні мережі застосовуються для розв'язання наступних задач:

- обробка і аналіз зображень;
- обробка високошвидкісних цифрових потоків;
- задачі криптографії;
- класифікація інформації в реальному часі;
- розпізнавання мови незалежно від диктора;
- вирішення трудомістких задач оптимізації;
- планування, застосування сил і засобів у великих масштабах;
- автоматизована система швидкого пошуку інформації;
- адаптивне управління і прогнозування;
- тощо.

2.2.2 Математичні засади нейронних мереж

Штучні нейрони, які відомі як нейронні клітини, моделюють структуру та деякі функції реальних біологічних нейронів живого організму. Будова та властивості штучних нейронних мереж, що утворюються із нейронів, споруджуються відповідно до тих завдань, що необхідно вирішити за допомогою нейромереж [31]. Штучні нейрони моделюють структуру та функції біологічних нейронів. Розглянемо одну з найвідоміших моделей штучних

нейронів як приклад. Базовий штучний нейрон можна моделювати як багатовходовий нелінійний пристрій із зваженими взаємозв'язками w_{ij} - синаптичними вагами [30]. Тіло клітини (сома) подається нелінійною, так званою обмежувальною, пороговою або активаційною функцією $\psi(u_j)$. У найпростішій моделі штучного нейрона підсумовується n зважених вхідних сигналів і передає одержаний результат через нелінійність на вихід за таким рівнянням [30]:

$$y_j = \psi\left(\sum_{i=1}^n w_{ij}x_i + \theta_j\right), \quad (2.1)$$

де θ_j ($\theta_j \in \mathbb{R}$) - зовнішній поріг, який ще називається зсувом або зміщенням; x_j - вхідні сигнали ($j = \overline{1, n}$); n - кількість вхідних сигналів; y_j - вихідні сигнали [30]. Штучний нейрон характеризується своєю нелінійністю і порогом θ_j . А також, вихідний сигнал, як правило, визначається монотонно зростаючою сигмоїдною функцією зваженої суми вхідних сигналів. Її можна математично описати як

$$y_j = \tanh \gamma u_j = \frac{1 - e^{-2\gamma u_j}}{1 + e^{-2\gamma u_j}} \quad (2.2)$$

для симетричного подання або

$$y_j = \frac{1}{1 + e^{-\gamma u_j}} \quad (2.3)$$

для несиметричного уніполярного подання за

$$u_j = \sum_{i=1}^n w_{ij} x_i, \quad (2.4)$$

де γ - позитивна константа або змінна, яка контролює «ступінчатість» (нахил) сигмоїдної функції [30]. На рисунку 2.6 представлено спрощену функціональну модель клітини штучного базового нейрона [30].



Рисунок 2.6 Модель клітини штучного базового нейрона [30]

До переваг нейронних мереж можна віднести наступні:

- **Нелінійність.** Відомо, що штучні нейрони можуть бути нелінійними та лінійними. НМ, які зроблені із нелінійних нейронів, також є нелінійними. Нелінійність - важлива властивість, особливо тоді, коли механізм, що відповідає за побудову вхідного сигналу, також є нелінійним. Наприклад, сюди можна віднести природну мову [32].
- **Перетворення даної вхідної інформації у вихідну інформацію.** Навчання з учителем - один із відомих методів навчання нейронних мереж. Суть полягає у тому, що ваги змінюються відповідно до розмічених навчальних прикладів. Приклади, що подаються на нейронну мережу, складаються з певного вхідного сигналу і бажаного результату на виході [32].
- **Адаптивність.** Відомо, що НМ можуть адаптувати свої ваги, відповідно до змін зовнішнього середовища. Наприклад, штучні нейронні мережі, які були

навчені працювати з деяким типом задач, можуть бути перенавчені для роботи з іншим типом задач [32].

- тощо.

Як вже було написано раніше, для розв'язання задачі прогнозування привабливості тварини із притулку будуть використовуватися нейронні мережі. Наприклад, до них можна віднести наступні:

- багат шаровий перцептрон (MLP) [32];
- рекурентні нейронні мережі (LSTM [33], мережа Хопфілда [34], мережа Хемінга [35] тощо);
- генеративні змагальні мережі (GAN) [36];
- згорткові нейронні мережі (LeNet [37], AlexNet [38], VGG16 [21], Xception [39] тощо);
- трансформери (GPT-2 [40], GPT-3 [41], BERT [42] тощо);
- тощо.

Попередній аналіз численних наукових джерел і висновків їх авторитетних авторів свідчить про переваги згорткових нейронних мереж та трансформерів для розв'язання таких задач як класифікація та регресія зображень, тому далі розглянемо саме ці дві категорії нейронних мереж.

2.2.3 Аналіз згорткових нейронних мереж

Згорткові нейронні мережі дуже часто застосовуються саме для вирішення проблеми розпізнавання образів із зображень. Вони досягають значних переваг над іншими архітектурами, забезпечуючи максимальну продуктивність та ефективність. Ці мережі використовуються у різних сферах та вирішують такі важливі завдання як розпізнавання облич, аналіз документів, аналіз клімату, розпізнавання різних зображень та ідентифікація об'єктів тощо [43].

Відомо, що згорткові нейронні мережі називаються так, бо мають в собі операцію згортки. Взагалі, згортка – це результат процесу обчислення нового

значення певного пікселя, при врахуванні значень пікселів, що його оточують. Для проведення даних розрахунків використовують матрицю, яку називають ядром згортки. Як правило, ядро згортки має розмір $a \times a$, де a - непарне число. Часом зустрічаються випадки, коли матриця згортки є прямокутною. Традиційно, що популярними ядрами згортки називають такими, що використовуються у наступних цілях: розмиття, підвищення чіткості тощо (рисунок 2.7).

Розмиття	Виділення границь	Підвищення чіткості
1/9	0	0
1/9	-1	-1
1/9	4	5
1/9	-1	-1
1/9	0	0

Рисунок 2.7 Ядра згортки

Автор статті [44] зазначає, що операція згортки має властивість - інваріантність щодо перенесення. Це вказує на те, що кожен згортковий фільтр відображає деяку множину властивостей. Використання алгоритму згорткової нейронної мережі дозволяє визначити, який набір властивостей відповідає цілому зображенню. Тобто, інтенсивність вихідного сигналу залежить не від порядку розташування властивостей, а від їх наявності. Однією із особливостей роботи згорткових мереж можна назвати застосування так званих карт ознак. Зображення сканується за допомогою вікна квадратного розміру. І тоді на вихідній сітці отримуються великі результуючі значення там, де є необхідні поєднання пікселів, та маленькі там, де знаходиться інформація зовсім не та, що шукається. Така сітка (решітка) називається картою ознак, про яку вже було

написано раніше. Також знаходиться не одна якась певна ознака, а декілька різних. Для кожної із таких знак є певне своє вікно, що містить свій набір вагових коефіцієнтів. Тому, у результаті, на виході отримується декілька карт ознак, що були знайдені у підсумку. Кожна карта ознак (вже як зображення) відправляється на вхід наступного шару розпізнавача і також отримує на виході ще кілька карт ознак. На рисунку 2.8 зображено схематично карта ознак, ядро згортання і наступна карта ознак.

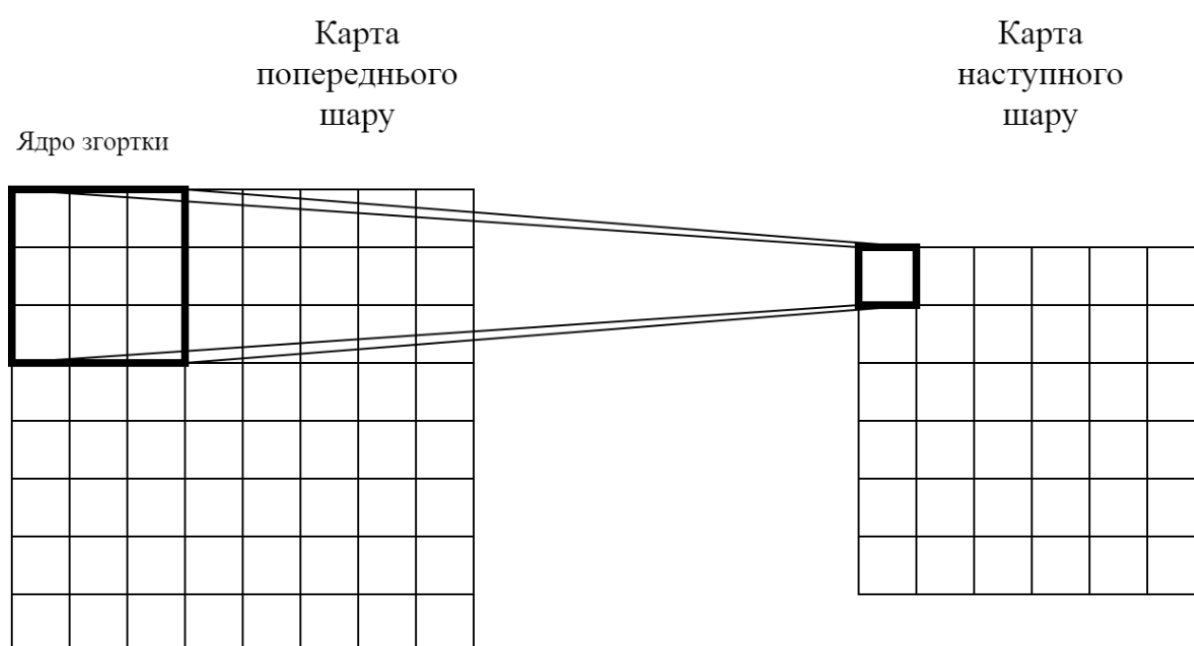


Рисунок 2.8 Згортання карти ознак у меншу карту

Також хотілося б зазначити, що у згортковій нейронній мережі також присутня операція пулінгу. Так як кількість необхідних потужних обчислювальних ресурсів починає експоненціально зростати разом зі збільшенням кількості всіх властивостей, що досліджуються, то для вирішення цієї проблеми була розроблена методика пулінга, який використовується на шарі підвибірки або, як його іще називають, шарі пулінга чи субдискретизації. Процес пулінгу простий - якщо деяка область зображення містить яскраво виражені певні властивості, то можливо відмовитися від пошуку властивостей у даній

області дослідження. За допомогою даної операції робиться не лише економія зайнятої пам'яті чи обчислювальної потужності, але і дозволяється очистити зображення від різноманітного шуму. Наприклад, за фільтр можна взяти матрицю, розміром 2×2 , що застосовується із кроком рівним 2.

Також існують різні варіанти нелінійних функцій, що використовуються для операції пулінгу, серед яких найбільш популярна конструкція максимального пулінгу, яка для кожного регіону знаходить і повертає максимальне значення (рисунки 2.9).

Як і було сказано вище, головна ідея полягає у тому, що точне розташування певної ознаки у просторі не так важливе, як її положення щодо інших ознак.



Рисунок 2.9 Операція максимального пулінгу

Відомо, що до переваг згорткових мереж можна віднести наступні:

- один із найкращих відомих алгоритмів розпізнавання зображень;
- має меншу кількість ваг, що треба налаштовувати, у порівнянні із повнозв'язною мережею. Завдяки цьому інформація узагальнюється і спрощується, а не зберігається для кожного пікселя окремо;
- інваріантність до зсуву та повороту картинки;

- зручне розпаралелення обчислень, що дозволяє реалізувати алгоритм на графічних процесорах.

Також, можна виділити наступні недоліки:

- велика кількість параметрів мережі, що треба налаштовувати;
- не завжди зрозуміло, як правильно підібрати параметри для кожної задачі, що потрібно вирішити;
- налаштування параметрів мережі істотно впливають на результат;
- існують різні можливі конфігурації згорткової мережі, але нема рекомендацій, яку все таки краще обрати для задачі, що вирішується.

До відомих прикладів згорткових нейронних мереж можна віднести наступні:

1. Неокогнітрон. Неокогнітрон вважають попередником всіх згорткових нейронних мереж. Це багат шарова нейронна мережа, що була вперше представлена у 1979-1980 роках японським вченим Кунікіхо Фукусімою [45-46]. Дана мережа використовувалася для розпізнавання рукописних символів і не тільки. Результати використання неокогнітрона у майбутньому надихнули вчених на створення згорткових нейронних мереж.
2. Згорткова мережа LeNet. Така згорткова 7-рівнева штучна нейронна мережа, що називається LeNet [47], вперше була запропонована Яном Лекуном у 1998 році. Вона є однією із перших ЗНМ, що була створена після багатьох спроб і різноманітних досліджень, що відбувалися, починаючи з 1988 року. Спочатку Ян Лекун у 1989 році вперше застосував метод зворотного поширення помилки для навчання згорткового ядра для таких цілей, як розпізнавання рукописних цифр. Таке навчання було повністю автоматизоване і було кращим за ручне налаштування коефіцієнтів. Згодом, це дослідження було успішно застосоване до розпізнавання ZIP-кодів – системи поштових індексів, що застосовується поштою США і послужило прототипом майбутній мережі LeNet. І вже у 1998 році науковці [47] представили різні методи розпізнавання рукописних цифр. Вони проводили дослідження і порівнювали знайдені моделі.

Ці дослідження мали величезний успіх і зібрали багато зацікавлених вчених навколо. Хоча, на даний момент, дана будова мережі не має найкращі результати точності прогнозування результатів, вона стала початковою точкою для багатьох майбутніх архітектур нейронних мереж, надавши цьому полю досліджень великий науковий інтерес. На рисунку 2.10 представлена повна будова даної мережі, що була опублікована Яном Лекуном в його науковій роботі [47].

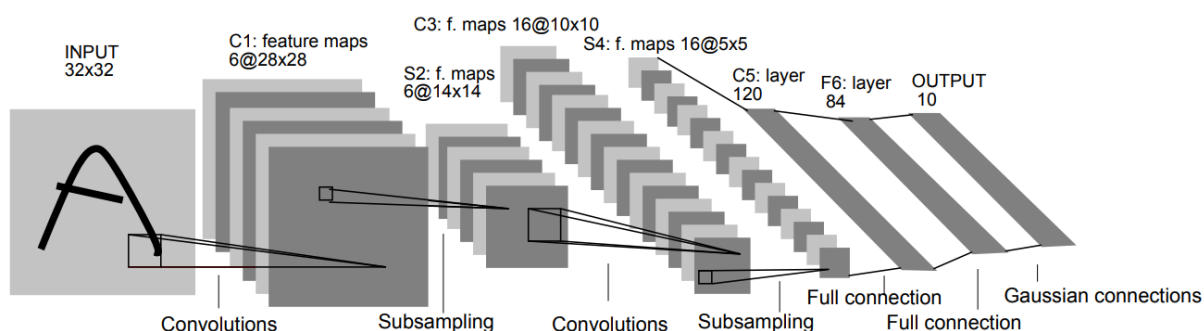


Рисунок 2.10 Будова мережі LeNet [47]

3. AlexNet. У період із 1998 по 2010 роки розвиток згорткових мереж значно сповільнився. Більшість науковців не бачили у них сенсу чи просто робили повільні темпи у своїх дослідженнях. Але, з іншої сторони, ставало більше доступної інформації та даних, на яких можна було навчати мережу, також зростала потужність обчислювальної техніки. Вже у 2012 році група дослідників, на чолі із Алексом Крижевським розробила свою архітектуру згорткової нейронної мережі [48]. Вона була представлена під час змагань ILSVRC [49], де вони і посіли перше місце. І через рік після публікації даної статті всі учасники конкурсу почали використовувати згорткову нейронну мережу для вирішення задачі класифікації. Будова даної мережі схожа на будову LeNet, але має більшу кількість фільтрів, у порівнянні із LeNet і тому вона була здатна класифікувати значно більшу кількість об'єктів. На рисунку 2.11 представлена будова AlexNet, що була опублікована в науковій роботі Алекса Крижевського і його команди.

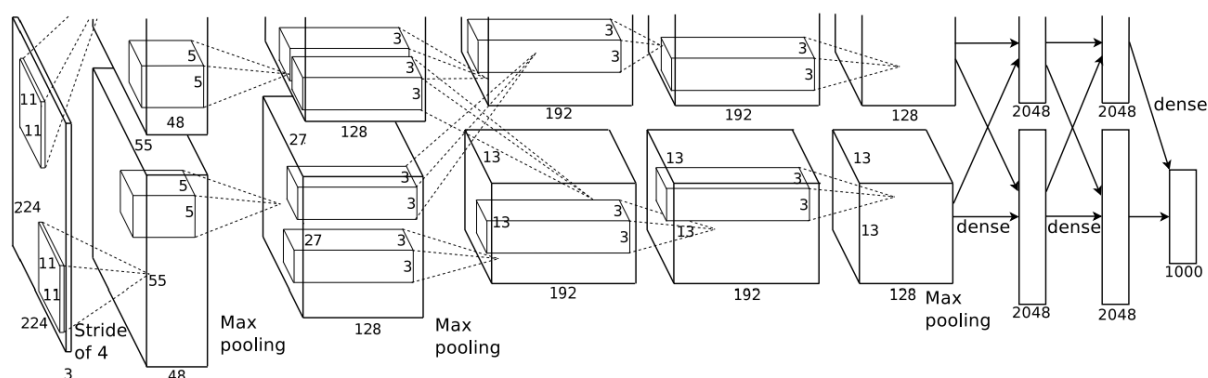


Рисунок 2.11 Будова AlexNet [48]

4. GoogLeNet (Inception). Наступна мережа - GoogLeNet [50]. Вона також, як і AlexNet, перемогла у конкурсі ILSVRC, але вже у 2014 році. Дослідники поставили собі за мету зменшення обчислювального навантаження нейронних мереж, створивши GoogLeNet, першу версію 3НМ Inception. Моделі глибокого навчання стали дуже корисними у вирішенні задачі класифікації різноманітних зображень чи відео після 2014 року. З врахування таких нових можливостей, така велика корпорація, як Google, почала також задумуватися про створення своєї нейронної мережі. Тому автори [50], які були співробітниками Google, почали розробляти свою архітектуру. Спочатку, вони створили модуль, який назвали Inception (рисунок 2.12) [50]. Основна ідея полягала у тому, що замість того, щоб застосовувати згорткові шари із різними параметрами у різних частинах мережі, створити один загальний блок, який буде виконувати операції згортки. Після цього, вони створили GoogLeNet, яка містила у собі таких дев'ять блоків Inception.

5. VGGNet. Ця мережа [21] посіла друге місце на конкурсі ILSVRC у 2014 році. Кінцева версія мала 16 пар згорткових і повнозв'язних шарів та мала однакові розміри фільтрів для згортки - 3×3 і 2×2 для операції пулінгу.

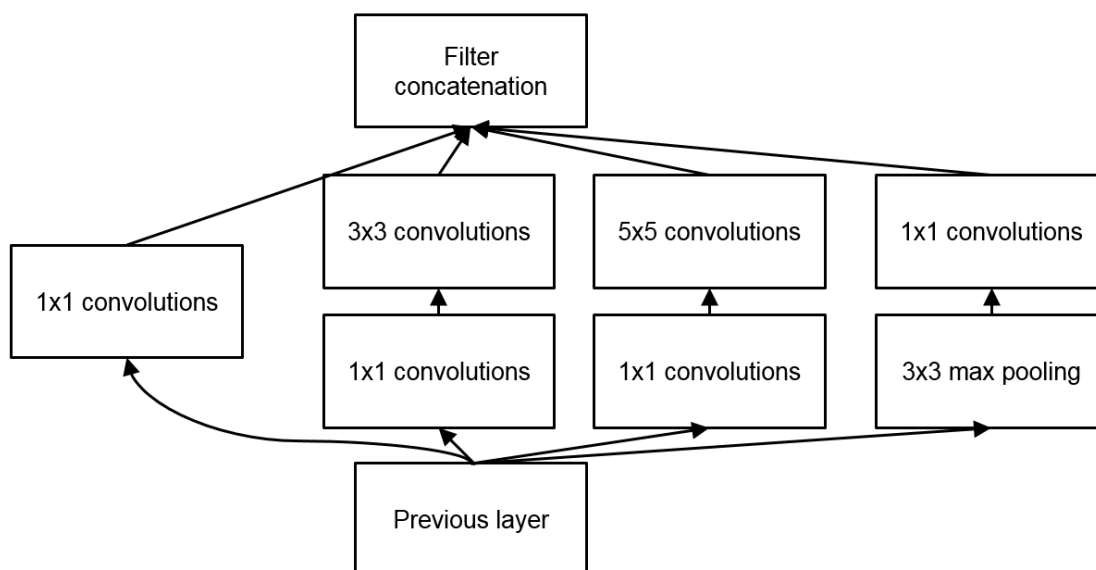


Рисунок 2.12 Модуль Inception [49]

6. ResNet. А ця мережа [51] стала переможцем конкурсу ILSVRC у 2015 році. У даній архітектурі відсутні повнозв'язні шари у кінці мережі.
7. Xception. Ця мережа [39] показує кращі результати після навчання. У табл. 2.1 представлено порівняльну характеристику деяких мереж [52].

Таблиця 2.1 Порівняння згорткових нейронних мереж

№	Модель	Топ-1 точність	Топ-5 точність	Параметри	Глибина
1	Xception	0.790	0.945	22,910,480	126
2	VGG16	0.713	0.901	138,357,544	23
3	VGG19	0.713	0.900	143,667,240	26
4	InceptionV3	0.779	0.937	23,851,784	159

Як бачимо, Xception, показує один із найточніших результатів після навчання. Автори моделі запропонували згорткову нейронну архітектуру нейронної мережі, що ґрунтується на повністю розділених по глибині згорткових шарах. Архітектура Xception (рисунок 2.13) має 36 згорткових шарів, що утворюють основу для вилучення ознак. Під час експериментального оцінювання науковці досліджували виключно класифікацію зображень, а отже і

використовували шар із логістичною регресією. Ці 36 згорткових шарів структуровано у 14 модулів, котрі мають усі лінійні залишкові з'єднання навколо себе, за винятком першого та останнього модулів.

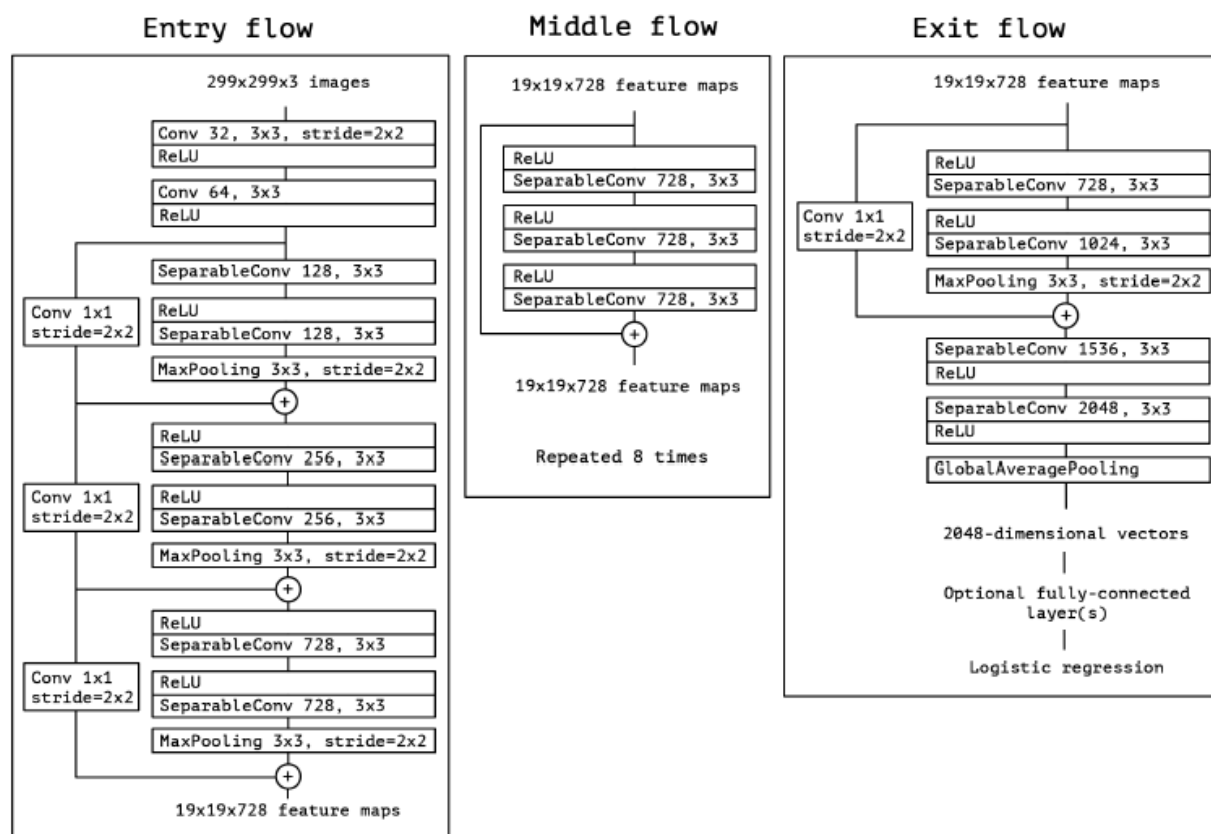


Рисунок 2.13 Архітектура мережі Xception [39]

З огляду на виконаний аналіз згорткових нейронних мереж можемо прийти до висновку, що для прогнозування привабливості тварин із притулку за фото можливо використати згорткову нейронну мережу Xception. Також проаналізуємо ще одну категорію нейронних мереж, а саме – розглянемо такі моделі як трансформери.

2.3.4 Аналіз трансформерів

Як відомо, задачі у комп'ютерному зорі, що використовують глибинне навчання, вже давно розв'язуються за допомогою домінуючих згорткових

нейронних мереж. Починаючи з AlexNet [48], про яку вже було написано вище, завдяки її революційній продуктивності на конкурсі класифікації зображень ImageNet, застосування згорткової архітектури стало досить популярним. Це відбулося за рахунок більш кращих зв'язків, більш кращих форм згортки тощо. Ці нові архітектурні досягнення у сфері комп'ютерного зору привели до більшої продуктивності та покращень, які загалом розвинули всю область комп'ютерного зору [53].

З іншої сторони, розвиток нейромережних архітектур у обробці природної мови пішов зовсім іншим шляхом, де сьогодні переважає архітектура нейронної мережі під назвою трансформер. Перевагою трансформер є здатність моделювати довгострокові залежності в даних [53].

Трансформер (Transformer) – це одна із моделей глибинного навчання, що використовує механізм “самоуваги”, диференційно зважуючи значущість кожної частини вхідних даних. Він використовується в основному в областях обробки природної мови (NLP) [54] і комп'ютерного зору (CV) [55].

Як і рекурентні нейронні мережі (RNN), трансформери призначені для обробки послідовних вхідних даних, таких як природна мова, для таких завдань, наприклад, як переклад і узагальнення тексту. Однак, на відміну від RNN, трансформери не обов'язково обробляють дані послідовно [54].

Проте, механізм уваги забезпечує контекст для будь-якої позиції у вхідній послідовності. Наприклад, якщо вхідними даними є речення природною мовою, то трансформеру не потрібно обробляти його початок, перш ніж оброблювати його кінець [54]. Натомість, він визначає контекст, який надає значення кожному слову в реченні. Ця функція дозволяє більше розпаралелювання, ніж RNN, і, отже, скорочує час навчання [54].

Цей тип нейронних мереж представлений у 2017 році дослідниками з Google Brain [54] і трансформери все частіше стають кращою моделлю задач NLP (Natural language processing) [56], замінюючи моделі RNN, такі як довготривала короткочасна пам'ять (Long short-term memory, LSTM) [33]. Додаткове

розпаралелювання під час навчання дозволяє проводити навчання на великих наборах даних, ніж це було можливо раніше. Це призвело до розробки попередньо навчених систем, таких як BERT [42] та GPT [40-41], які були натреновані на великих наборах мовних даних, таких як корпуси Вікіпедії та Common Crawl [57], і можуть бути добре оптимізовані і налаштовані під конкретні задачі [58]. До задач NLP, що можна вирішувати за допомогою трансформерів, можна віднести такі:

- прогнозування наступного речення [58];
- моделювання мов [58];
- генерація відповідей на запитання [59];
- перефразовування [60];
- сентиментальний аналіз [61].

Великий успіх у галузі мови спонукав дослідників досліджувати адаптацію трансформерів для комп'ютерного зору, де він продемонстрував багатообіцяючі результати в деяких завданнях, зокрема класифікації зображень та у завданнях, де використовуються як зорове, так і мовне моделювання [53].

Тож розглянемо трансформери для комп'ютерного зору детальніше.

2.3.4.1 Vision Transformer

Зоровий трансформер (Vision Transformer, ViT) — це трансформер, що призначений для виконання завдань комп'ютерного зору, таких як розпізнавання зображень [61]. Так як інформаційні технології швидко розвиваються, то у 2020 році Vision Transformers були адаптовані для виконання завдань у сфері комп'ютерного зору.

Ідея в полягала в тому, щоб розбити вхідні зображення як серію ділянок, які після перетворення у вектори розглядаються як слова в звичайному трансформері.

Якщо в області NLP механізм “уваги” трансформерів намагався зафіксувати зв'язки між різними словами тексту, що підлягає аналізу, то в

комп'ютерному зорі трансформери намагаються зафіксувати зв'язки між різними частинами зображення. У 2021 році модель із застосуванням лише трансформера продемонструвала кращу продуктивність і більшу ефективність, ніж CNN, щодо класифікації зображень [61]. Автори дослідження, що було проведене в червні 2021 року, додали трансформер до нейронної мережі Resnet. У результаті значно знизилися втрати моделі та підвищилась точність [62]. У тому ж році було запропоновано кілька важливих варіантів Vision Transformers. Ці модифікації в основному більш ефективні, точніші та краще підходять до певної області.

Серед найбільш актуальних є Swin Transformer [53], який завдяки деяким модифікаціям механізму “уваги” та багатоступінчастому підходу досяг найсучасніших результатів у деяких наборах даних виявлення об’єктів, таких як COCO [63]. Іншим цікавим варіантом є TimeSformer, розроблений для завдань із розуміння відео та здатний захоплювати просторову та тимчасову інформацію за допомогою використання розділеної уваги простору-часу [64].

Vision Transformers також змогли вийти у світ за рамки дослідницької лабораторії в одну з найважливіших областей комп'ютерного зору — автономне водіння. Інженери Tesla показали під час Tesla AI Day [65], що їхній автопілот Tesla використовує трансформер на багатокамерній системі в автомобілях.

2.3.4.2 Transformers vs CNN

Продуктивність ViT залежить від вибраного оптимізатора, гіперпараметрів, що стосуються набору даних і глибини самої мережі. Варіант “чистого” трансформера базується на тому, щоб поєднати трансформер із передньою частиною згорткової нейронної мережі. Базова версія ViT використовує згортку 16×16 з кроком 16.

На відміну від згортки 3×3 з кроком 2, збільшує стабільність, а також покращує точність [66]. Відмінностей між CNN і Vision Transformer досить багато і в основному вони полягають саме в їхніх архітектурних відмінностях.

Фактично, згорткові нейронні мережі досягають чудових результатів навіть при навчанні на таких обсягах даних, які не такі вже і великі, як були б необхідні для Vision Transformers.

Ця різна поведінка виникає через те, що в згортковій нейронній мережі присутні деякі індуктивні переконання, які можуть бути використані цими мережами, щоб швидше зрозуміти особливості аналізованих зображень, навіть якщо, з іншого боку, вони в кінцевому підсумку обмежують їх, роблячи більш складні для розуміння глобальні відносин між елементами нейронної мережі [67].

З іншого боку, Vision Transformers позбавлені цих упереджень, що приводить до того, що вони можуть фіксувати також глобальні та ширші зв'язки, але ціною більш обтяжливого навчання з точки зору даних. Vision Transformers також виявилися набагато більш стійкими до спотворення вхідного зображення, наприклад, змагальних партій або перестановок [68].

2.3.4.3 Застосування Vision Transformers

Зорові трансформери використовуються в завданнях комп'ютерного зору з гарними результатами, а в деяких випадках воно є найкращими. Серед найбільш актуальних сфер застосування можна віднести наступні приклади:

- детектування deepfake на відео;
- детектування об'єктів;
- сегментація зображень;
- водіння на автопілоті;
- класифікація зображень;
- аналіз аномалій;
- кластерний аналіз.

2.3.4.4 Swin Transformer

Swin Transformer є одним із найкращих архітектур серед трансформерів для комп'ютерного зору. Розглянемо його детальніше. Автори даної моделі із Microsoft Research Asia запропонували її у 2021 році. Назва даного трансформеру походить із того, що дослідники використовують ієрархічний трансформер, побудова якого базується на обчисленні за допомогою зміщених вікон (Shifted Windows) [53]. Зміщена схема вікон забезпечує більшу ефективність, обмежуючи обчислення “самоуваги” локальними вікнами, що не перекриваються, а також дозволяючи cross-window підключення. Представлена дослідниками ієрархічна архітектура має гнучкість для моделювання в різних масштабах і має лінійну обчислювальну складність щодо розміру зображення [53]. Ці властивості Swin Transformer роблять його сумісним із широким спектром завдань комп'ютерного зору, включаючи класифікацію зображень (87,3 топ-1 точність на ImageNet-1K) і задачі прогнозування, такі як виявлення об'єктів (58,7 box AP і 51,1 mask AP на COCO testdev). і семантична сегментація (53,5 mIoU на ADE20K val) [53].

Swin Transformer створює ієрархічні карти ознак і має лінійну обчислювальну складність до розміру зображення. На рис. 2.14 представлено порівняння Swin Transformer із попередньо відомим ViT. Запропонований дослідниками [53] Swin Transformer будує ієрархічні карти ознак шляхом об'єднання частин зображення (виділено сірим контуром на рис. 2.15) у більш глибоких шарах і має лінійну складність обчислень для вхідного розміру зображення через обчислення “самоуваги” лише в кожному локальному вікні (виділено червоним контуром на рис. 2.15). Таким чином, він може служити основою загального призначення як для класифікації зображень, так і для інших задач розпізнавання [53]. І, навпаки, трансформери попередньої моделі (ViT) створюють карти ознак з єдиною низькою роздільною здатністю і мають квадратичну складність обчислень для розміру вхідного зображення через глобальне обчислення “самоуваги” [53].

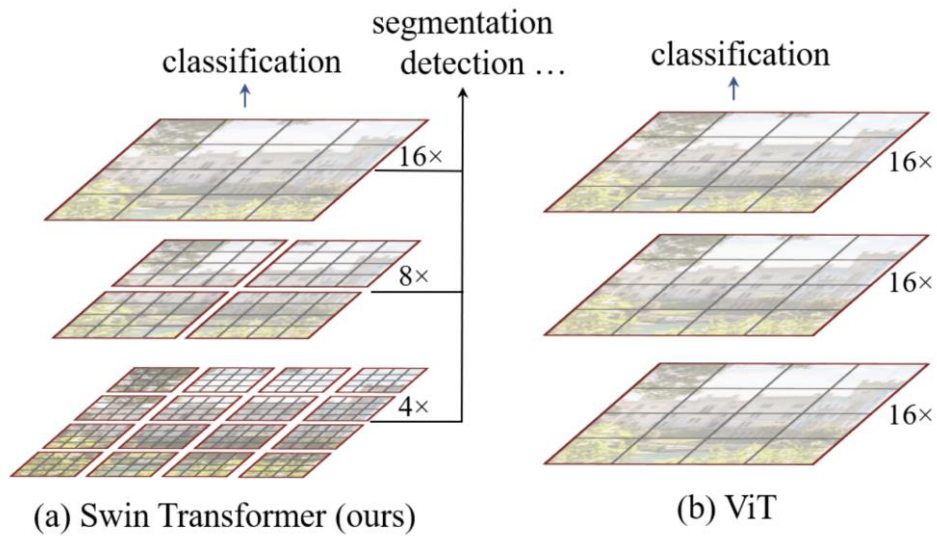


Рисунок 2.14 Порівняння Swin Transformer (a) із ViT (b) [53]

Ключовим елементом архітектури Swin Transformer є його зміщення віконної перегородки між послідовними шарами “самоуваги”, як показано на рисунку 2.15. У шарі l (ліворуч) прийнято звичайну схему розподілу вікон, і в кожному вікні обчислюється “самоувага”. У наступному шарі $l + 1$ (праворуч) розбиття вікна зміщується, в результаті чого з'являються нові вікна. Обчислення “самоуваги” в нових вікнах перетинає границі попередніх вікон у шарі l , забезпечуючи зв'язки між ними [53].

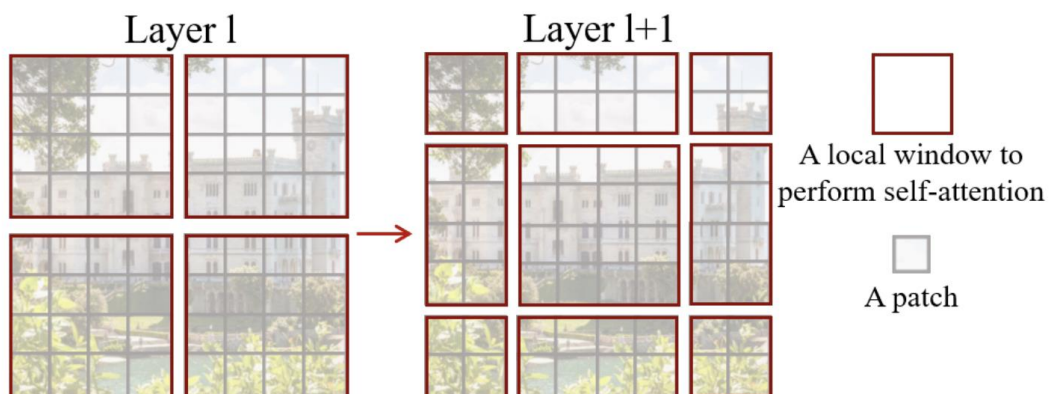


Рисунок 2.15 Зміщена схеми вікон [53]

Запропонований авторами [53] Swin Transformer досягає високої продуктивності в таких задачах розпізнавання, як: класифікація зображень, детектування об'єктів та семантична сегментація. Він значно перевершує моделі ViT / DeiT [69] і ResNe(X)t з подібною затримкою для трьох завдань. Розглянемо архітектуру Swin Transformer детальніше. На рисунку 2.16 (а) представлено архітектуру зменшеної версії даного трансформера (Swin-T). Спочатку трансформер розбиває вхідне кольорове RGB зображення на ділянки, що не перекриваються, за допомогою модуля розщеплення ділянок, такого як ViT. Кожен патч розглядається як «токен», а його функція встановлюється як конкатенація необроблених значень RGB пікселя. У цій реалізації автори використовують розмір ділянки 4×4 , а, отже, розмір об'єкта кожної ділянки становить $4 \times 4 \times 3 = 48$. Лінійний вбудовуваний шар застосовується до цієї об'єкти з необробленим значенням, щоб спроектувати її на довільний розмір (позначається як C) [53]. Декілька блоків трансформерів (блоки Swin Transformer) із модифікованим обчисленням «самоуваги» застосовуються до цих токенів. Блоки Transformer підтримують кількість маркерів ($\frac{H}{4} \times \frac{W}{4}$), та разом із лінійним вкладенням називаються «Stage 1» [53]. Щоб створити ієрархічне представлення, кількість токенів зменшується шляхом злиття шарів, оскільки нейронна мережа стає глибшою. Перший шар злиття ділянок об'єднує ознаки кожної групи з сусідніх ланок 2×2 і накладає лінійний шар на $4C$ -вимірні об'єднані об'єкти. Це зменшує кількість маркерів на $2 \times 2 = 4$ (зниження роздільної здатності в 2 рази), а вихідний розмір встановлюється на $2C$. Блоки Swin Transformer згодом застосовуються для трансформації ознак, з роздільною здатністю $\frac{H}{8} \times \frac{W}{8}$ [53]. Цей перший блок об'єднання ділянок і перетворення функцій позначається як «Stage 2». Процедура повторюється двічі, як «Stage 3» і «Stage 4», з вихідною роздільною здатністю $\frac{H}{16} \times \frac{W}{16}$ та $\frac{H}{32} \times \frac{W}{32}$ відповідно [53].

Ці етапи (stages) разом створюють ієрархічне представлення з такими ж роздільними можливостями карт ознак, що й у типових згорткових мереж, наприклад, VGG та ResNet. У результаті запропонована архітектура може зручно замінити основні відомі мережі в існуючих методах для різних завдань комп'ютерного зору [53].

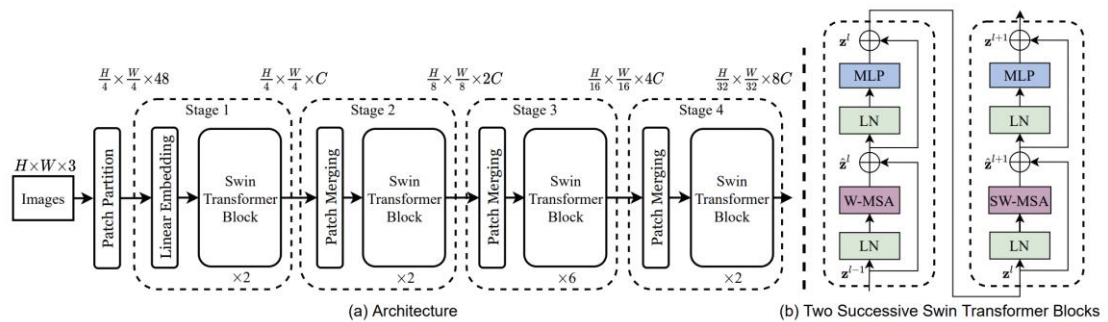


Рисунок 2.16 Будова Swin Transformer (a) і два послідовних блоки Swin Transformer (b) [53]

Swin Transformer створено шляхом заміни стандартного модуля “самоуваги” з кількома головами (multi-head self attention, MSA) в блоці трансформера (Transformer block) модулем на основі зміщених вікон, при цьому інші шари залишаються такими ж. Як показано на рис. 2.16 (b), блок Swin Transformer складається з модуля MSA зі зміщеним вікном, за яким слідує двошаровий багатошаровий перцептрон (MLP) з нелінійністю GELU між ними. Шар LayerNorm (LN) додається перед кожним модулем MSA і кожним MLP, а залишкове з’єднання застосовується після кожного модуля. Більше формул наведено авторами у [53]. Тобто, аналізуючи далі [53], бачимо, що Swin Transformer досягає найсучаснішої продуктивності щодо детектування об’єктів на наборі даних COCO [63] та семантичної сегментації на наборі даних ADE20K [70], значно перевершуючи попередні найкращі методи. Можна зробити висновок, що для прогнозування привабливості тварин із притулку за фото можливо використати також і Swin Transformer.

2.3 Особливості застосування бібліотек Python для реалізації нейронних мереж

2.3.1 Вибір мови Python як мови програмування

Для навчання нейронних мереж взято хмарне середовище розробки, що надається фірмою Kaggle [71], так як сам конкурс проходив на цій платформі. Нагадаємо, що Kaggle, яка є дочірньою компанією Google LLC, — це онлайн-спільнота дослідників даних і конкурсантів з машинного навчання.

Kaggle дозволяє користувачам знаходити та публікувати різні набори даних, досліджувати та створювати моделі у веб-середовищі для data science (за допомогою Kaggle Notebooks), працювати з іншими науковцями даних та інженерами з машинного навчання, а також можна взяти участь у змаганнях для вирішення data science проблем.

До переваг використання Kaggle як хмарного середовища розробки на Python та R можна віднести наступні:

- надається 30 годин на тиждень безкоштовного використання GPU для запуску своїх “блокнотів” (Kaggle Notebooks);
- надається 20 годин на тиждень безкоштовного використання TPU для запуску своїх “блокнотів” (Kaggle Notebooks);
- надається 100 GB дискового простору для власних файлів, наприклад, для завантажених наборів даних;
- тощо.

Для подальшого створення інтелектуальної технології розпізнавання облич у відеопотоці було вирішено взяти за середовище розробки PyCharm [72] і мову програмування Python [73]. До переваг середовища розробки PyCharm:

- легке та швидке виправлення помилок у коді в один клік;
- перевірка правопису слів у коментарях на різних мовах світу;
- форматування коду в один клік;
- тощо.

До переваг Python у застосуванні для машинного навчання можна віднести наступні [74]:

- Велика кількість різних бібліотек. Python пропонує широкий вибір бібліотек для розробки штучного інтелекту. Вони спрощують доступ, обробку та перетворення даних [74].
- Висока читабельність. Python славиться своїм компактним, читабельним кодом і практично не має собі рівних щодо зручності використання. Це зробило його улюбленою мовою для штучного інтелекту та глибокого навчання. Також, Python читається як повсякденна англійська мова, що робить розвиток штучного інтелекту простішим та менш складним [74].
- Гнучкість мови. Python пропонує на вибір використання об'єктно-орієнтованого програмування або написання простого коду. Немає вагомих причин для перекомпіляції вихідного коду; розробники можуть актуалізувати будь-які зміни та спостерігати за результатами. Розробники програмного забезпечення можуть приєднатися до Python та інших мов для досягнення своїх цілей [74].
- Великий вибір варіантів для візуалізації даних. Як вже згадувалося раніше, Python має великий набір бібліотек, і деякі з них пропонують дивовижні засоби візуалізації. Це дуже корисно для штучного інтелекту, оскільки візуалізація передбачає представлення даних у зручному для читання форматі [74].

Тепер перейдемо до аналізу та вибору бібліотек для створення інтелектуальної технології прогнозування привабливості тварини із притулку за фото.

2.3.2 Вибір бібліотек

Для Python існує багато різноманітних бібліотек, які можна використовувати для статистичних обчислень та візуалізації даних чи для машинного навчання. Наприклад, таких як:

- Matplotlib [75]. Використовується для візуалізації даних, побудови графіків та різних діаграм;
- Numpy [76], для обробки великих об'ємів даних, масивів та таблиць;
- TensorFlow [77]. Це безкоштовна бібліотека програмного забезпечення з відкритим вихідним кодом для машинного навчання та штучного інтелекту. Її можна використовувати в ряді завдань, але вона зосереджена на навчанні та використанні глибинних нейронних мереж [78].
- Keras [79]. Це програмна бібліотека з відкритим вихідним кодом, яка надає інтерфейс Python для штучних нейронних мереж. Keras виступає в якості інтерфейсу для бібліотеки TensorFlow. Keras містить багато реалізацій для часто використовуваних елементів для побудови нейронної мережі, таких як шари, цільові функції, функції активації, оптимізатори та безліч інструментів, що полегшують роботу з графічними та текстовими даними для спрощеного написання коду, необхідного для написання та запуску глибинних нейронних мереж [79]. На додаток до стандартних нейронних мереж, Keras підтримує згорткові та рекурентні нейронні мережі. Він підтримує інші загальні рівні утиліти, такі як вилучення, нормалізація пакетів і об'єднання [79].
- PyTorch [80]. Це фреймворк для машинного навчання з відкритим кодом, що побудований на бібліотеці Torch [81], який використовується для таких додатків, як комп'ютерне бачення та обробка природної мови [82], переважно розроблений лабораторією дослідження штучного інтелекту Facebook (FAIR) [83]. Це програмне забезпечення з відкритим вихідним кодом, яке було розроблене з ліцензією BSD.
- FastAi [84]. Це бібліотека для глибинного навчання, яка надає спеціалістам високорівневі компоненти. Завдяки цим компонентам можна швидко та легко створювати найсучасніші результати в стандартних областях глибинного навчання. Також бібліотека надає дослідникам низькорівневі компоненти, які можна змішувати та поєднувати для створення нових підходів та моделей.

Бібліотека прагне виконувати ці обидві речі без істотних компромісів у простоті використання, гнучкості та продуктивності. Це можливо завдяки ретельно багат шаровій архітектурі, яка виражає загальні основні закономірності багатьох методів глибинного навчання та обробки даних у термінах роз'єднаних абстракцій. Ці абстракції можна висловити коротко і чітко, використовуючи динамізм основної мови Python і гнучкість бібліотеки PyTorch. Бібліотека `fastai` організована навколо двох основних цілей: бути доступною і швидко продуктивною, а також бути налаштовуваною. Вона побудована на основі ієрархії API нижнього рівня, які забезпечують компоновані будівельні блоки. Таким чином, користувачеві, який хоче переписати частину високорівневого API або додати певну поведінку відповідно до своїх потреб, не потрібно вчитися використовувати найнижчий рівень (рис. 2.17). `Fastai` включає:

- нову систему диспетчеризації типів для Python разом із семантичною ієрархією типів для тензорів;
 - оптимізована для графічного процесора (GPU) бібліотека комп'ютерного зору;
 - оптимізатор, який реорганізує загальну функціональність сучасних оптимізаторів на дві основні частини, що дозволяє реалізувати алгоритми оптимізації в 4-5 рядках коду;
 - нова двостороння система зворотного виклику, яка може отримати доступ до будь-якої частини даних, моделі чи оптимізатора та змінити їх у будь-який момент під час навчання;
 - новий API блоку даних;
 - і багато іншого.
- Django [85]. Це безкоштовний веб-фреймворк на основі Python з відкритим кодом, який відповідає архітектурному шаблону `model–template–views` (MTV) [86]. Цей фреймворк підтримується Django Software Foundation (DSF).
 - Flask [87]. Це також веб-фреймворк, що розроблений на мові програмування Python. Його класифікують як мікрофреймворк, бо він не

потребує якихось певних інструментів чи бібліотек [88]. Flask підтримує деякі такі розширення, які вміють додавати функції програми, начебто вони реалізовані за допомогою самого Flask. Також, є деякі розширення для об'єктно-реляційних картографів, різних технологій відкритої аутентифікації, перевірки форм, обробки завантаження, та кількох деяких загальних інструментів, що пов'язані з фреймворком [89]. До програм, які використовують фреймворк Flask, належать Pinterest і LinkedIn.

- тощо.

З огляду на вищенаписане, для розробки та дослідження нейронних мереж для прогнозування привабливості тварини із притулку за фото підійдуть як бібліотеки Tensorflow & Keras, так і PyTorch & Fastai. Для написання серверної частини (логіки додатку) обрана бібліотека Flask. Перейдемо тепер до створення логіки бази даних.

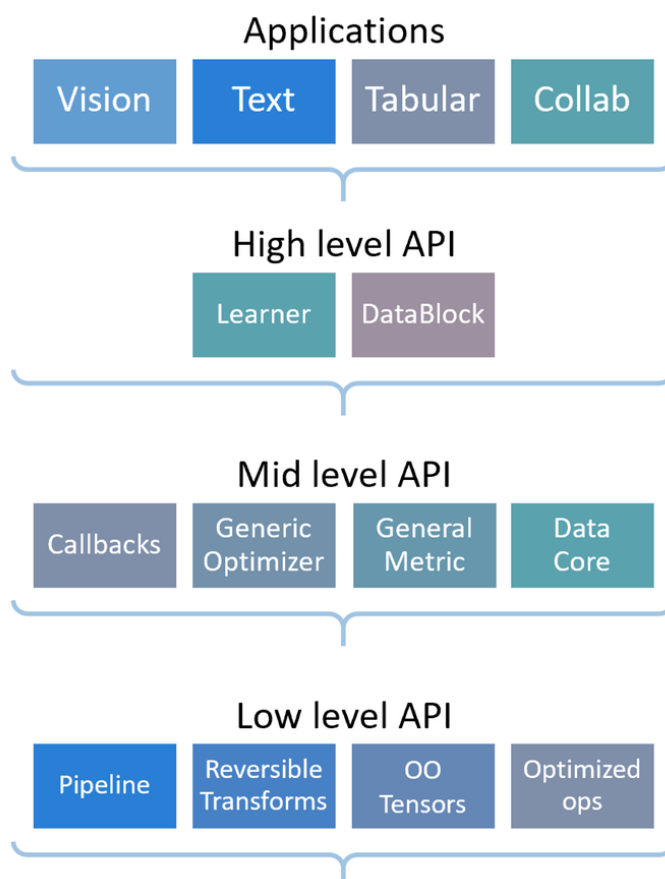


Рисунок 2.17 Структура побудови fastai [84]

2.4 Логіка бази даних

2.4.1 Аналіз особливостей предметної області

Перед тим як розробити базу даних (БД) інтелектуальної технології, що буде містити всю необхідну інформацію, потрібно детальніше дослідити предметну область (ПрО). БД складатиметься із двох таблиць - записів про наявних тварин у притулку і записів про всі випадки прогнозування привабливості тварин із притулку за фото, де міститься оцінка привабливості фотографії. Такі можливості для створення бази даних надає MySQL – система управління базами даних (СУБД), яка використовується із Python [90].

2.4.2 MySQL - середовище реалізації БД

Для розробки БД вибрано середовище MySQL [91]. Це вільна система управління базами даних, яка є дуже швидкою, надійною та легкою у використанні. Проектування БД відбувалося у середовищі Workbench. Це спеціальний офіційний інструмент для кращого візуального бачення БД при її створенні. Для запуску серверу використовується MySQL Server, який є елементом системи MySQL.

2.3.3 Фізична модель БД

Тепер розглянемо фізичну модель БД, що була створена у середовищі MySQL Workbench, яке є графічним інтерфейсом для MySQL. У цьому середовищі легко розробляти базу даних, будувати діаграми та запускати MySQL Server (рисунок 2.18). Опишемо таблиці бази даних (табл. 2.2). У таблиці прийняті такі позначення ключів:

- PK – первинний ключ (primary key);
- FK – зовнішній ключ (foreign key).
- Також є наступні типи даних:
- INT – ціле число;
- VARCHAR() – символічне значення;

- TEXT() – текстове значення;
- DATETIME() – дата та час;
- DOUBLE - дійсне число;
- ENUM() - вибір із множини перелічених.

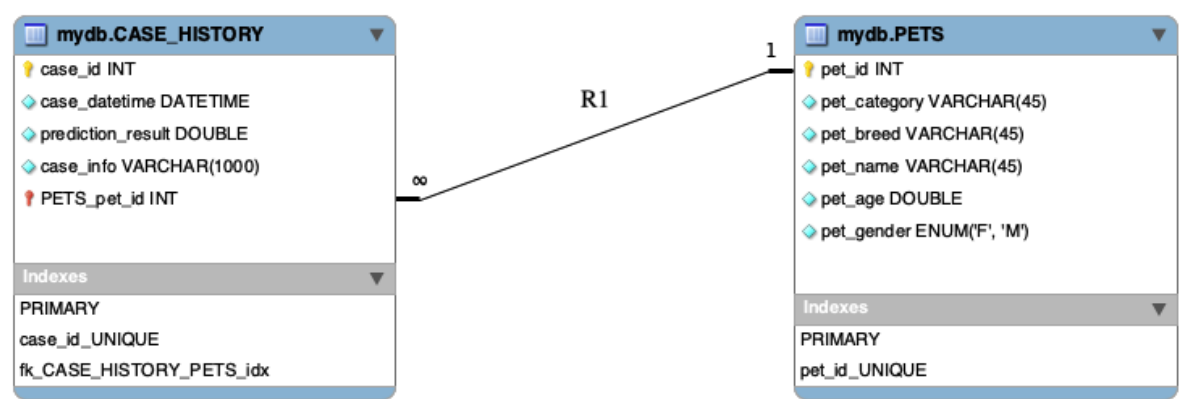


Рисунок 2.18 Фізична модель БД

Таблиця 2.2 Склад та характеристики атрибутів таблиць

Атрибут	Опис українською	Тип даних (Datatype)	Ключ (PK or FK)	Обов'язкове значення (NOT NULL)	Унікальний Індекс (UNIQUE)	Автоінкрементне поле (AUTO_INCREMENT)
Таблиця PETS						
pet_id	ІД тварини із притулку	INT	PK	+	+	+
pet_category	тип тварини (собака чи кіт)	VARCHAR(45)		+		
pet_breed	порода тварини	VARCHAR(45)		+		
pet_name	кличка тварини	VARCHAR(45)		+		
pet_age	вік тварини	DOUBLE		+		
pet_gender	стать тварини (ж / ч)	ENUM('F', 'M')		+		

Продовження таблиці 2.2

Атрибут	Опис українською	Тип даних (Datatype)	Ключ (PK or FK)	Обов'язкове значення (NOT NULL)	Унікальний Індекс (UNIQUE)	Автоінкрементне поле (AUTO_INCREMENT)
Таблиця CASE_HISTORY						
case_id	ІД випадку прогнозування привабливості тварини із притулку за фото	INT	PK	+	+	+
case_datetime	час випадку	DATETIME		+		
prediction_result	результат прогнозування (оцінка привабливості тварини із притулку за фото)	DOUBLE		+		
case_info	додаткова інформація про випадок	VARCHAR(1000)		+		
PETS_pet_id	ІД тварини	INT	FK	+		

Також, розглянемо таблицю опису зв'язків для даної схеми, що були позначені на фізичній моделі як R1. Результати представлені у таблиці 2.3.

Таблиця 2.3 Опис зв'язків схеми

Назва зв'язку	Відношення	Тип відношення	Опис
R1	1:n	ідентифікуюче	Одна тварина, але прогнозувати її фотографію за привабливістю можна багато разів (тобто, може бути багато випадків у базі даних).

Після цього, таблиці у MySQL Workbench були заповнені відповідною інформацією. Для того, щоб можна було взаємодіяти із БД (робити запити, чи створювати нові записи), необхідно створити БД на сервері та скопіювати вже готову БД та всі її дані таблиць. Як вже було вказано вище, це буде MySQL Server (рисунок 2.19). Після того, як відбудеться успішне створення БД на сервері, синхронізація вже раніше створеної моделі із тою, що зберігається на сервері, отримаємо кінцеву готову БД (рисунок 2.20). На рис. 2.21 - 2.22 представлено таблиці, що були експортовані до MySQL Server, інформація про які вже була написана у таблицях 2.2 - 2.3.

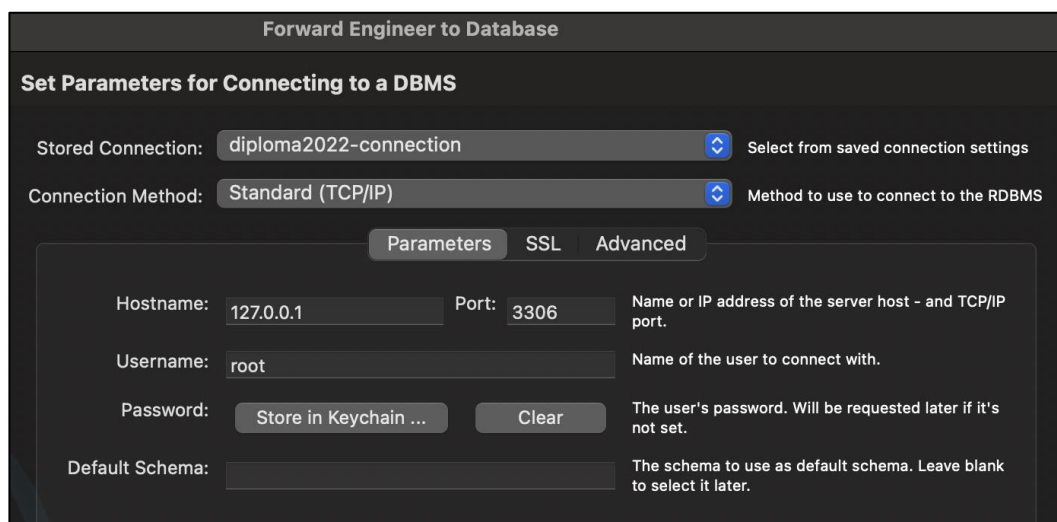


Рисунок 2.19 Підключення до БД на сервері MySQL

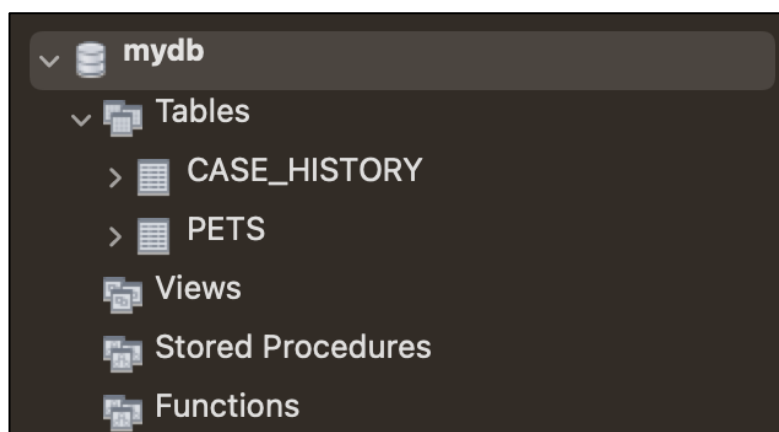
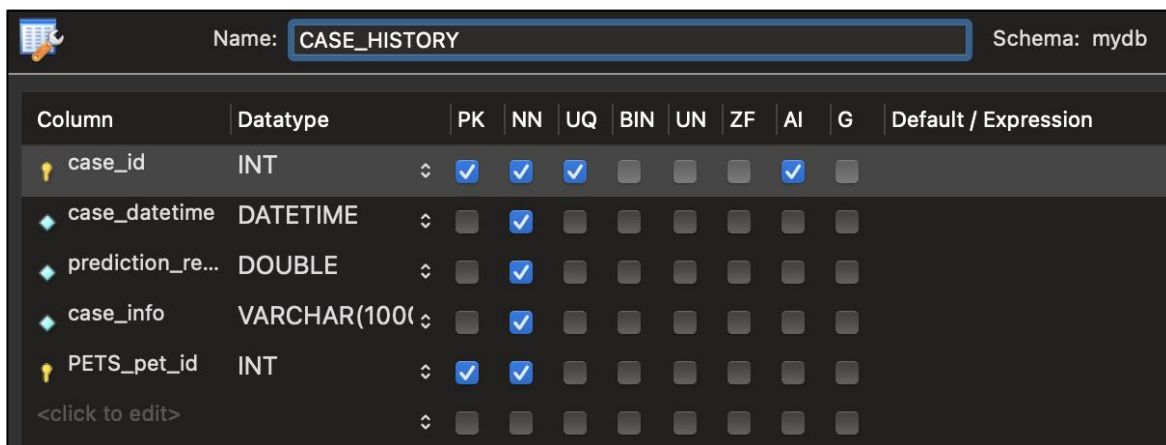


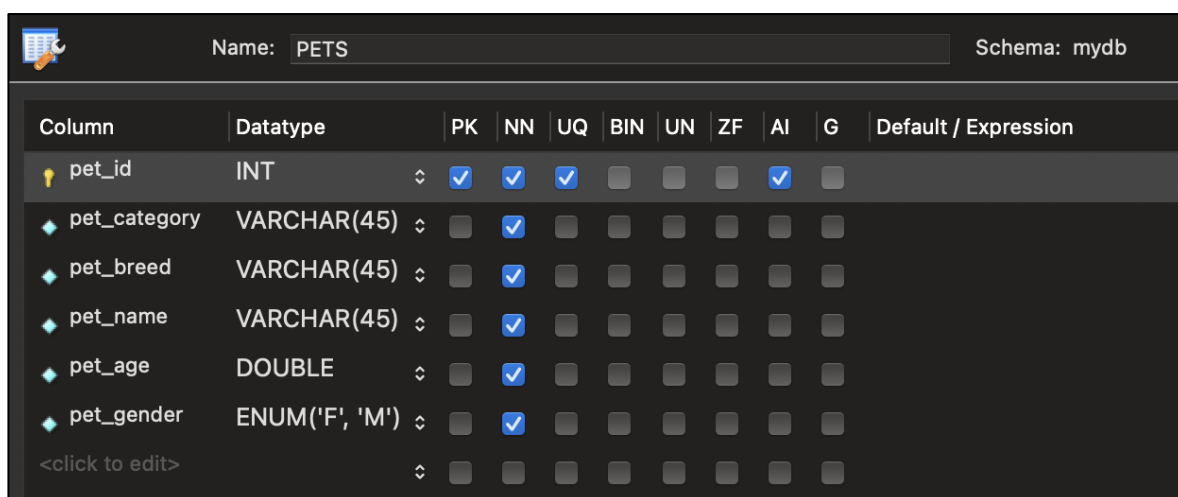
Рисунок 2.20 Готова БД на сервері MySQL Server



MySQL Server interface showing the structure of the **CASE_HISTORY** table in the **mydb** schema. The table has the following columns and attributes:

Column	Datatype	PK	NN	UQ	BIN	UN	ZF	AI	G	Default / Expression
case_id	INT	☒	☒	☒	☐	☐	☐	☒	☐	
case_datetime	DATETIME	☐	☒	☐	☐	☐	☐	☐	☐	
prediction_re...	DOUBLE	☐	☒	☐	☐	☐	☐	☐	☐	
case_info	VARCHAR(100)	☐	☒	☐	☐	☐	☐	☐	☐	
PETS_pet_id	INT	☒	☒	☐	☐	☐	☐	☐	☐	
<click to edit>		☐	☐	☐	☐	☐	☐	☐	☐	

Рисунок 2.21 Таблиця CASE_HISTORY на MySQL Server



MySQL Server interface showing the structure of the **PETS** table in the **mydb** schema. The table has the following columns and attributes:

Column	Datatype	PK	NN	UQ	BIN	UN	ZF	AI	G	Default / Expression
pet_id	INT	☒	☒	☒	☐	☐	☐	☒	☐	
pet_category	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	
pet_breed	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	
pet_name	VARCHAR(45)	☐	☒	☐	☐	☐	☐	☐	☐	
pet_age	DOUBLE	☐	☒	☐	☐	☐	☐	☐	☐	
pet_gender	ENUM('F', 'M')	☐	☒	☐	☐	☐	☐	☐	☐	
<click to edit>		☐	☐	☐	☐	☐	☐	☐	☐	

Рисунок 2.22 Таблиця PETS на MySQL Server

2.5 Логіка інтерфейсу

Було вирішено зробити веб-додаток у стилі “Landing Page”, тобто, щоб вся інформація для користувача була відображена на одній сторінці без якихось додаткових лишніх переходів на інші сторінки. За результатами опитування потенційних клієнтів, даний стиль є зручним для користувача, тому було вибрано саме оформлення веб-сайту у стилі “Landing Page”.

Веб-сайт починається титульним аркушем (рис. 2.23). На ньому описано назву випускної кваліфікаційної роботи, автора, вставлений логотип. Справа від тексту розміщується абстрактне зображення для естетики. Прогортаючи далі,

користувач бачить частину власне для прогнозування привабливості тварини із притулку за фото (рис. 2.24-2.25). Там є віконце для попереднього перегляду обраної фотографії перед завантаженням; поле для вводу додаткової інформації про тварину, що зображена на обраній фотографії; кнопка для вибору файлу; кнопка для аналізу зображення; кнопка для вибору тварини із бази даних, якій відповідає завантажене зображення; кнопка для того, щоб оновити сторінку і почати прогнозування заново. Прогортаючи далі, користувач має можливість вивантажити інформацію із бази даних у окремі файли (рис. 2.26). До таких відносяться: інформація про всіх наявних тварин із притулку, що є у базі даних та файл із обліком ведення всіх випадків (результатів прогнозування привабливості тварини із притулку за фото) із вказанням дати прогнозування привабливості. Далі користувач може ознайомитися із параметрами використаної нейронної мережі, почитати додатково про використану бібліотеку Python, відвідати сайт змагань (PetFinder.my - Pawpularity Contest), натиснувши на відповідну кнопку. Також користувач може написати листа розробнику, перейти на сайт університету, факультету тощо (рис. 2.27-2.28).

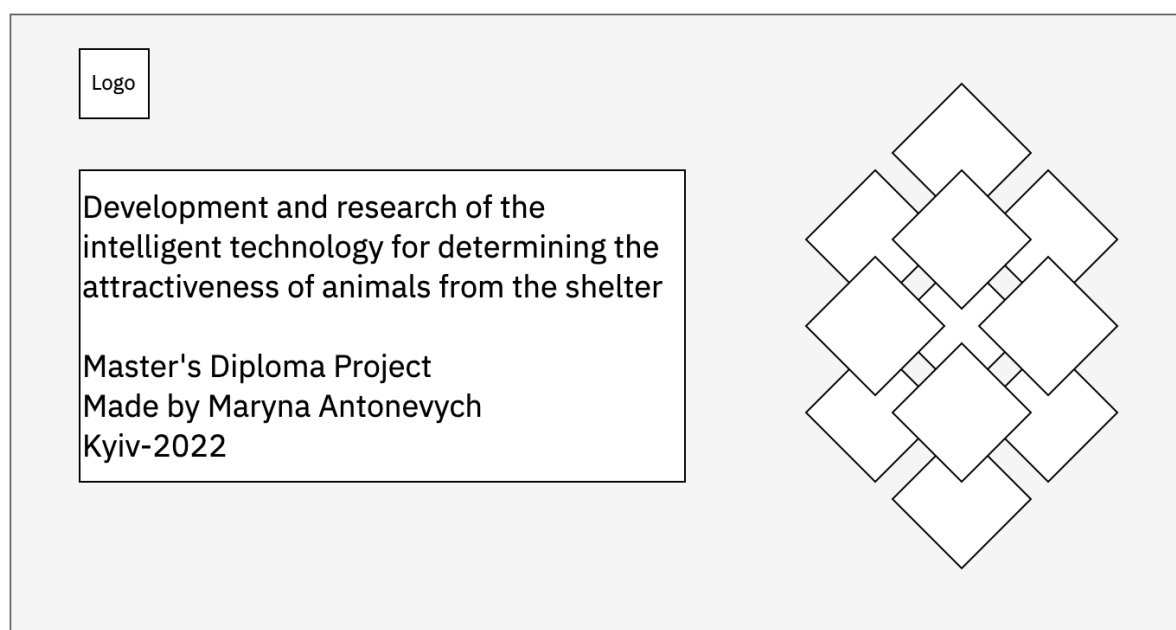


Рисунок 2.23 Landing Page

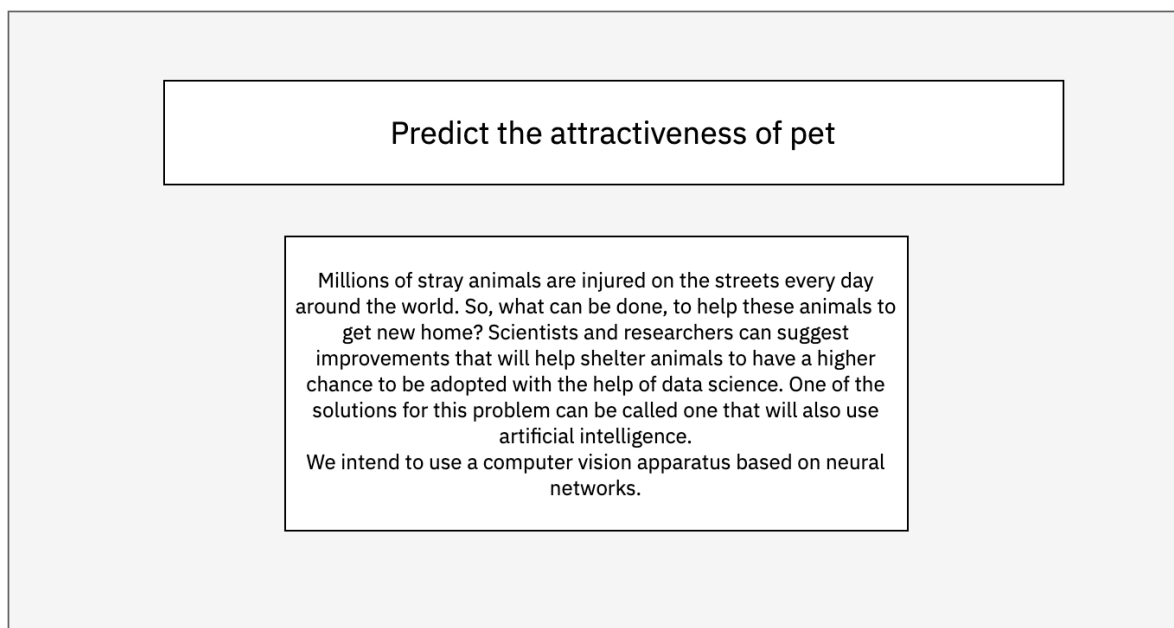


Рисунок 2.24 Landing Page (продовження)

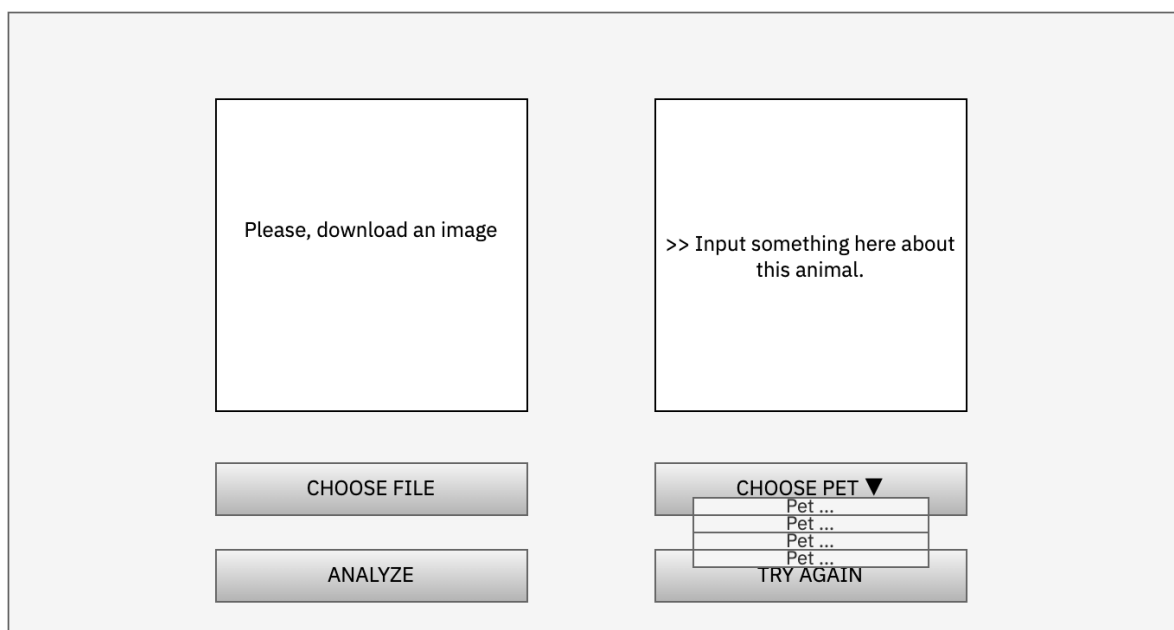


Рисунок 2.25 Landing Page (продовження)

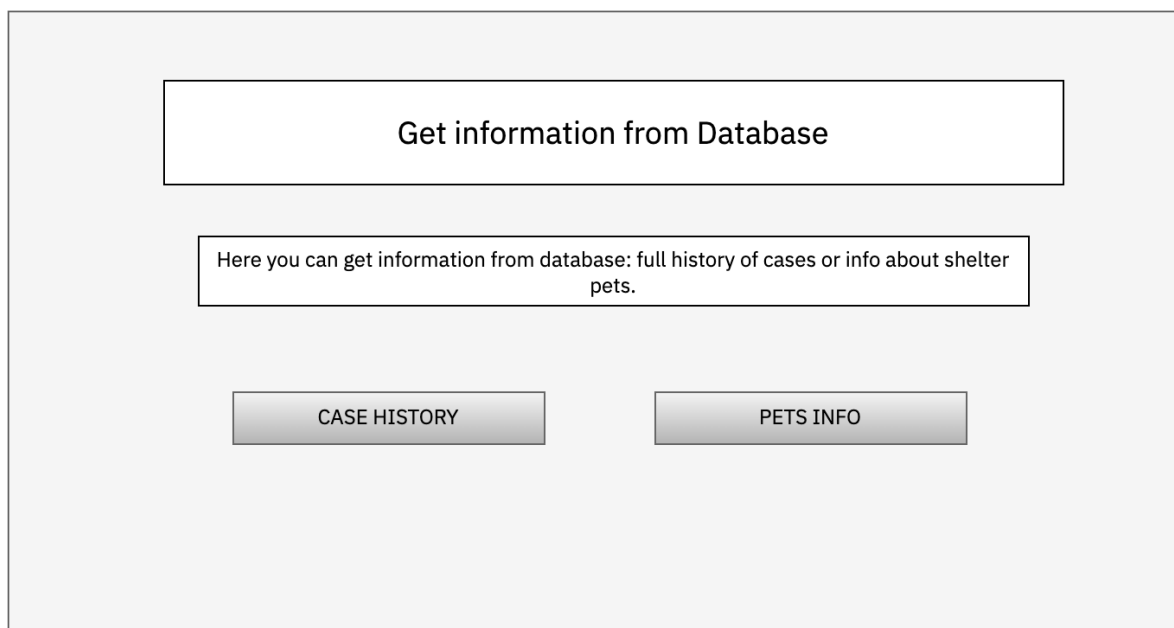


Рисунок 2.26 Landing Page (продовження)

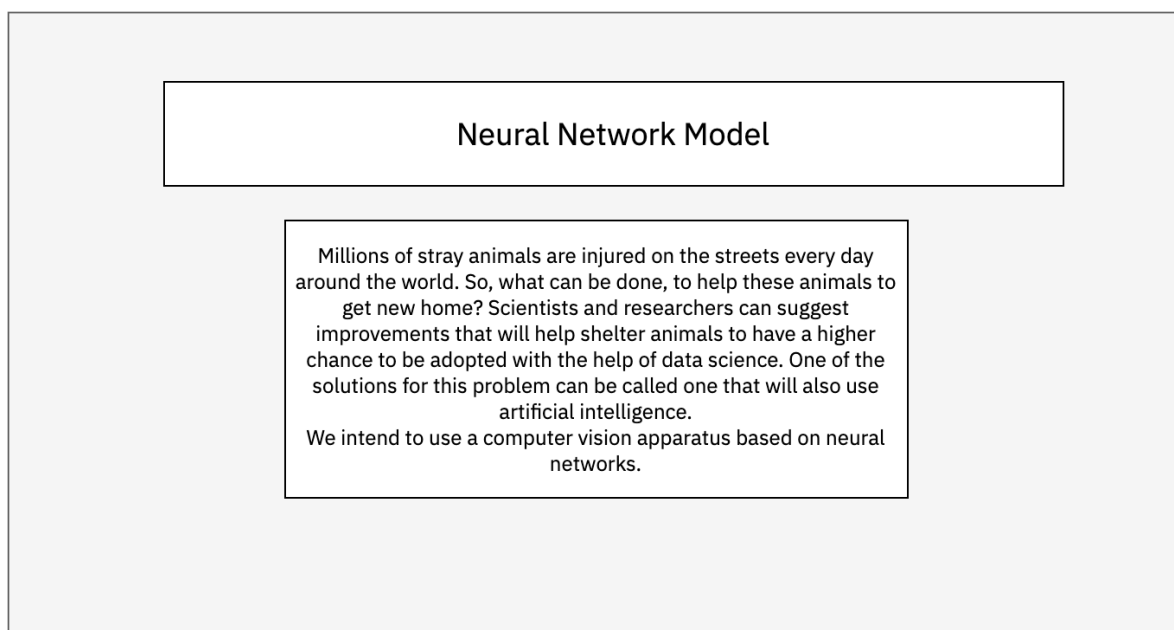


Рисунок 2.27 Landing Page (продовження)

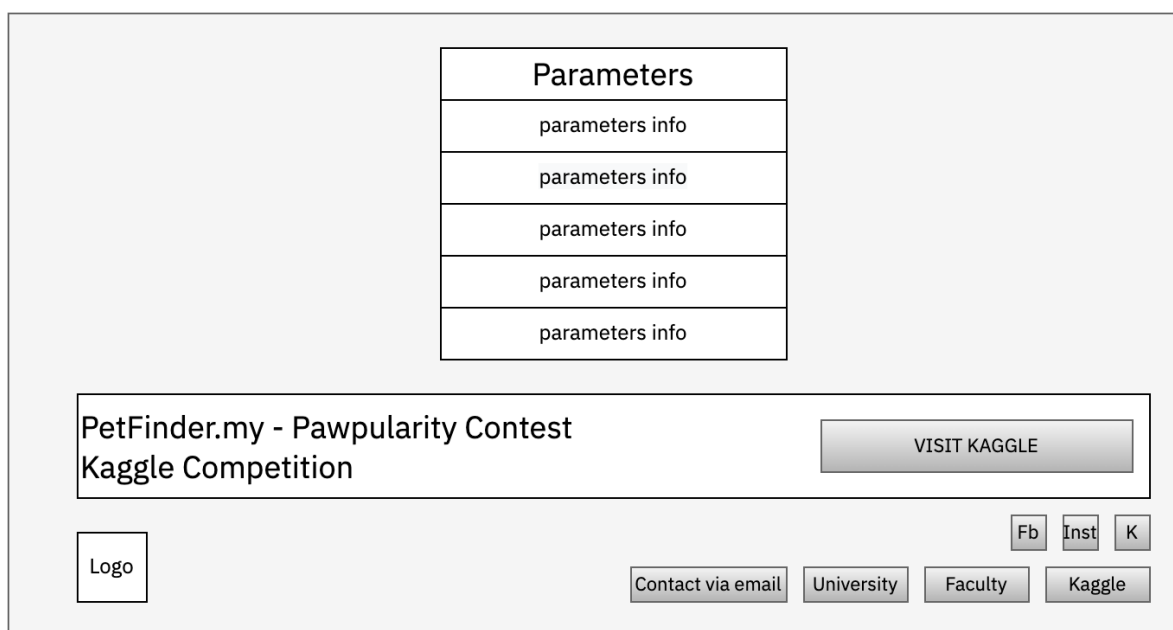


Рисунок 2.28 Landing Page (продовження)

2.5. Висновки до розділу 2

У другому розділі було спроектовано засади інтелектуальної технології. Побудовано DFD та структурну діаграми. Виконано аналіз нейромережних засобів для виконання випускної кваліфікаційної роботи. Визначено переваги та недоліки нейронних мереж. Вибрано як основні засоби для дослідження згорткові нейронні мережі (Xception) та трансформери (Swin Transformer). Вибрано мову програмування Python для написання коду. Встановлено її основні переваги. За результатами попереднього аналізу, визначено основні бібліотеки, що переважають за всіма основними показниками і були вибрані для написання логіки додатку Tensorflow & Keras та PyTorch & FastAi. Якщо їх всі порівнювати, то можна сказати, що продуктивність менша у бібліотеки Keras, у порівнянні із Tensorflow та PyTorch. TensorFlow та PyTorch використовуються програмістами для різних високопродуктивних моделей та великих наборів даних, які потребують швидкого виконання. За допомогою Tensorflow & Keras можливо побудувати легко і швидко згорткову нейронну мережу Xception. Із точки зору відладки коду, то PyTorch має кращі можливості, чим інші дві бібліотеки. Також,

бібліотека FastAi містить необхідні моделі трансформерів, а саме - Swin Transformer. Для організації серверної сторони додатку було розглянуто дві бібліотеки, а саме – Django та Flask для порівняння. Наприклад, Flask має модульну структуру, тоді як Django поєднує все разом. Щодо їх продуктивності, то можна сказати, що тут вони мають приблизно однакові значення. Django був розроблений для швидкого розвитку складних веб-додатків. Він надає розробникам необхідні інструменти для реалізації масштабованого та підтримуваного функціоналу. З іншого боку, простота Flask дозволяє розробникам швидше створювати менші додатки. Проаналізувавши доступну інформацію, вирішено використовувати Flask, так як він швидкий і зручний у використанні та має доступні бібліотеки для створення логіки додатку. За результатами попереднього аналізу вибрано базу даних MySQL. Створено та описано логіку бази даних MySQL. Побудовано EER-діаграму, описано фізичну модель БД. Запущено MySQL Server, що дозволило поєднати логіку інтерфейсу із логікою бази даних. Створено та описано макети для побудови логіки інтерфейсу.

РОЗДІЛ 3 РЕАЛІЗАЦІЯ ІНТЕЛЕКТУАЛЬНОЇ ТЕХНОЛОГІЇ. РЕЗУЛЬТАТИ ДОСЛІДЖЕНЬ ТА ЕКСПЕРИМЕНТІВ

3.1 Вхідні обмеження для проведення експериментів

Для реалізації та написання нейронної мережі обрано середовище Kaggle Notebooks. Обмеження, що надаються даною системою:

- 9 годин GPU на одну сесію запуску коду;
- 30 годин GPU на тиждень;
- 15.9 GB GPU;
- 13 GB ОЗП;
- 2 submissions на конкурс в день.

3.2 Реалізація та вибір оптимальної мережі

Всього було проведено вдалих більше 30 досліджень (тобто таких, яких подано як submissions на конкурс). Для того, щоб вибрати оптимальну структуру мережі необхідно розуміти, яким чином можна покращувати навчання мережі, щоб проводити різні експерименти. Відомо, що на результати навчання мережі впливає не лише її структура та вхідний набір даних, але і гіперпараметри самої мережі. До них можна віднести наступні: розмір вхідного зображення (image size); розмір партії зображень (batch size); кількість епох навчання (epochs); коефіцієнт швидкості навчання (learning rate); яка частина набору даних відноситься до навчальної і тестової вибірок (fraction); через скільки епох закінчується навчання, якщо не покращується результат (patience). Також, важливим є те, чи навчаються у мережі лише останні шари, чи всі. Розмір вхідного зображення із набору даних визначається самою мережею та її структурою. Розмір партії зображень - це кількість зображень, що подаються за раз на навчання до мережі. Розмір партії зображень, як правило, має бути кратний степені двійки ($2^n, n \in \mathbb{N}$). Кількість епох навчання визначає, як довго мережа буде навчатися. Коефіцієнт швидкості навчання визначає, з яким кроком

нейромережа буде навчатися. Відомо, що якщо цей коефіцієнт великий, мережа буде навчатися швидко, але існує ризик її перенавчання, тому що можна перестрибнути оптимум. Якщо коефіцієнт швидкості навчання маленький, то мережа повільно буде покращувати свою точність, можливо, необхідно буде збільшити кількість епох, але так зменшується ризик перенавчання мережі. Поділ на навчальну та тестові вибірки, як правило, обирають один із таких: 70% / 30%, 80% / 20%, 90% / 10% відповідно. Також, користувач може задати числове значення (patience) примусового зупинки навчання мережі, якщо з кожною епохою точність мережі не буде покращуватися. Розглянемо спочатку результати експериментів №1 з використанням згорткових нейронних мереж, які представлено у Табл. 3.1.

Таблиця 3.1 Результати експериментів №1

№ з/п	Назва параметру	Назва моделі						
		Модель №1	Модель №2	Модель №3	Модель № 4	Модель № 5	Модель № 6	Модель № 7
1	Дата	21.10.2021 1 19:36:32	11.11.2021 21 20:17:03	11.11.2021 1 21:48:26	16.11.2021 1 19:04:53	17.11.2021 1 20:37:52	18.11.2021 1 10:28:08	18.11.2021 1 13:05:44
2	Час роботи коду	4008.3s - GPU	4039.0s - GPU	2304.2s - GPU	4664.8s - GPU	2249.9s - GPU	3141.3s - GPU	2972.1s - GPU
3	Private Score	-	-	-	21.95402	28.28059	24.04538	23.48509
4	Public Score	-	-	-	21.94387	28.04129	24.01819	23.70700
5	Бібліотека	tensorflow.keras						
6	“Pawpularity” data transform	Log1p()						
7	image_size	(224, 224)				(150, 150)		
8	batch_size	32		8	32			64
9	epochs	20		10	20			
10	Базова модель	Користувачка CNN				Xception		
11	base_model.trainable	-				False	True	
12	Останній шар	Dense(1, activation = 'linear')						
13	Результат	Notebook Threw Exception			Succeeded			

Модель-1. Опишемо деякі особливості навчання Моделі-1 та основні кроки написання коду.

1. Спочатку було проаналізовано набір даних (рис 3.1).
2. Після цього проаналізовано колонку “Pawpularity”, у якій записано відповідні оцінки привабливості тварини із притулку за фото. На рис. 3.2-3.3 представлено основні статистичні характеристики даної оцінки (медіана, середньо квадратичне відхилення, мінімальне та максимальні значення тощо).
3. Також, тут проаналізовано і побудовано графік розподілу частот значень “Pawpularity” (рис. 3.4). Потім вирішено застосувати до даної колонки перетворення $\text{Log1p}()$ (рис. 3.5), щоб зробити нормальний розподіл у значеннях “Pawpularity” (рис. 3.6). Це робилося через те, що медіана та середнє значення були не близькі між собою. Отримаємо тоді нові медіану та середнє значення (рис. 3.7).
4. На рис. 3.8 представлено структуру даної нейронної мережі.
5. У результаті мережа навчилася, але при поданні коду на конкурс виникла помилка “Notebook Threw Exception”. Інші параметри наведені у табл. 3.1



Рисунок 3.1 Приклад набору даних

```
stats = df_train['Pawpularity'].describe()
stats
```

```
count    9912.000000
mean      38.039044
std       20.591990
min        1.000000
25%       25.000000
50%       33.000000
75%       46.000000
max       100.000000
Name: Pawpularity, dtype: float64
```

Рисунок 3.2 Статистичні характеристики цільового значення - “Pawpularity”

```
print("Median = ", df_train['Pawpularity'].median())
print("Mean = ", df_train['Pawpularity'].mean())
```

```
Median = 33.0
Mean = 38.03904358353511
```

Рисунок 3.3 Медіана та середнє значення “Pawpularity”

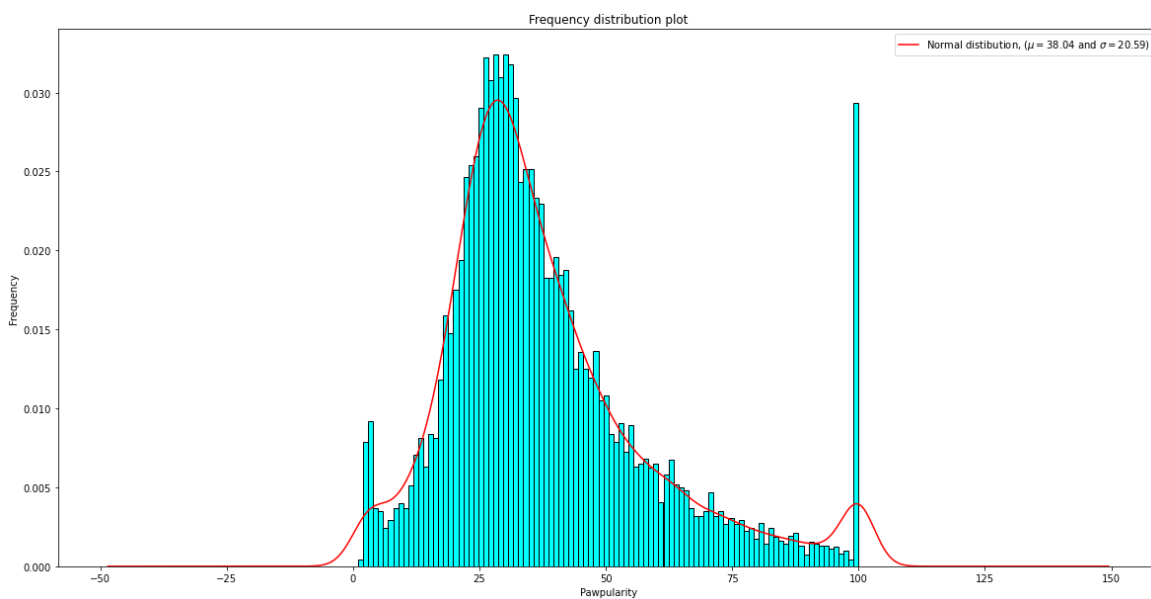


Рисунок 3.4 Частотний розподіл значень “Pawpularity”

```
df_train["Pawpularity"] = np.log1p(df_train["Pawpularity"])
plot_distribution(df_train)
```

Рисунок 3.5 Застосування $\text{Log1p}()$ перетворення

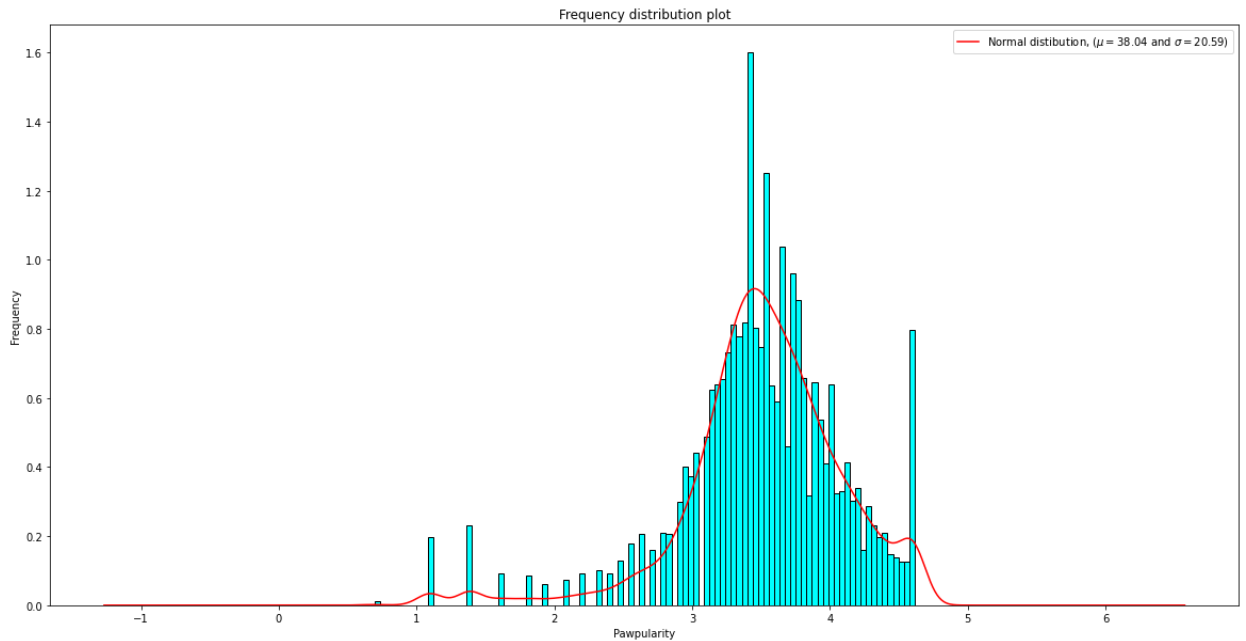


Рисунок 3.6 Частотний розподіл значень “Pawpularity” після перетворення $\text{Log1p}()$

```
print("New median = ", df_train['Pawpularity'].median())
print("New mean = ", df_train['Pawpularity'].mean())
```

```
New median = 3.5263605246161616
New mean = 3.5175952132252193
```

Рисунок 3.7 Медіана та середнє значення “Pawpularity” після перетворення $\text{Log1p}()$

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 54, 54, 96)	34944
max_pooling2d (MaxPooling2D)	(None, 27, 27, 96)	0
batch_normalization (Batch Normalization)	(None, 27, 27, 96)	384
conv2d_1 (Conv2D)	(None, 17, 17, 256)	2973952
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 256)	0
batch_normalization_1 (Batch Normalization)	(None, 8, 8, 256)	1024
conv2d_2 (Conv2D)	(None, 6, 6, 384)	885120
batch_normalization_2 (Batch Normalization)	(None, 6, 6, 384)	1536
conv2d_3 (Conv2D)	(None, 4, 4, 384)	1327488
batch_normalization_3 (Batch Normalization)	(None, 4, 4, 384)	1536
conv2d_4 (Conv2D)	(None, 2, 2, 256)	884992
max_pooling2d_2 (MaxPooling2D)	(None, 1, 1, 256)	0
batch_normalization_4 (Batch Normalization)	(None, 1, 1, 256)	1024
flatten (Flatten)	(None, 256)	0
dense (Dense)	(None, 4096)	1052672
dropout (Dropout)	(None, 4096)	0
batch_normalization_5 (Batch Normalization)	(None, 4096)	16384
dense_1 (Dense)	(None, 4096)	16781312
dropout_1 (Dropout)	(None, 4096)	0
batch_normalization_6 (Batch Normalization)	(None, 4096)	16384
dense_2 (Dense)	(None, 1000)	4097000
dropout_2 (Dropout)	(None, 1000)	0
batch_normalization_7 (Batch Normalization)	(None, 1000)	4000
dense_3 (Dense)	(None, 1)	1001

Total params: 28,080,753
 Trainable params: 28,059,617
 Non-trainable params: 21,136

Рисунок 3.8 Архітектура Моделі-1

Модель-2. Модель-2 мала таку ж архітектуру, як і Модель-1 (різниця лише у режимі зчитування даних) і також при валідації коду на конкурс викинула ту ж саму помилку “Notebook Threw Exception”.

Модель-3. Модель-3 мала таку ж архітектуру, як і Модель-1 та Модель-2 (різниця лише у режимі зчитування даних) і також при валідації коду на конкурс викинула ту ж саму помилку “Notebook Threw Exception”.

Модель-4. Модель-4 мала таку ж архітектуру, як і Модель-1, Модель-2 та Модель-3. Результати: Private Score = 21.95402, Public Score = 21.94387.

Модель-5. Модель-5 мала в собі за базову модель модель Xception (рис. 3.9), всі етапи попередньої обробки даних були такими ж, як і в Модель-1 - Модель-4. Результати: Private Score = 28.28059, Public Score = 28.04129. Бачимо, що дана модель дала гірші результати у порівнянні із попередньою моделлю. На рис 3.10 представлена історія навчання моделі.

Model: "model"

Layer (type)	Output Shape	Param #
input_2 (InputLayer)	[(None, 150, 150, 3)]	0
xception (Functional)	(None, 5, 5, 2048)	20861480
global_average_pooling2d (G1	(None, 2048)	0
dense (Dense)	(None, 1)	2049
Total params: 20,863,529		
Trainable params: 2,049		
Non-trainable params: 20,861,480		

Рисунок 3.9 Архітектура Моделі-5

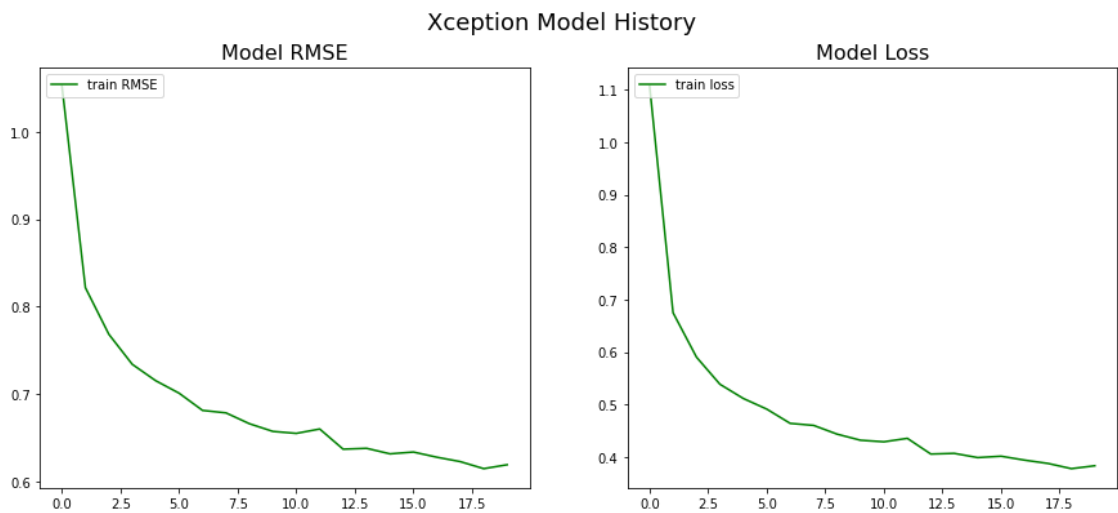


Рисунок 3.10 Історія навчання Моделі-5

Модель-6. Модель-6 мала в собі за базову модель модель Xception. Різниця між даною і Моделлю-5 у тому, що тут були розморожені шари, тому модель має більше шарів для навчання (рис. 3.11). На рис. 3.12 зображено історія навчання мережі.

Model: "model"

Layer (type)	Output Shape	Param #
input_2 (InputLayer)	[(None, 150, 150, 3)]	0
xception (Functional)	(None, 5, 5, 2048)	20861480
global_average_pooling2d (G1	(None, 2048)	0
dense (Dense)	(None, 1)	2049

Total params: 20,863,529
Trainable params: 20,809,001
Non-trainable params: 54,528

Рисунок 3.11 Архітектура Моделі-6

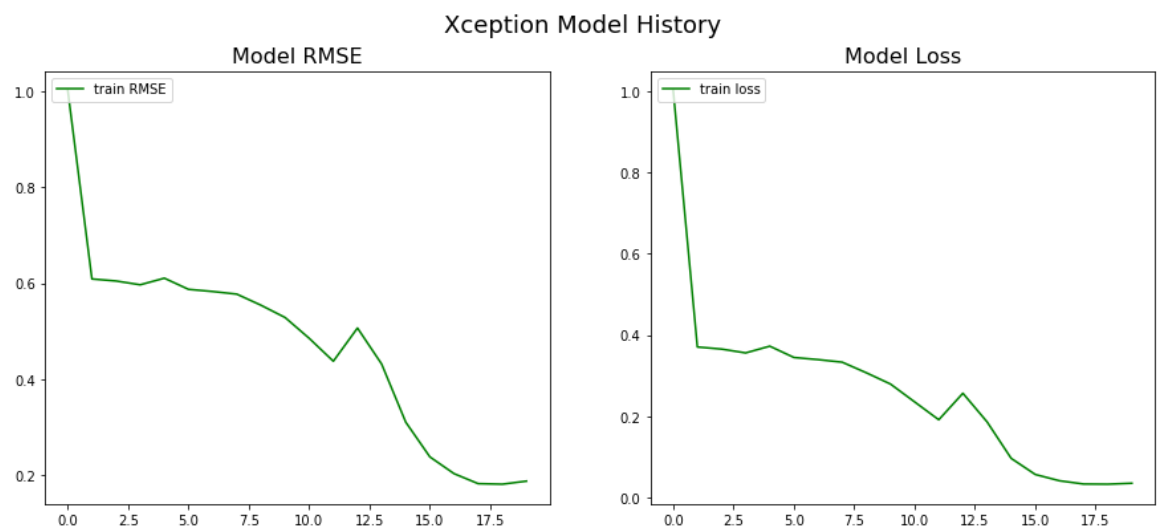


Рисунок 3.12 Історія навчання Моделі-6

Модель-7. Модель-7 мала в собі за базову модель модель Xception. Різниця між даною і Моделлю-6 у тому, що тут збільшений розмір партії зображень до 64. Тут також були розморожені шари, тому модель має більше шарів для навчання (рис. 3.13). На рис. 3.14 зображено історія навчання мережі.

Model: "model"

Layer (type)	Output Shape	Param #
=====		
input_2 (InputLayer)	[(None, 150, 150, 3)]	0

xception (Functional)	(None, 5, 5, 2048)	20861480

global_average_pooling2d (Gl	(None, 2048)	0

dense (Dense)	(None, 1)	2049
=====		
Total params: 20,863,529		
Trainable params: 20,809,001		
Non-trainable params: 54,528		

Рисунок 3.13 Архітектура Моделі-7

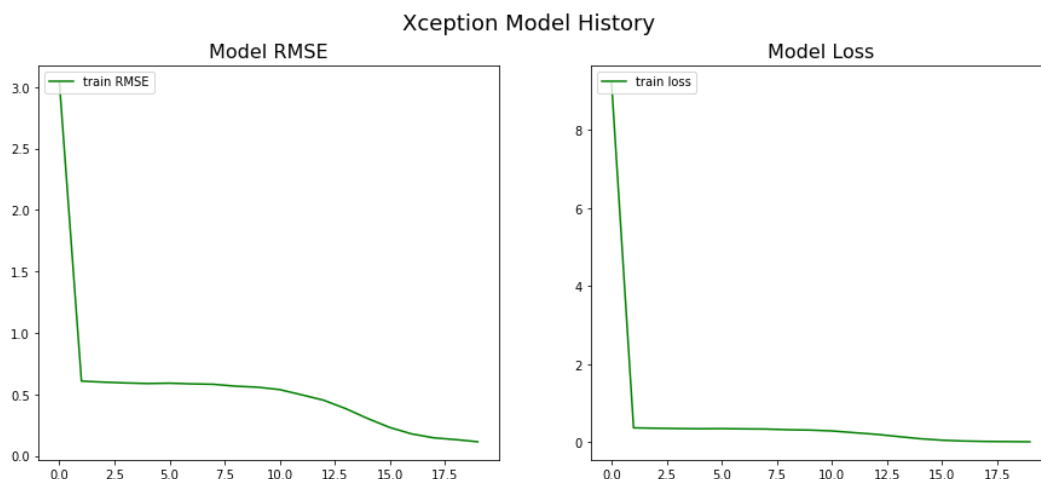


Рисунок 3.14 Історія навчання Моделі-7

Аналізуючи отримані результати, вирішено спробувати застосувати трансформери. Результати експериментів №2 представлені у Табл. 3.2.

Таблиця 3.2 Результати експериментів №2

№ з/п	Назва параметру	Назва моделі		
		Модель №8	Модель №9	Модель №10
1	Дата	19.11.2021 23:02:08	20.11.2021 22:22:47	14.12.2021 18:49:35
2	Час роботи коду	3412.8s - GPU	2286.9s - GPU	4740.7s - GPU
3	Private Score	17.31907	17.54421	20.54231
4	Public Score	18.32707	18.32592	20.55770
5	Бібліотека	fastai		
6	“Pawpularity” data transform	normed_score (Pawpularity/100)		
7	Fraction (val data)	0.2		
8	batch_size	32	16	
9	num_workers	8		
10	item_tfms	224		
12	y_block	RegressionBlock		
13	pretrained	True		False
14	rmse	res = 100 * torch.sqrt(F.mse_loss(F.sigmoid(input.flatten()), target))		
15	epochs	10		
16	learning rate	2e-5		9.99e-07
17	patience	3		
18	Результат	Succeeded		

Модель-8. Дана модель мала за базову модель Swin Transformer. Результати: Private Score = 17.31907, Public Score = 18.32707. Як бачимо, є покращення у порівнянні із використанням Xception. Опишемо основні етапи коду.

1. Спочатку було проаналізовано колонку “Pawpularity”, у якій записано відповідні оцінки привабливості тварини із притулку за фото. На рис. 3.15 представлено основні статистичні характеристики даної оцінки (медіана, середнє значення та середньоквадратичне відхилення). Далі було побудовано гістограму із розподілом “Pawpularity” (рис. 3.16).



Рисунок 3.15 Статистичні характеристики

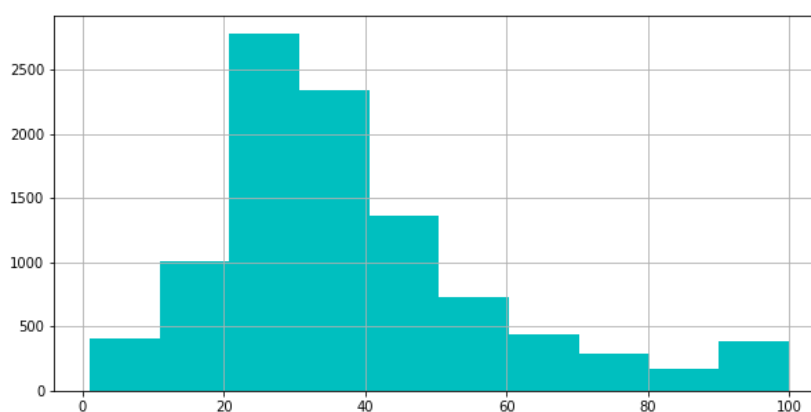


Рисунок 3.16 Гістограма розподілу “Pawpularity”

2. Пораховано кількість унікальних значень “Pawpularity” у наборі (рис. 3.17).

```
unique_values = len(df_train['Pawpularity'].unique())
unique_values

100
```

Рисунок 3.17 Кількість унікальних значень “Pawpularity” у наборі даних

3. Після цього цільове значення “Pawpularity” було нормалізоване (рис. 3.18).

```
# normalize data
df_train['norm_score'] = df_train['Pawpularity'] / Pawpularity_max
df_train['norm_score']
```

Рисунок 3.18 Нормалізовані значення “Pawpularity”

4. Тоді завантажуються зображення (рис. 3.19).

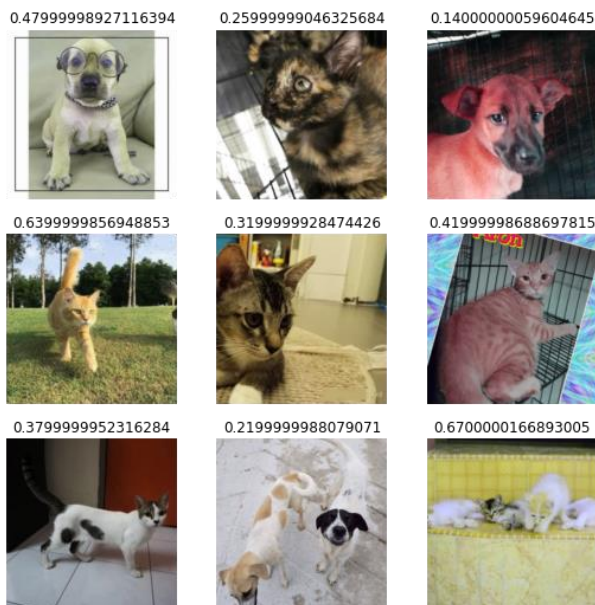


Рисунок 3.19 Приклад партії зображень

5. Після цього навчаємо Swin Transformer (рис. 3.20), моніторимо значення 'rmse_petfinder'. Історія навчання представлена на рис. 3.21.

epoch	train_loss	valid_loss	rmse_petfinder	time
0	0.659039	0.652504	19.118677	06:37
1	0.644083	0.638864	17.434219	06:37
2	0.640309	0.639228	17.441889	06:37
3	0.636847	0.640044	17.539978	06:37
4	0.631934	0.638606	17.376192	06:36
5	0.622020	0.641703	17.778921	06:36
6	0.616147	0.642486	17.806934	06:35
7	0.610556	0.643522	17.901937	06:34

Рисунок 3.20 Навчання Моделі-8

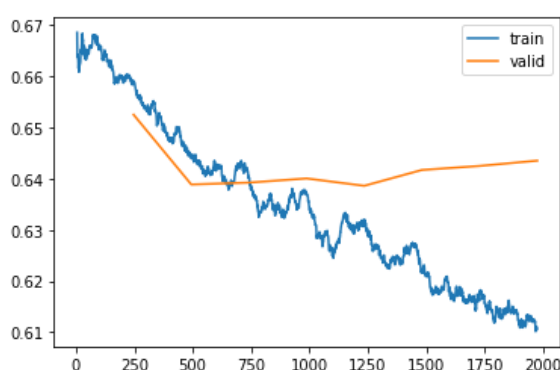


Рисунок 3.21 Історія навчання Моделі-8

Модель-9. Використовує ту ж архітектуру, що і Модель-8. Результати: Private Score = 17.54421, Public Score = 18.32592. Навчання мережі представлено на рис. 3.22. Історія навчання представлена на рис. 3.23.

Модель-10. Дана модель використовувала також архітектуру Swin Transformer. Private Score = 20.54231, Public Score = 20.55770. Навчання мережі представлено на рис. 3.24. Історія навчання представлена на рис. 3.25.

epoch	train_loss	valid_loss	rmse_petfinder	time
0	0.655828	0.648852	18.534834	07:04
1	0.639718	0.638185	17.140970	07:03
2	0.636705	0.641092	17.441559	07:04
3	0.635432	0.639459	17.259691	07:04
4	0.627469	0.639372	17.301783	07:03

Рисунок 3.22 Навчання Моделі-9

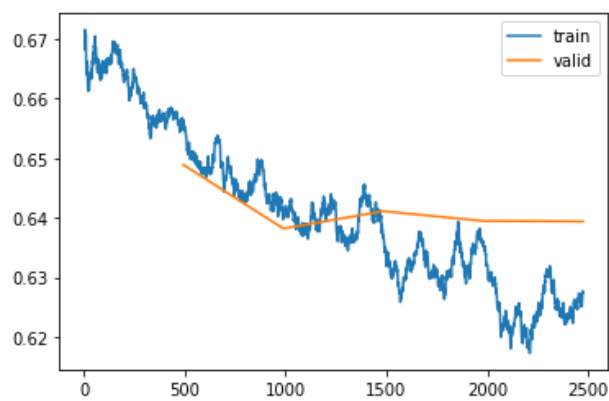


Рисунок 3.23 Історія навчання Моделі-9

epoch	train_loss	valid_loss	rmse_petfinder	time
0	0.667558	0.659335	20.317188	07:35
1	0.667881	0.663090	20.542372	07:34
2	0.662922	0.659471	20.282778	07:33
3	0.666985	0.659936	20.284552	07:34
4	0.663823	0.659223	20.268068	07:34
5	0.667746	0.661763	20.674791	07:34
6	0.661936	0.659296	20.261015	07:32
7	0.661488	0.659245	20.274170	07:33
8	0.665090	0.659346	20.306673	07:33
9	0.664069	0.659262	20.281128	07:33

Рисунок 3.24 Навчання Моделі-10

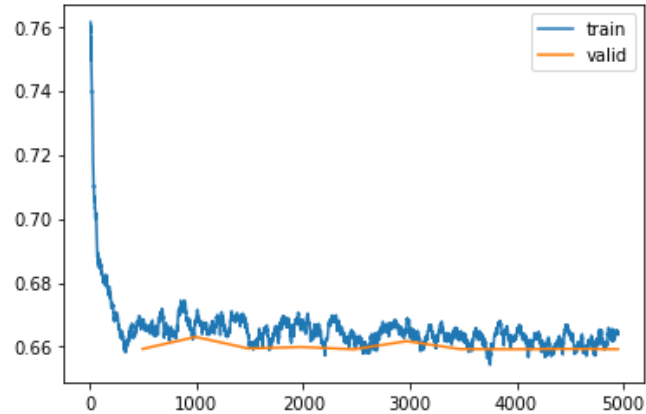


Рисунок 3.25 Історія навчання Моделі-10

Аналізуючи отримані результати, бачимо, що використання Swin Transformer покращило фінальні результати. Але, вирішено знову використати згорткову нейронну мережу з іншими налаштуваннями, беручи до уваги передобробку даних, що була використана для Swin Transformer (експерименти №2). Результати експериментів №3 представлені у Табл. 3.3.

Таблиця 3.3 Результати експериментів №3

№ з/п	Назва параметру	Назва моделі			
		Модель №11	Модель № 12	Модель № 13	Модель № 14
1	Дата	15.12.2021 20:05:38	15.12.2021 21:04:47	15.12.2021 22:17:52	16.12.2021 09:38:05
2	Час роботи коду	2539.1s - GPU	2835.6s - GPU	5102.6s - GPU	5077.8s - GPU
3	Private Score	20.51204	25.68816	25.10287	24.45946
4	Public Score	20.50848	26.10331	24.83669	25.03970
5	Бібліотека	tensorflow.keras			
6	“Pawpularity” data transform	normed_score (Pawpularity/100)			
7	image_size	(150, 150)			
8	batch_size	64			
9	epochs	20		40	
	Fraction (val data)	0.2			0.05
10	Базова модель	Xception			
11	base_model.trainable	True			
12	Останній шар	Dense(1,activation='sigmoid')			
13	Learning rate	0.001	1.25e-5		
14	rmse	rmse_res = K.sqrt(mse(Pawpularity_max*y_true, Pawpularity_max*y_pred))			
15	Результат	Succeeded			

Модель-11. Модель-11 використовує архітектуру згорткової нейронної мережі Xception, але підхід до передобробки даних був вже взятий із підходу, що застосовувався у експериментах №2. Також, змінилася функція активація на останньому шарі. Більше того, видозмінився підрахунок формули *RMSE*. Також, для кращого розуміння, було взято декілька різних метрик. Private Score = 20.51204, Public Score = 20.50848. Опишемо основні етапи написання коду.

1. Спочатку було проаналізовано набір даних та представлено основні статистичні характеристики даної оцінки як і в попередніх експериментах.
2. Потім було пораховано кількість унікальних значень і нормалізовано "Pawpularity" так, як це було на попередніх кроках.
3. На рис. 3.26 представлено архітектуру даної моделі.

```
Model: "Xception"
```

Layer (type)	Output Shape	Param #
xception (Functional)	(None, 5, 5, 2048)	20861480
global_average_pooling2d (Gl	(None, 2048)	0
dense (Dense)	(None, 100)	204900
batch_normalization_4 (Batch	(None, 100)	400
dropout (Dropout)	(None, 100)	0
dense_1 (Dense)	(None, 50)	5050
batch_normalization_5 (Batch	(None, 50)	200
dense_2 (Dense)	(None, 1)	51

```

Total params: 21,072,081
Trainable params: 21,017,253
Non-trainable params: 54,828

```

Рисунок 3.26 Архітектура Моделі-11

4. На рис. 3.27 представлено метрики, які було взято для навчання даної мережі.

```
metrics=[tf.keras.metrics.MeanSquaredError(),
          tf.keras.metrics.RootMeanSquaredError(),
          tf.keras.metrics.MeanAbsoluteError(),
          tf.keras.metrics.MeanAbsolutePercentageError(),
          tf.keras.metrics.MeanSquaredLogarithmicError(),
          tf.keras.metrics.CosineSimilarity(axis=1),
          tf.keras.metrics.LogCoshError(),
          ])
```

Рисунок 3.27 Метрики мережі

Модель-12. Модель-12 подібна до Моделі-11. За винятком іншого коефіцієнта навчання. Результати: Private Score = 25.68816, Public Score = 26.10331. Як бачимо, результати стали ще гірші у порівнянні із попередньою моделлю.

Модель-13. Модель-13 подібна до попередніх двох. Було збільшено кількість епох, щоб подивитися, чи не застряє нейронна мережа у локальному оптимумі. Результати: Private Score = 25.10287, Public Score = 24.83669. Але, як видно із отриманих результатів, суттєвого покращення не відбулося навіть при збільшенні кількості епох.

Модель-14. Модель-14 подібна до Моделей 11 - 13. Змінено поділ навчальної до тестової вибірок, так як відомо, що якщо спробувати збільшити навчальний набір даних, то мережа може бути більш краще навченою. Результати: Private Score = 24.45946, Public Score = 25.03970. Але, як видно із отриманих результатів, суттєвого покращення не відбулося навіть при збільшенні розміру навчального набору даних епох. На рис. 3.28 представлено історію навчання даної моделі.

Отримані результати експериментів №3 свідчать, що у порівнянні із експериментами №2, застосовані згорткові нейронні мережі показують гірші результати, чим Swin Transformer. Тому, було вирішено продовжити експерименти, але лише із використанням Swin Transformer.

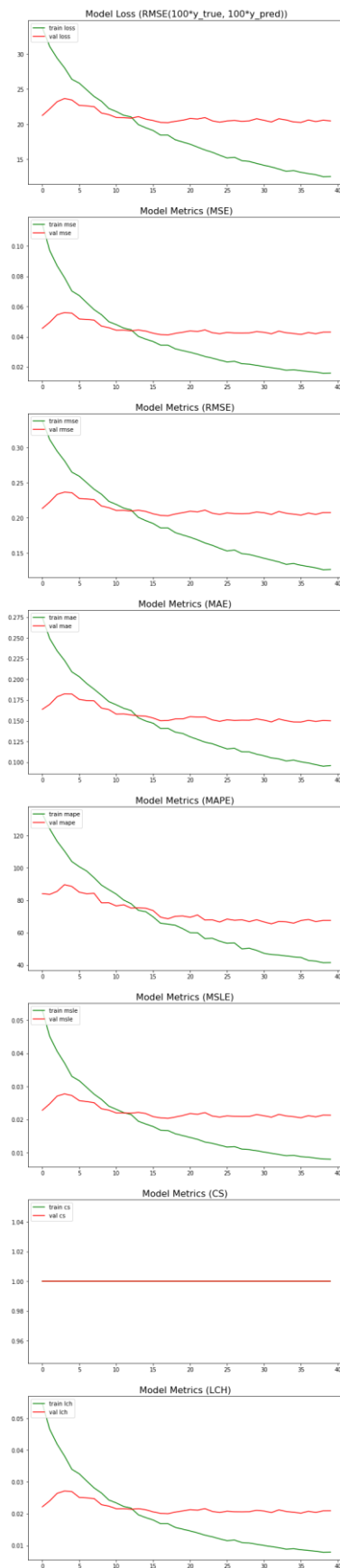


Рисунок 3.29 Історія навчання Моделі-14

Розглянемо наступні експерименти. У них використовується Swin Transformer. Також, до цих експериментів застосовано ідею K-FOLDS. Для зручності відображення, розіб'ємо дану частину на декілька таблиць, так як саме з використанням методу K-FOLDS було виконано найбільше експериментів. Відомо, що для аналізу даних та подальшого вдалого поділу на навчальну та тестову вибірки методом K-FOLDS, необхідно щоб набір даних був поділений на групи. Тоді, маючи групи, метод K-FOLDS, а саме його різновид Stratified K-FOLDS, зможе поділити набір даних стратифікованим чином, тобто, щоб у навчальній та тестовій вибірці зберігалася пропорція відповідних елементів із кожної групи. У задачі класифікації класи виступають групами (наприклад, може бути n штук елементів класу-1, m штук елементів класу-2, k штук елементів класу-3 тощо). Так як прогнозування привабливості тварин із притулку за фото відноситься до задач регресії, то нам необхідно логічно розділити весь набір даних на групи за певною ознакою. Одним із варіантів, щоб виконати такий поділ, є такий, що переводить точкову оцінку (наприклад, привабливість тварини за фото = 98.5%) у інтервальну (наприклад, інтервал, куди будуть відноситися всі фотографії із привабливістю [90%;100%]). Таким чином, зможемо розділити весь набір даних на групи (інтервали). Для поділу елементів на інтервали є багато різних методів. Саме тут використовуються два найвідоміших - Rice's Rule (формула 3.1) та Sturges' rule (формула 3.2). За допомогою цих формул можна порахувати оптимальну кількість інтервалів (bins) для набору даних. Перша частина результатів експериментів №4 представлена у Табл. 3.4.

$$K = \sqrt[3]{n} * 2, \quad (3.1)$$

$$K = 1 + 3.322 \log n, \quad (3.2)$$

де K – кількість інтервалів, n – кількість вхідних даних. У нашому випадку, n – це розмір набору даних.

Таблиця 3.4 Результати експериментів №4

№ з/п	Назва параметру	Назва моделі				
		Модель-15	Модель-16	Модель-17	Модель-18	Модель-19
1	Дата	21.12.2021 21:08:44	22.12.2021 22:19:04	23.12.2021 21:28:54	28.12.2021 14:35:28	29.12.2021 13:59:56
2	Час роботи коду	10228.5s - GPU	3176.0s - GPU	6784.9s - GPU	17911.1s - GPU	17923.7s - GPU
3	Private Score	17.16056	17.48014	17.22240	17.16304	17.16304
4	Public Score	17.96472	18.23046	17.85096	17.80600	17.80600
5	Бібліотека	fastai				
6	“Pawpularity” data transform	normed_score (Pawpularity/100)				
7	seed	999				
8	num_bins	Rice rule	Sturges' rule			
9	N_FOLDS	5	7	3	7	
10	rmse	res = torch.sqrt(F.mse_loss(Pawpularity_max*F.sigmoid(input_.flatten()), Pawpularity_max*target))				
11	Fraction (val data)	0.2				
12	batch_size	32	8			
13	num_workers	8				
14	item_tfms	224				
15	batch_tfms	setup_aug_tfms([Brightness(), Contrast(), Hue(), Saturation()])				
16	y_block	RegressionBlock				
17	pretrained	True				
18	epochs per FOLD	5	10	5		
19	learning rate	2e-5				
20	patience	2	3			
21	Фінальна оцінка	Середнє значення оцінок кожної із Fold моделей				
	Результат	Succeeded				

Модель-15. Модель-15 - це перша модель, що використовувала в собі метод K-FOLDS для навчання мережі. Результати: Private Score = 17.16056, Public Score = 17.96472. Опишемо основні етапи.

1. Аналізуємо набір даних. (статистичні показники, гістограма розподілу значень “Rawpularity”, кількість унікальних значень). Нормуємо “Rawpularity”. Ці кроки такі ж, як і у попередніх експериментах.
2. Для того, щоб поділити набір даних із точковою оцінкою “Rawpularity” за допомогою методу StratifiedKFold, нам спочатку потрібно точкову оцінку “Rawpularity” перетворити у інтервальну (бо інтервальна оцінка вже може бути інтерпретована як клас). Для вибору оптимальної кількості інтервалів (bins) в гістограмі скористаємося відомим правилом Rice’s Rule (рис. 3.29-3.30).

```
# Rice rule
num_bins = int(np.ceil(2*((len(df_train))**(1./3))))
num_bins

43
```

Рисунок 3.29 Поділ набору даних на інтервали Rice’s rule

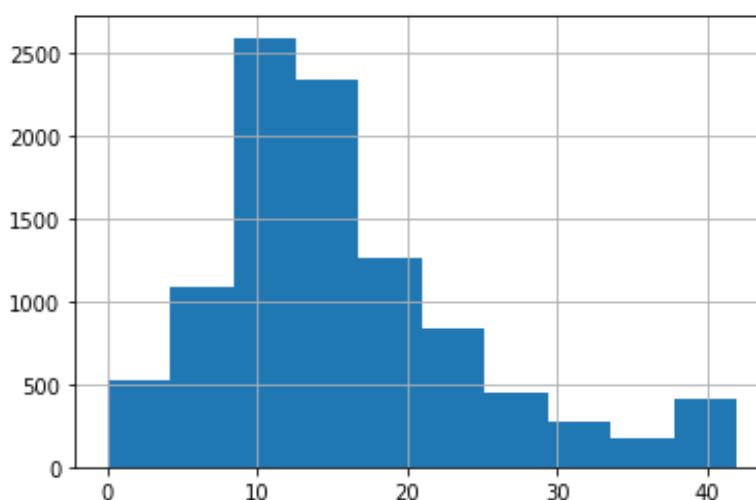


Рисунок 3.30 Переведення точкової оцінки “Rawpularity” в інтевальну

3. Після того, як точкову оцінку “Rawpularity” переведено в інтевальну, ділимо за допомогою StratifiedKFold набір даних на частини, $N_FOLDS = 5$ (рис 3.31).

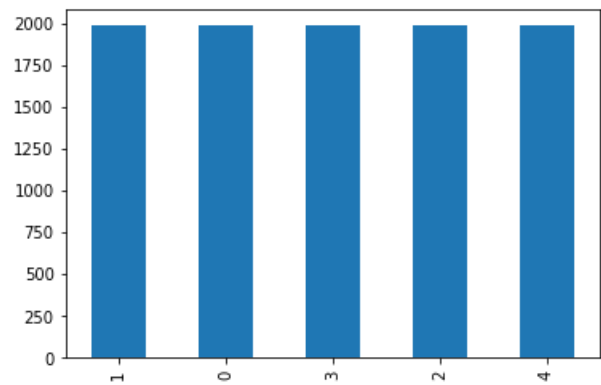


Рисунок 3.31 Кількість елементів у кожному Fold

4. Результати навчання для кожного $Fold_i, i \in \overline{0,4}$ наведено на рис. 3.32-3.36. Історія навчання наведена на рис. 3.37.
5. Фінальні результати рахуються як середнє значення $Fold_i, i \in \overline{0,4}$.

Fold 0 results

epoch	train_loss	valid_loss	rmse	time
0	0.653848	0.645281	18.077417	06:35
1	0.644681	0.641541	17.565077	06:35
2	0.635093	0.641712	17.544096	06:35
3	0.630722	0.641953	17.504389	06:35
4	0.629036	0.642481	17.561075	06:35

Рисунок 3.32 Результати навчання ($Fold_0$) Моделі-15

Fold 1 results

epoch	train_loss	valid_loss	rmse	time
0	0.650013	0.644604	18.033939	06:35
1	0.641108	0.639194	17.253284	06:35
2	0.635048	0.636977	16.976000	06:35
3	0.630024	0.637135	16.986944	06:34
4	0.628438	0.637396	17.029650	06:34

Рисунок 3.33 Результати навчання ($Fold_1$) Моделі-15

Fold 2 results				
epoch	train_loss	valid_loss	rmse	time
0	0.650185	0.646473	18.138243	06:35
1	0.639680	0.644164	17.804115	06:34
2	0.635677	0.646854	18.064281	06:34
3	0.630722	0.643762	17.773474	06:32
4	0.627171	0.643642	17.718702	06:34

Рисунок 3.34 Результати навчання ($Fold_2$) Моделі-15

Fold 3 results				
epoch	train_loss	valid_loss	rmse	time
0	0.648391	0.645691	18.074539	06:36
1	0.642952	0.642498	17.686152	06:35
2	0.633677	0.641245	17.481773	06:35
3	0.632187	0.640882	17.430136	06:35
4	0.628367	0.641021	17.461569	06:35

Рисунок 3.35 Результати навчання ($Fold_3$) Моделі-15

Fold 4 results				
epoch	train_loss	valid_loss	rmse	time
0	0.650769	0.646705	18.167580	06:35
1	0.642806	0.643459	17.758022	06:35
2	0.632622	0.642578	17.648285	06:34
3	0.629221	0.642917	17.640223	06:35
4	0.625429	0.642542	17.615326	06:35

Рисунок 3.36 Результати навчання ($Fold_4$) Моделі-15

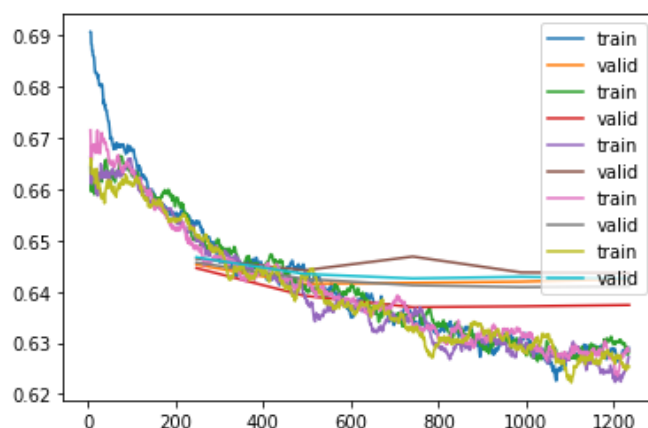


Рисунок 3.37 Історія навчання Моделі-15

Модель-16. У Моделі-16 замість Rice's Rule використано Sturges' rule (рис. 3.38). Поділено на 7 Folds. Розмір партії зменшено до 8. І кількість епох на один Fold збільшено до 10. Результати: Private Score = 17.48014, Public Score = 18.23046. На рис. 3.39 зображено історію навчання.



Рис. 3.38 Sturges' rule

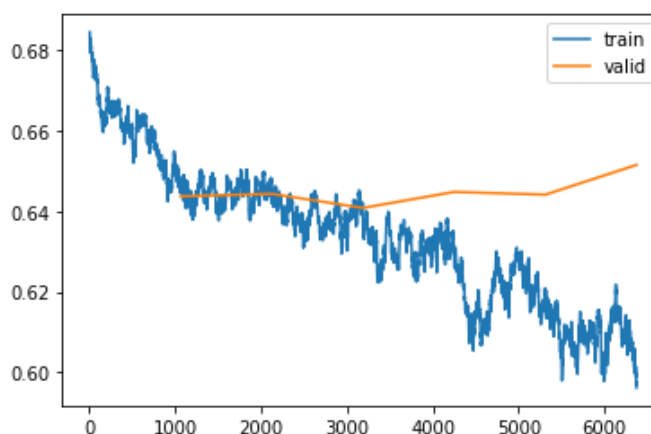


Рисунок 3.39 Історія навчання Моделі-16

Модель-17. Модель-17 використала поділ набору даних на 3 Folds. Результати: Private Score = 17.22240, Public Score = 17.85096. На рис. 3.40 зображено історію навчання.

Модель-18. Модель-18 використала поділ набору даних на 7 Folds. Результати: Private Score = 17.16304, Public Score = 17.80600. На рис. 3.41 зображено історію навчання.

Модель-19. Модель-19 використала поділ набору даних на 7 Folds. Результати: Private Score = 17.16304, Public Score = 17.80600. На рис. 3.42 зображено історію навчання. Ця модель є аналогічною до Моделі-18.

Наступна частина результатів експериментів №4 представлені у Табл. 3.5.

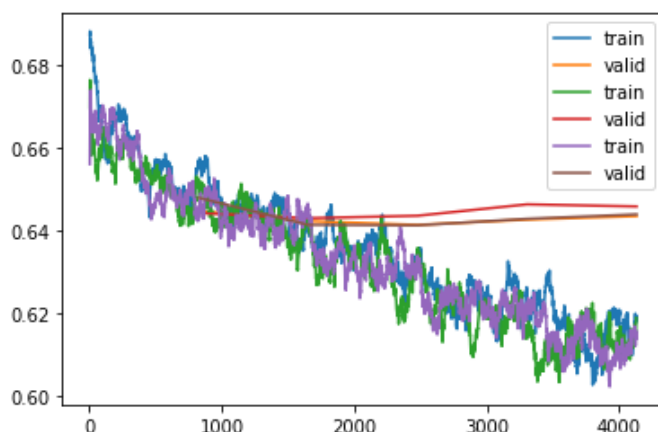


Рисунок 3.40 Історія навчання Моделі-17

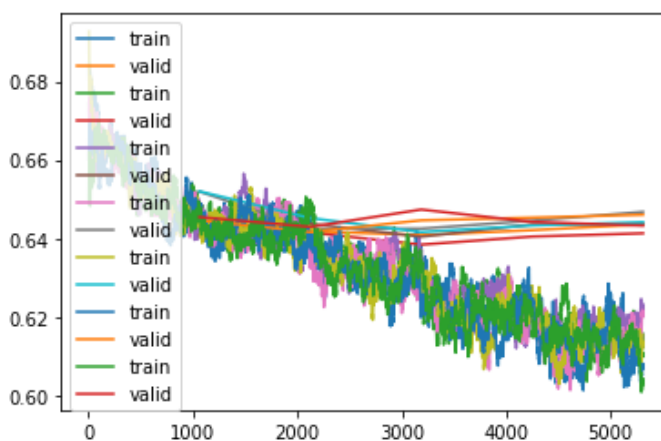


Рисунок 3.41 Історія навчання Моделі-18

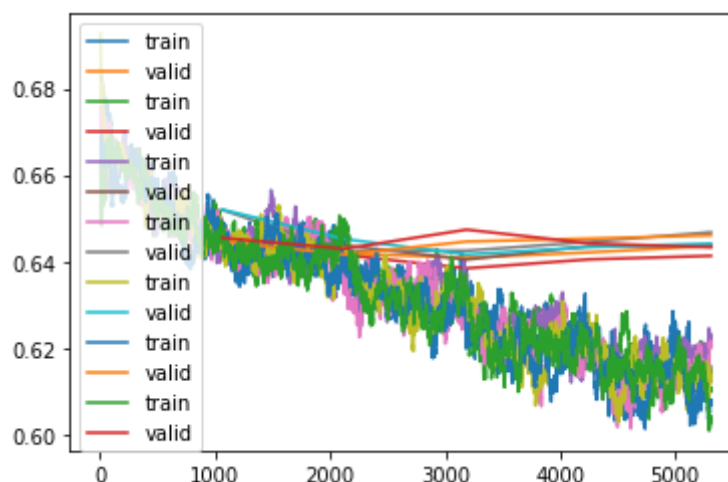


Рисунок 3.42 Історія навчання Моделі-19

Таблиця 3.5 Результати експериментів №4 (продовження)

№ з/п	Назва параметру	Назва моделі				
		Модель-20	Модель-21	Модель-22	Модель-23	Модель-24
1	Дата	30.12.2021 10:26:00	31.12.2021 09:02:42	01.01.2022 20:50:55	02.01.2022 21:14:26	03.01.2022 23:10:08
2	Час роботи коду	21423.1s - GPU	25567.7s - GPU	20498.9s - GPU	20392.9s - GPU	22943.3s - GPU
3	Private Score	17.12543	17.13021	17.11781	17.63489	17.14309
4	Public Score	17.86297	17.85239	17.83321	18.08927	17.85644
5	Бібліотека	fastai				
6	“Pawpularity” data transform	normed_score (Pawpularity/100)				
7	seed	999				
8	num_bins	Sturges' rule				
9	N_FOLDS	7	9	7		7
10	rmse	res = torch.sqrt(F.mse_loss(Pawpularity_max*F.sigmoid(input_.flatten()), Pawpularity_max*target))				
11	Fraction (val data)	0.2		0.1		
12	batch_size	8				
13	num_workers	8				
14	item_tfms	224				
15	batch_tfms	setup_aug_tfms([Brightness(), Contrast(), Hue(), Saturation()])				

Продовження таблиці 3.5

№ з/п	Назва параметру	Назва моделі				
		Модель-20	Модель-21		Модель-22	Модель-23
16	y_block	RegressionBlock				
17	pretrained	True				
18	epochs per FOLD	7	6		8	7
19	learning rate	1.5e-5	2e-5		1e-4	1e-5
20	patience	3	2	3	3	3
21	Фінальна оцінка	Середнє значення оцінок кожної із Fold моделей				
22	Результат	Succeeded				

Модель-20. Модель-20 використала поділ набору даних на 7 Folds. Результати: Private Score = 17.12543, Public Score = 17.86297. На рис. 3.43 зображено історію навчання.

Модель-21. Модель-21 використала поділ набору даних на 9 Folds. Результати: Private Score = 17.13021, Public Score = 17.85239. На рис. 3.44 зображено історію навчання.

Модель-22. Модель-22 використала поділ набору даних на 7 Folds. Результати: Private Score = 17.11782, Public Score = 17.83321. На рис. 3.45 зображено історію навчання.

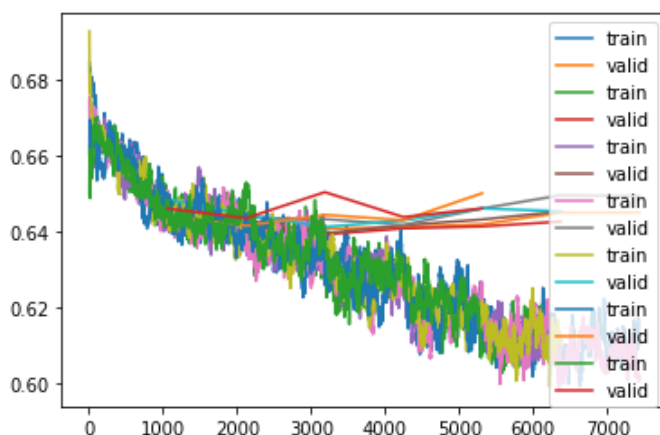


Рисунок 3.43 Історія навчання Моделі-20

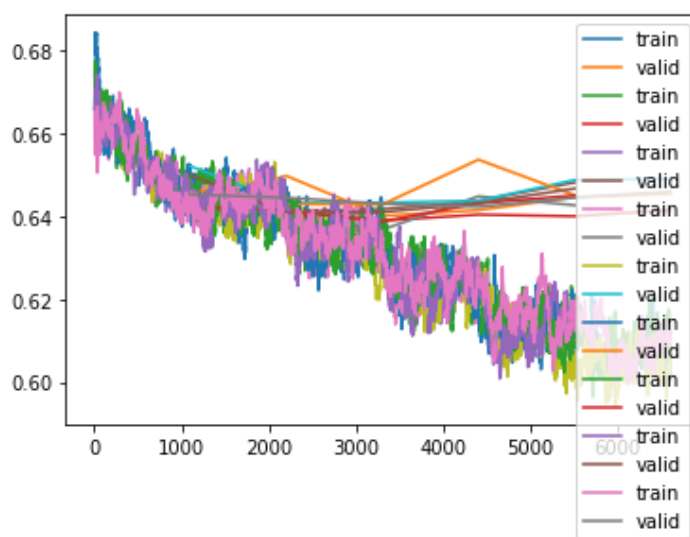


Рисунок 3.44 Історія навчання Моделі-21

Модель-23. Модель-23 використала поділ набору даних на 7 Folds. Результати: Private Score = 17.63489, Public Score = 18.08927. На рис. 3.46 зображено історію навчання.

Модель-24. Модель-24 використала поділ набору даних на 7 Folds. Результати: Private Score = 17.14309, Public Score = 17.85645. На рис. 3.47 зображено історію навчання.

Наступна частина результати експериментів №4 представлені у Табл. 3.6.

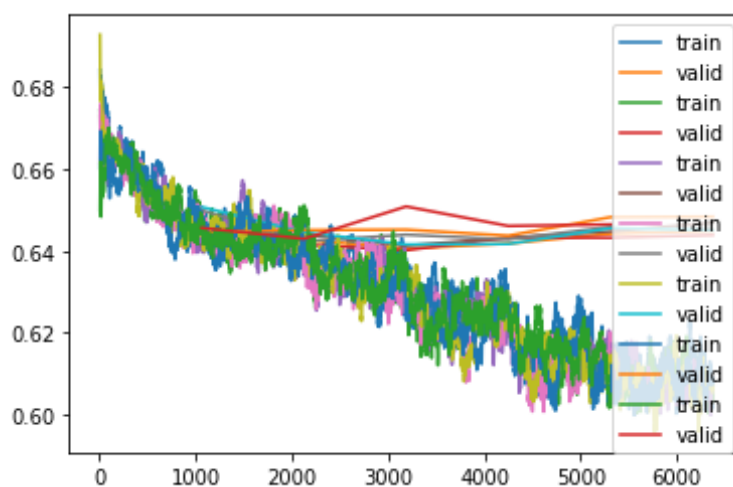


Рисунок 3.45 Історія навчання Моделі-22

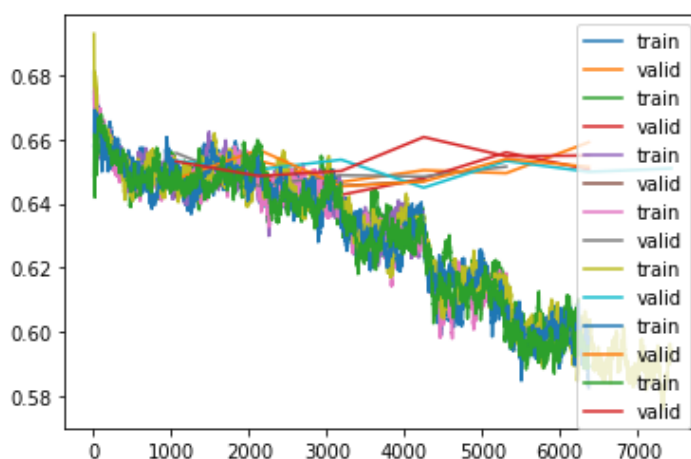


Рисунок 3.46 Історія навчання Моделі-23

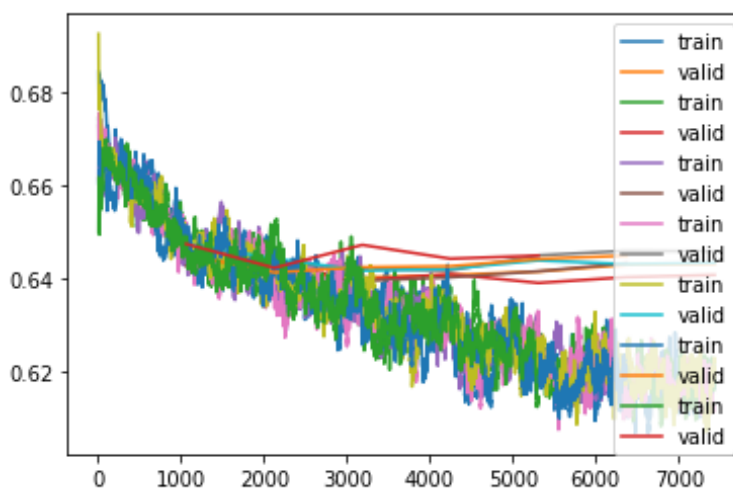


Рисунок 3.47 Історія навчання Моделі-24

Таблиця 3.6 Результати експериментів №4 (продовження)

№ з/п	Назва параметру	Назва моделі				
		Модель-25	Модель-26	Модель-27	Модель-28	Модель-29
1	Дата	04.01.2022 19:19:59	05.01.2022 18:58:46	07.01.2022 23:16:28	08.01.2022 10:10:59	08.01.2022 20:59:21
2	Час роботи коду	31557.0s - GPU	21476.7s - GPU	3592.2s - GPU	15819.1s - GPU	15949.8s - GPU
3	Private Score	-	17.24364	17.31046	17.23398	17.29332
4	Public Score	-	17.93117	18.15309	17.90847	17.92456

Продовження таблиці 3.6

№ з/п	Назва параметру	Назва моделі				
		Модель-25	Модель-25	Модель-25	Модель-25	Модель-25
5	Бібліотека	fastai				
6	“Pawpularity” data transform	normed_score (Pawpularity/100)				
7	seed	999				
8	num_bins	Sturges' rule				
9	N_FOLDS	10	7	10		
10	rmse	res = torch.sqrt(F.mse_loss(Pawpularity_max*F.sigmoid(input_.flatten()), Pawpularity_max*target))				
11	Fraction (val data)	0.1				
12	batch_size	8				
13	num_workers	8				
14	item_tfms	224				
15	batch_tfms	setup_aug_tfms([Brightness(), Contrast(), Hue(), Saturation()])				
16	y_block	RegressionBlock				
17	pretrained	True				
18	epochs per FOLD	6				
19	learning rate	5e-5	5e-6			
20	patience	3				
21	Фінальна оцінка	Середнє значення оцінок кожної із Fold моделей	Навчали і зберігали результати лише $Fold_i, i = 4$	Навчали і зберігали результати лише $Fold_i, i \in \overline{0, 4}$	Навчали $Fold_i, i \in \overline{0, 4}$, а зберігали лише $Fold_i, i \in \overline{1, 4}$	
22	Результат	Notebook Timeout	Succeeded			

Модель-25. Модель-25 використала поділ набору даних на 10 Folds, що призвело до збільшення навчання нейронної мережі. Модель-22 навчилися, це зайняло майже 9 годин. І вже, при поданні на конкурс, коли код автоматично запускається ще раз з початку, виникла помилка часу виконання, тому що є

обмеження на час роботи коду на Kaggle (максимально 9 годин). На рис. 3.48 зображено історію навчання.

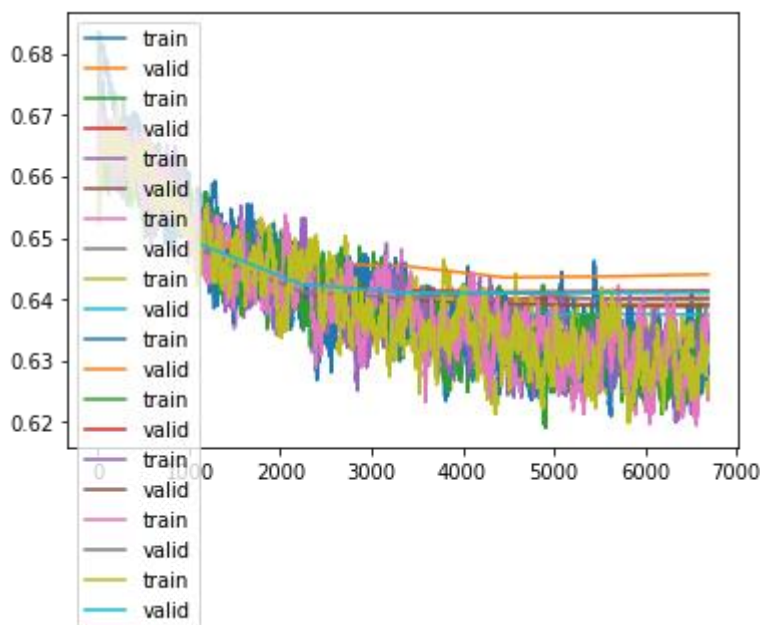


Рисунок 3.48 Історія навчання Моделі-25

Модель-26. Модель-26 використала поділ набору даних на 7 Folds. Результати: Private Score = 17.24364, Public Score = 17.93117. На рис. 3.49 зображено історію навчання.

Модель-27. Модель-27 використала поділ набору даних на 10 Folds. Але код запускався не для всіх Folds, а лише для $Fold_i, i = 4$. Результати: Private Score = 17.31047, Public Score = 18.15309. На рис. 3.50 зображено історію навчання.

Модель-28. Модель-28 використала поділ набору даних на 10 Folds. Але код запускався не для всіх Folds, а лише для $Fold_i, i \in \overline{0,4}$. Результати: Private Score = 17.23398, Public Score = 17.90847. На рис. 3.51 зображено історію навчання.

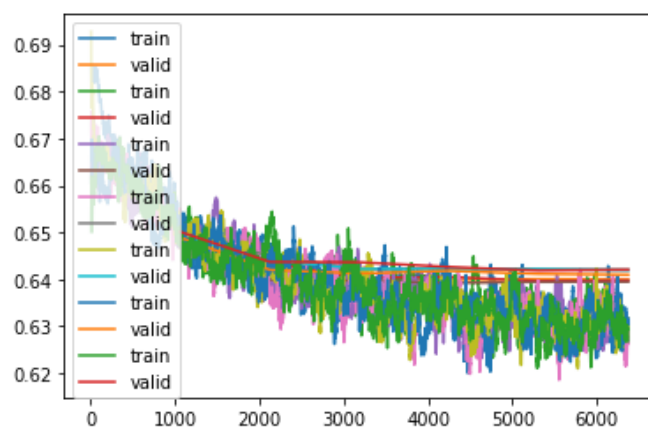


Рисунок 3.49 Історія навчання Моделі-26

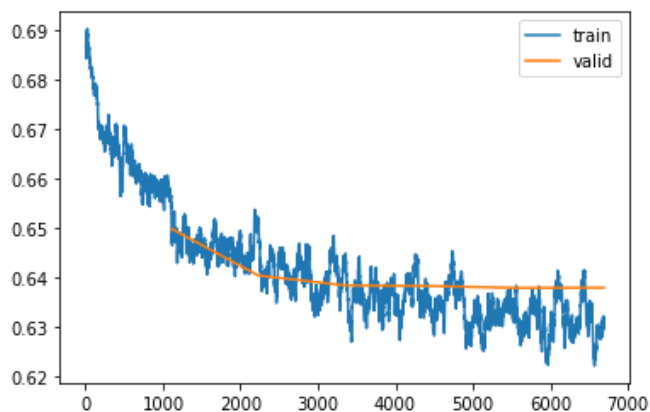


Рисунок 3.50 Історія навчання Моделі-27

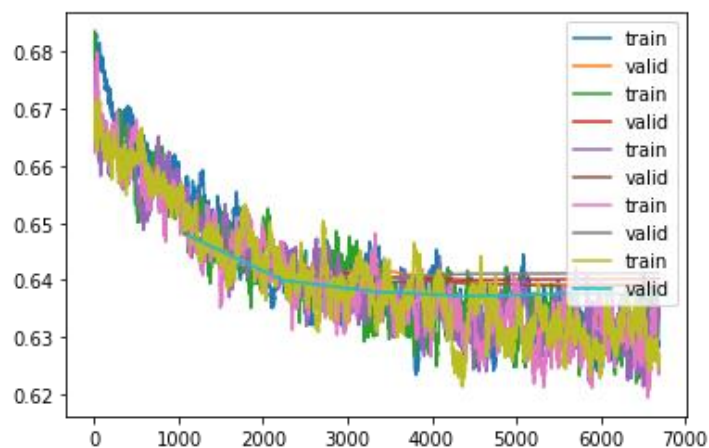


Рисунок 3.51 Історія навчання Моделі-28

Модель-29. Модель-29 використала поділ набору даних на 10 Folds. Але код запускався не для всіх Folds, а лише для $Fold_i, i \in \overline{0,4}$ і зберігали лише $Fold_i, i \in \overline{1,4}$. Результати: Private Score = 17.29332, Public Score = 17.92456. На рис. 3.52 зображено історію навчання.

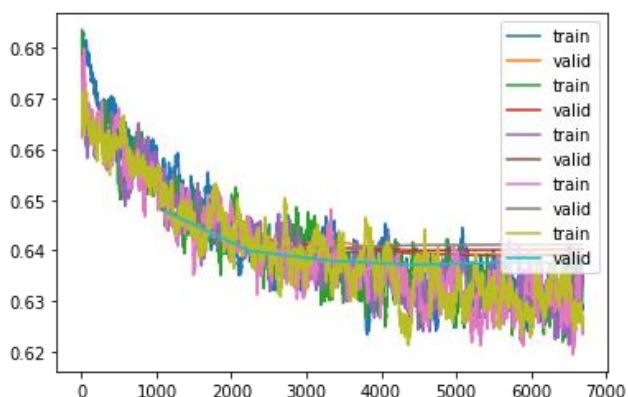


Рисунок 3.52 Історія навчання Моделі-29

Наступна частина результати експериментів №4 представлені у Табл. 3.7.

Таблиця 3.7 Результати експериментів №4 (продовження)

№ з/п	Назва параметру	Назва моделі			
		Модель-30	Модель-31	Модель-32	Модель-33
1	Дата	09.01.2022 17:15:29	11.01.2022 10:56:00	12.01.2022 01:15:29	12.01.2022 17:18:42
2	Час роботи коду	15800.8s - GPU	20563.3s - GPU	18006.5s - GPU	17186.5s - GPU
3	Private Score	17.23398	17.11781	17.45335	17.44020
4	Public Score	17.90847	17.83321	18.00456	17.92516
5	Бібліотека	fastai			
6	“Pawpularity” data transform	normed_score (Pawpularity/100)			
7	seed	999			
8	num_bins	Sturges' rule			
9	N_FOLDS	10		7	

Продовження таблиці 3.7

№ з/п	Назва параметру	Назва моделі			
		Модель-30	Модель-31	Модель-32	Модель-33
10	rmse	res = torch.sqrt(F.mse_loss(Pawpularity_max*F.sigmoid(input_.flatten()), Pawpularity_max*target))			
11	Fraction (val data)	0.2			
12	batch_size	8			
13	num_workers	8			
14	item_tfms	224			
15	batch_tfms	setup_aug_tfms([Brightness(), Contrast(), Hue(), Saturation()])			
16	y_block	RegressionBlock			
17	pretrained	True			
18	epochs per FOLD	6		5	
19	learning rate	5e-6	2e-5	3e-6	Для кожного Fold окремий learning rate, у діапазоні від 10e-6 до 10e-2
20	patience	3		2	
21	Фінальна оцінка	Навчали і зберігали результати лише $Fold_i, i \in \overline{0,4}$		Середнє значення оцінок кожної із Fold моделей	
22	Результат	Succeeded			

Модель-30. Модель-30 використала поділ набору даних на 10 Folds. Код запускався не для всіх Folds, а лише для $Fold_i, i \in \overline{0,4}$. Результати: Private Score = 17.23398, Public Score = 17.90847 На рис. 3.53 зображено історію навчання.

Модель-31. Модель-31 використала поділ набору даних на 7 Folds. Результати: Private Score = 17.11782, Public Score = 17.83321. На рис. 3.54 зображено історію навчання. Ця модель є аналогічною до Моделі-22.

Модель-32. Модель-32 використала поділ набору даних на 7 Folds. Результати: Private Score = 17.45335, Public Score = 18.00456. На рис. 3.55 зображено історію навчання.

Модель-33. Модель-33 використала поділ набору даних на 7 Folds. Результати: Private Score = 17.44020, Public Score = 17.92516. Це була остання модель, що подавалася на submission під час проведення конкурсу.

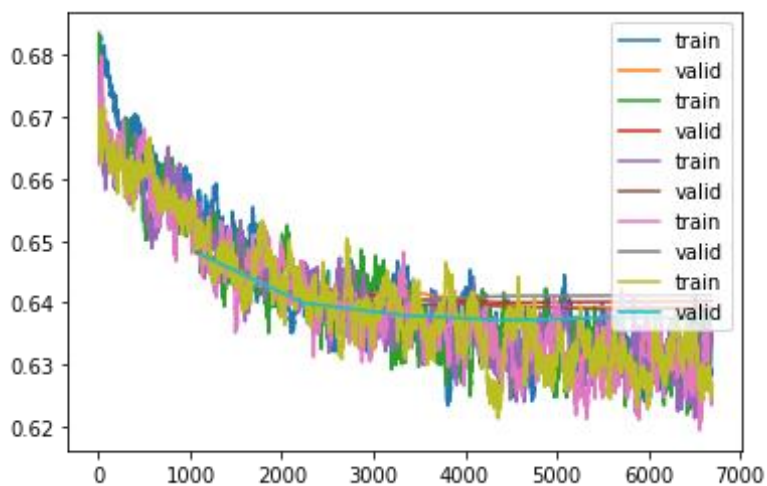


Рисунок 3.53 Історія навчання Моделі-30

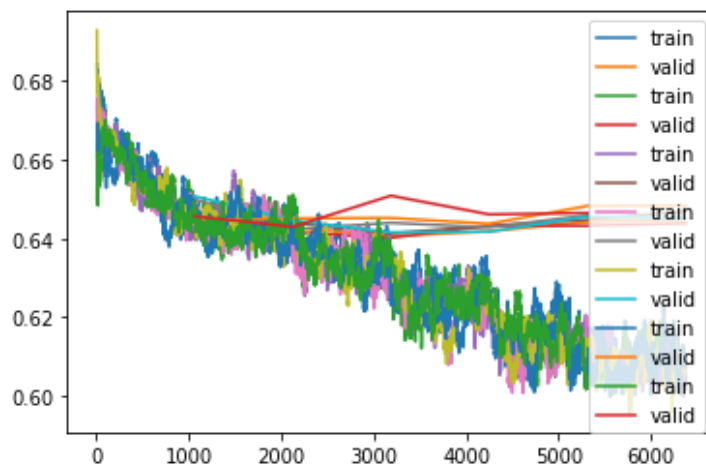


Рисунок 3.54 Історія навчання Моделі-31

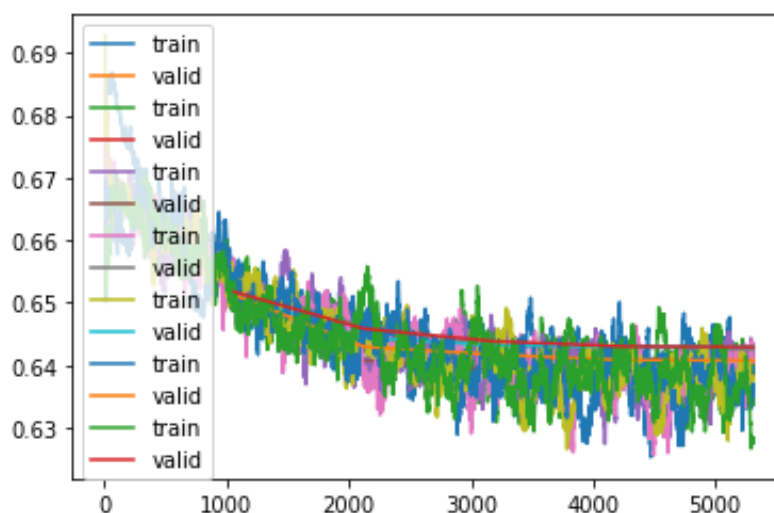


Рисунок 3.55 Історія навчання Моделі-32

Як бачимо, найкращі результати серед групи експериментів №4 показала Модель-31 із результатом Private Score = 17.11781 та Public Score = 17.83321.

Аналізуючи отримані графіки навчання моделей та їх *RMSE*, можна прийти до висновку, що навіть плавний спадаючий графік навчання моделей на відкритих даних не гарантує якісний результат Private Score та Public Score. Це пов'язано з тим, що Private Score та Public Score формуються на основі 75% та 25% прихованих фотографій із іншої вибірки відповідно. Тобто таких, на яких мережа не має можливості навчитися. Звідси слідує висновок, що навіть зменшення втрат навчання на відкритому наборі даних не гарантує точно такого ж результату на прихованому наборі даних.

Порівнюючи Private Score та Public Score між собою, можна сказати, що результат Private Score може бути більшим за Public Score і навпаки. Як приклад, можна навести Модель-18, яка має найменший Public Score = 17.80600 серед всіх моделей, але Private Score = 17.16304 у неї більший, чим у Моделі-31.

Також, було проведено експерименти №5 вже після закінчення конкурсу з метою покращити існуючі результати. Розглянемо їх у Табл. 3.8.

Таблиця 3.8 Результати експериментів №5

№ з/п	Назва параметру	Назва моделі
		Модель-34
1	Дата	15.04.2022 22:14:12
2	Час роботи коду	20217.6s - GPU
3	Private Score	17.08097
4	Public Score	17.90201
5	Бібліотека	fastai
6	“Pawpularity” data transform	normed_score (Pawpularity/100)
7	seed	999
8	num_bins	Sturges' rule
9	N_FOLDS	7
10	rmse	res = torch.sqrt(F.mse_loss(Pawpularity_max*F.sigmoid(input_.flatten()), Pawpularity_max*target))
11	Fraction (val data)	0.1
12	batch_size	8
13	num_workers	8
14	item_tfms	224
15	batch_tfms	setup_aug_tfms([Brightness(), Contrast(), Hue(), Saturation()])
16	y_block	RegressionBlock
17	pretrained	True
18	epochs per FOLD	6
19	learning rate	2e-5
20	patience	3
21	Фінальна оцінка	Середнє значення оцінок кожної із Fold моделей
22	Результат	Succeeded

Модель-34. Модель-34 використала поділ набору даних на 7 Folds. Результати: Private Score = 17.08097, Public Score = 17.90201. На рис. 3.56-3.62 представлено навчання кожного із Folds. На рис. 3.63 представлено історію навчання мережі. На рис. 3.64 знімок екрану із Private Score та Public Score Моделі-34 із сайту Kaggle.com.

Fold 0 results				
epoch	train_loss	valid_loss	rmse	time
0	0.643291	0.645076	17.418671	08:17
1	0.642339	0.642979	16.912695	08:18
2	0.636257	0.642331	16.919899	08:18
3	0.623331	0.644252	16.943672	08:19
4	0.613556	0.645781	17.011297	08:18

Рисунок 3.56 Результати навчання (Fold 0) Моделі-34

Fold 1 results				
epoch	train_loss	valid_loss	rmse	time
0	0.641674	0.649030	17.585093	08:20
1	0.642229	0.645718	17.342482	08:18
2	0.631204	0.637766	16.337038	08:18
3	0.622376	0.640542	16.602400	08:18
4	0.602049	0.641795	16.639246	08:18
5	0.612993	0.642279	16.686745	08:18

Рисунок 3.57 Результати навчання (Fold 1) Моделі-35

Fold 2 results				
epoch	train_loss	valid_loss	rmse	time
0	0.646150	0.642604	16.966703	08:19
1	0.644309	0.638100	16.467430	08:19
2	0.639252	0.640168	16.665762	08:19
3	0.618775	0.641444	16.724682	08:19
4	0.613862	0.643182	17.015232	08:19

Рисунок 3.58 Результати навчання (Fold 2) Моделі-34

Fold 3 results				
epoch	train_loss	valid_loss	rmse	time
0	0.644695	0.644044	17.040936	08:19
1	0.640707	0.642751	16.875189	08:19
2	0.631035	0.642375	16.827433	08:19
3	0.623144	0.647279	17.333691	08:18
4	0.606796	0.648815	17.398142	08:19
5	0.611297	0.650318	17.485939	08:20

Рисунок 3.59 Результати навчання (Fold 3) Моделі-34

Fold 4 results				
epoch	train_loss	valid_loss	rmse	time
0	0.648506	0.645819	17.371634	08:19
1	0.644327	0.650965	18.286100	08:19
2	0.628387	0.643451	17.083652	08:18
3	0.626162	0.641603	16.919794	08:18
4	0.612185	0.644337	17.183104	08:19
5	0.605473	0.643913	17.073513	08:18

Рисунок 3.60 Результати навчання (Fold 4) Моделі-34

Fold 5 results				
epoch	train_loss	valid_loss	rmse	time
0	0.640512	0.644980	17.113544	08:20
1	0.640024	0.641543	16.770023	08:20
2	0.636254	0.640009	16.530115	08:18
3	0.620218	0.646360	17.226076	08:18
4	0.620173	0.647418	17.255497	08:18
5	0.608401	0.648370	17.349083	08:18

Рисунок 3.61 Результати навчання (Fold 5) Моделі-34

Fold 6 results

epoch	train_loss	valid_loss	rmse	time
0	0.641138	0.647199	17.501120	08:18
1	0.642871	0.649098	18.083582	08:18
2	0.624334	0.640699	16.737337	08:19
3	0.620310	0.642668	17.028461	08:18
4	0.620121	0.643868	17.034914	08:18
5	0.609383	0.645949	17.288437	08:18

Рисунок 3.62 Результати навчання (Fold 6) Моделі-34

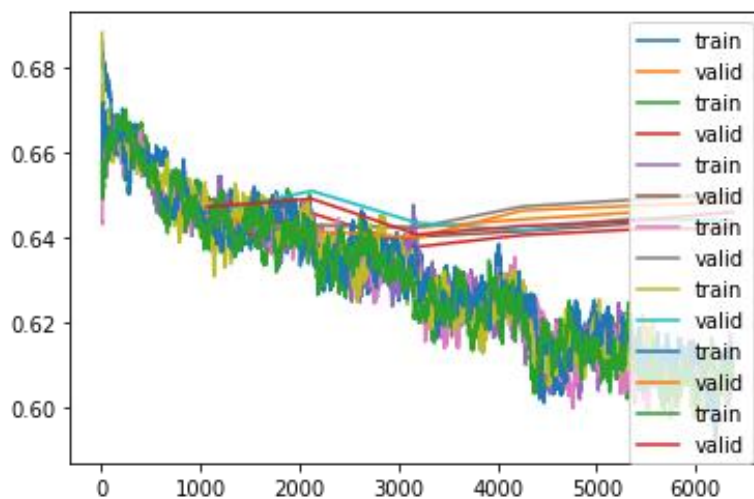


Рисунок 3.63 Історія навчання Моделі-34

Notebook Data Logs Comments (0) Settings


Competition Notebook

PetFinder.my - Pawpularity Contest
 Run
Private Score
Public Score
Best Score
Version 1 of 1

20217.6s - GPU
 17.08097
 17.90201
 17.08097 V1

GPU 

Рисунок 3.64 Знімок екрану із Private Score та Public Score Моделі-34

Після цього вирішено провести ще експерименти №6 (таблиця 3.10), але, при рахуванні фінального результату Моделі, використовуємо не середнє значення кожного із Folds результатів, а рахуємо фінальний результат із використанням вагових коефіцієнтів для кожного із Folds. Для цих експериментів базуємося на Моделі-34. Очевидно, що при використанні середнього значення кожен Folds $i=0,6$ Моделі-34 має однаковий ваговий коефіцієнт. Для цього виконуємо такі кроки:

1. Подаємо кожний вже навчений Fold Моделі-34 на конкурс як submission. Отримуємо Private Score і Public Score кожного із Fold (Табл. 3.9):

Таблиця 3.9 Результати кожного із $Fold_i, i = \overline{0,6}$ для Моделі-34

№ з/п	Назва параметру	$Fold_i, i = \overline{0,6}$ для Моделі-34						
		Fold 0	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Fold 6
1	Дата	04.05.2022 13:54:26	04.05.2022 14:11:13	04.05.2022 14:14:22	04.05.2022 14:19:44	04.05.2022 14:23:18	04.05.2022 14:26:27	04.05.2022 14:30:46
2	Час роботи коду	89.3s - GPU	83.6s - GPU	75.7s - GPU	80.5s - GPU	72.5s - GPU	77.3s - GPU	81.5s - GPU
3	Private Score	17.64737	17.35439	17.42758	17.36677	17.40185	17.32875	17.38898
4	Public Score	18.35185	18.08066	18.06819	18.20229	18.28371	18.03779	18.23405

2. Тоді, для рахунку фінального результату, із використанням всіх $Fold_i, i = \overline{0,6}$ Моделі-34, рахуємо не середнє значення всіх $Fold_i, i = \overline{0,6}$ Моделі-34, а використовуємо зважене значення всіх Private Score для кожного $Fold_i, i = \overline{0,6}$ Моделі-34. Це зважене значення будемо рахувати двома способами.

Спосіб-1:

Крок 1. Отримуємо знайдені Private Score кожного $Fold_i, i = \overline{0,6}$ Моделі-34 (рис. 3.65) та виписуємо їх.

```
# 1. Private Score
privateScore = [17.64738,
                17.35440,
                17.42759,
                17.36677,
                17.40185,
                17.32875,
                17.38898]
```

Рисунок 3.65 Private Score $Fold_i, i = \overline{0,6}$ Моделі-34

Крок 2. Знаходимо обернено пропорційні Private Score кожного $Fold_i, i = \overline{0,6}$ Моделі-34 (рис. 3.66-3.67) за формулою:

$$invProportPrivateScore_i = 1 / privateScore_i, \quad (3.3)$$

де $privateScore_i$ - Private Score кожного $Fold_i, i = \overline{0,6}$ Моделі-34.

Крок 3. Нормуємо обернено пропорційні Private Score $Fold_i, i = \overline{0,6}$ Моделі-34 так, щоб вони в сумі давали 1 (рис. 3.68-3.69), за формулою:

$$normPrivateScore_i = \frac{invProportPrivateScore_i}{\sum_{i=0}^6 invProportPrivateScore_i, i = \overline{0,6}}, \quad (3.4)$$

Крок 4. Після цього результати кожного $Fold_i, i = \overline{0,6}$ Моделі-34 сумуються із відповідним ваговим коефіцієнтом, отриманим на кроці 3.

```
# 2. Знаходимо обернено пропорційні значення Private Score
invProportPrivateScore = [1.0/x for x in privateScore]
print("invProportPrivateScore = ", invProportPrivateScore)
print()
```

Рис. 3.66 Рахуємо обернено пропорційні Private Score кожного $Fold_i, i = \overline{0,6}$ Моделі-34

```
invProportPrivateScore = [0.05666563535210326, 0.057622274466417746, 0.0573802
8034857373, 0.05758123128250101, 0.0574651545668995, 0.057707566904710385, 0.05
750768590222083]
```

Рис. 3.67 Обернено пропорційні Private Score кожного $Fold_i, i = \overline{0,6}$ Моделі-34

```
# 3. Нормалізуємо, щоб вагові коефіцієнти давали в сумі = 1
normPrivateScore = [x/sum(invProportPrivateScore) for x in invProportPrivateScore]
print("normPrivateScore = ", normPrivateScore)
print()

print("sum(normPrivateScore) = ", sum(normPrivateScore))
```

Рис. 3.68 Рахуємо нормалізовані обернено пропорційні Private Score кожного $Fold_i, i = \overline{0,6}$ Моделі-34

```
normPrivateScore = [0.14098390138890957, 0.14336401613957353, 0.14276193562578
732, 0.14326190084239124, 0.14297310238236824, 0.1435762234259606, 0.1430789201
950094]

sum(normPrivateScore) = 0.9999999999999998
```

Рис. 3.69 Нормалізовані обернено пропорційні значення Private Score кожного $Fold_i, i = \overline{0,6}$ Моделі-34

Також, було розглянуто і інший спосіб (Спосіб-2). Основна відмінність полягає у тому, що на кроці 2 рахуємо не просто обернено пропорційні значення, а їх квадрати (рис. 3.70-3.71) за формулою:

$$invProportPrivateScore_i = \sqrt{(1 / privateScore_i)}, \quad (3.5)$$

де $privateScore_i$ - Private Score кожного $Fold_i, i = \overline{0,6}$ Моделі-34.

```
# 2. Знаходимо обернено пропорційні значення Private Score
invProportPrivateScore = [math.sqrt(1.0/x) for x in privateScore]
print("invProportPrivateScore = ", invProportPrivateScore)
print()
```

Рис. 3.70 Рахуємо квадрати обернено пропорційних значень Private Score кожного $Fold_i, i = \overline{0,6}$ Моделі-34

```
invProportPrivateScore = [0.2380454480810403, 0.24004640065291075, 0.239541813
36162114, 0.23996089531942702, 0.2397189074038581, 0.2402239931911681, 0.239807
60184410507]
```

Рис. 3.71 Квадрати обернено пропорційні значення Private Score кожного $Fold_i, i = \overline{0,6}$ Моделі-34

Тоді нормалізовані вагові коефіцієнти будуть такими (рис. 3.72-3.73):

```
# 3. Нормалізуємо, щоб вагові коефіцієнти давали в сумі = 1
normPrivateScore = [x/sum(invProportPrivateScore) for x in invProportPrivateScore]
print("normPrivateScore = ", normPrivateScore)
print()

print("sum(normPrivateScore) = ", sum(normPrivateScore))
```

Рис. 3.72 Рахуємо нормалізовані квадрати обернено пропорційні значення Private Score кожного $Fold_i, i = \overline{0,6}$ Моделі-34

```
normPrivateScore = [0.14191799515702622, 0.14311092356499758, 0.14281009858666
338, 0.14305994697375812, 0.14291567855734177, 0.1432168007291589, 0.1429685564
310541]

sum(normPrivateScore) = 1.0
```

Рис. 3.73 Нормалізовані квадрати обернено пропорційні значення Private Score кожного $Fold_i, i = \overline{0,6}$ Моделі-34

Таблиця 3.10 Результати експериментів №6

№ з/п	Назва параметру	Назва способу підрахунку вагових коефіцієнтів	
		Спосіб-1	Спосіб-2
1	Дата	06.05.2022 11:34:30	06.05.2022 15:34:02
2	Час роботи коду	314.2s - GPU	250.1s - GPU
3	Private Score	17.09001	17.08993
4	Public Score	17.87513	17.87503
5	Фінальна оцінка	Підрахунок вагових коефіцієнтів, Спосіб-1	Підрахунок вагових коефіцієнтів, Спосіб-2
6	Результат	Succeeded	Succeeded

Аналізуючи отримані результати та порівнюючи їх із базовою Моделлю-34 бачимо, що результати виходять також досить непогані. Порівнюючи Public Score, Спосіб-2 дає більш точні результати. А за Private Score, все ж базова Модель-34 (Додаток В) дала кращі результати. Тому, у веб-додатку буде використовуватися саме вона.

3.3 Веб-додаток

Спочатку опишемо структуру всього проекту для запуску веб-додатку (рис. 3.74). Середовище написання коду - PyCharm. Проект складається із декількох файлів та тек, у кожній із яких містяться файли проекту із відповідним програмним кодом.

- `.ipynb_checkpoints` - системна папка, що автоматично створюється із точками збереження `*.ipynb` файлів;
- `static` - спеціальна тека, що використовується Flask. У ній містяться наступні теки:
 - `css` - тека, що містить в собі файл стилю `"style.css"` для веб-додатку;
 - `dataset` - тека, що містить в собі набір даних PetFinder.my;
 - `images` - тека, що містить в собі зображення, що відображаються на сайті (наприклад, логотип) для веб-додатку;
 - `js` - тека, що містить в собі файли `"*.js"` JavaScript для веб-додатку;

- `models` - тека, що містить в собі файли кожного $Fold_i, i = \overline{0,6}$ Моделі-34;
- `uploaded_images` - тека, що містить в собі зображення тварин, що передаються на сайт для прогнозування привабливості тварини із притулку за фото, потім вони автоматично видаляються звідти;
- `templates` - спеціальна тека, що використовується Flask. Вона містить в собі файл “`index.html`”, це основний файл веб-додатку;
- “`app.py`” - код, що містить точку запуску всього веб-додатку (Додаток Г);
- “`database_func.py`” - код, що містить деякі функції для взаємодії із базою даних MySQL Server (Додаток Г);
- “`neural_net_func.py`” - код, що містить деякі функції для взаємодії із нейронною мережею Модель-34 ($Fold_i, i = \overline{0,6}$) (Додаток Д).

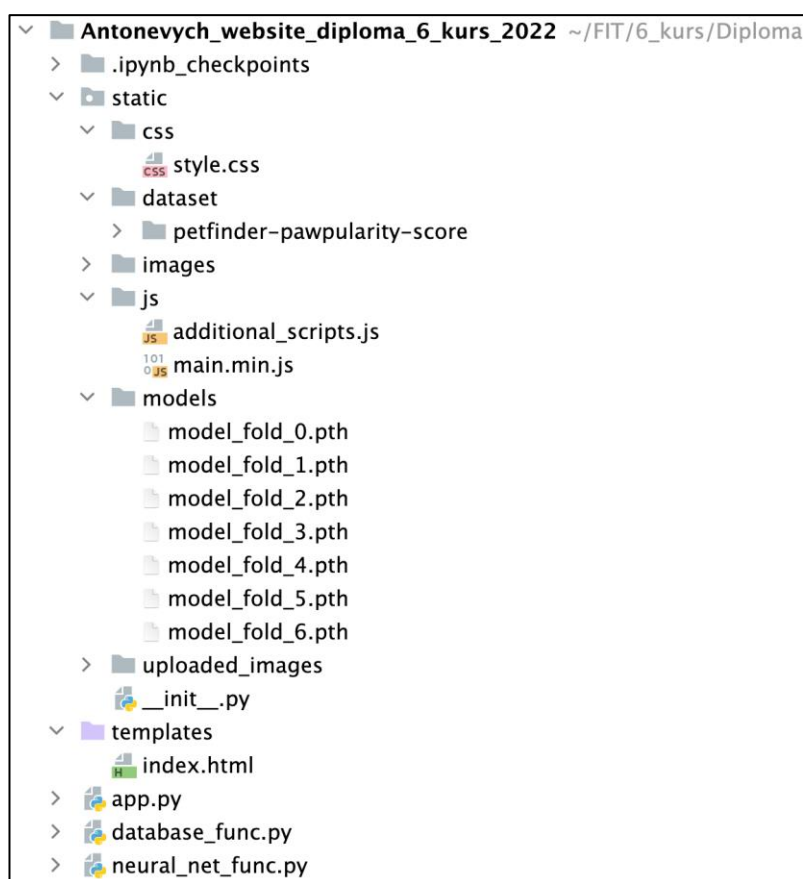


Рисунок 3.74 Структура проекту для запуску веб-додатку

Також хотілося б зазначити, що файл із навчанням мережі на сайті Kaggle.com “fork-of-my-fork-of-pawpularity-swin-transformer.ipynb” наведений у Додатку В.

Тепер, розглянемо роботу інтелектуального додатку, що був створений за допомогою веб-складових HTML/CSS/JavaScript, навченої нейронної мережі серверної сторони Python Flask та бази даних MySQL. Запуск відбувається на локальному комп'ютері (localhost).

У Додатку Е представлено веб-додаток. Там можна побачити назву веб-сайту, його автора. Також, там показано знімки екрану області, куди завантажувється фотографія користувачем та додається вся необхідна інформація для бази даних. Там відображено знімок екрану - обробка виключень. Коли користувач натиснув на кнопку “ANALYZE”, але не обрав необхідну фотографію, то з'являється відповідне повідомлення. Там представлено знімок екрану - обробка виключень. Коли користувач натиснув на кнопку “ANALYZE”, але не заповнив всю інформацію, то з'являється відповідне повідомлення. Там показано частину, звідки можливо вивантажити необхідну інформацію із бази даних (Додаток Є). На сайті є відповідних дві кнопки – “CASE HISTORY” та “PET'S INFO”. Більше того, там представлено частину, де можна ознайомитися детальніше із параметрами нейронної мережі чи перейти на сторінку змагань PetFinder.my - Pawpularity Contest на сайті Kaggle.com. І там показано частину, де присутні додаткові кнопки, наприклад, написати листа розробнику (мені), перейти на сайт факультету, університету тощо.

3.4 Висновки до розділу 3

У третьому розділі були проведені експерименти. Всього було проведено більше 30 вдалих досліджень (тобто таких, яких подано як submissions на конкурс). Експерименти проводилися із використанням згорткових нейронних мереж і трансформерів. Аналізуючи отримані результати, бачимо, що використання Swin Transformer покращило фінальні результати у порівнянні із

згортковими нейронними мережами. Як бачимо, найкращі результати серед групи експериментів №4 показала Модель-31 із результатом Private Score = 17.11781 та Public Score = 17.83321.

Аналізуючи отримані графіки навчання моделей та їх *RMSE*, можна прийти до висновку, що навіть плавний спадаючий графік навчання моделей на відкритих даних не гарантує якісний результат Private Score та Public Score. Це пов'язано з тим, що Private Score та Public Score формуються на основі 75% та 25% прихованих фотографій із іншої вибірки відповідно. Тобто таких, на яких мережа не має можливості навчитися. Звідси слідує висновок, що навіть зменшення втрат навчання на відкритому наборі даних не гарантує точно такого ж результату на прихованому наборі даних.

Порівнюючи Private Score та Public Score між собою, можна сказати, що результат Private Score може бути більшим за Public Score і навпаки. Як приклад, можна навести Модель-18, яка має найменший Public Score = 17.80600 серед всіх моделей, але Private Score = 17.16304 у неї більший, чим у Моделі-31.

Модель-34 використала поділ набору даних на 7 Folds. Результати: Private Score = 17.08097, Public Score = 17.90201. Як бачимо, дана модель отримала найкращий Private Score, тому саме вона буде використовуватися у веб-додатку.

Після цього був створений інтелектуальний додаток за допомогою веб-складових HTML/CSS/JavaScript, серверної сторони Python Flask та бази даних MySQL, із використанням MySQL Workbench та MySQL Server.

ВИСНОВКИ

У першому розділі розглянуто аспекти вирішення проблем привабливості тварин із притулку. Розглянуто проблему визначення привабливості тварин, базуючись на основних принципах і аспектах інформаційних технологій, а саме - штучного інтелекту та розпізнавання образів. Відомо, що у першу чергу, коли потенційні власники планують взяти тварину із притулку, вони заходять на сайт і роздивляються фотографії собак чи котів. Очевидно, що тварини з якісними фотографіями на сайті притулку викликають набагато більший інтерес і вони, як правило, отримують свій дім швидше. Такі сайти, як правило, не оснащені інтелектуальними технологіями, які можуть давати оцінку привабливості тварини за фото.

Через це, створення саме такої технології та подальше її використання допоможе пришвидшити процес пошуку та підбору нових власників тваринам із притулку, адже керівники притулку зможуть аналізувати фотографії, що будуть завантажуватися у профілі тварин та покращувати їх саме за допомогою використання даної інтелектуальної технології.

Така інтелектуальна технологія дозволяє об'єктивізувати вибір тих людей, які б могли взяти ту чи іншу тварину. Ті люди, що не планують брати тварину, можуть подивитися на якісну фотографію та змінити свій вибір і взяти тварину із притулку.

Також, було здійснено огляд існуючих засобів та технологій прогнозування привабливості тварин із притулку. Проаналізовано та досліджено рішення із відкритих джерел.

У другому розділі було спроектовано елементи інтелектуальної технології. Побудовано DFD та структурну діаграми. Виконано аналіз нейромережних засобів для виконання випускної кваліфікаційної роботи. Визначено переваги та недоліки нейронних мереж. Вибрано як основні засоби для дослідження згорткові нейронні мережі (Xception) та трансформери (Swin Transformer).

У третьому розділі проведені експерименти. Всього було проведено більше 30 вдалих досліджень. Експерименти проводилися із використанням згорткових нейронних мереж і трансформерів. Аналізуючи отримані результати, бачимо, що використання Swin Transformer покращило фінальні результати у порівнянні із згортковими нейронними мережами. Створений інтелектуальний додаток за допомогою веб-складових HTML/CSS/JavaScript, серверної сторони Python Flask та бази даних MySQL, із використанням MySQL Workbench та MySQL Server.

Аналізуючи отримані результати, можливо запропонувати майбутні покращення та вдосконалення. Як відомо, нейронні мережі можна вчити не лише за відомими правилами (наприклад, back propagation), а і з використанням методики підбору випадкових коефіцієнтів. До таких методик можна віднести ті, що використовують еволюційні оптимізаційні методи. Такі методи при виконанні певних умов можуть давати не лише локальний оптимум, а й глобальний оптимум. До таких можна віднести наступні методи: генетичні алгоритми, еволюційні стратегії, метод деформованих зірок [92-93] тощо.

Запропоновані у магістерській випускній кваліфікаційній роботі результати знайшли своє практичне відображення у розробці системи підтримки прийняття рішень, результати якої використані та впроваджені. Це підтверджується довідкою про використання результатів випускної кваліфікаційної роботи від Громадської Організації Черкаське міське товариство захисту тварин "Друг" (Додаток А), опублікованими тезами доповіді "Development and Research of the Intelligent Technology for Predicting the Popularity of Animals From the Shelter" та виступом на VIII Міжнародній науково-практичній конференції Information Technology and Implementation (Satellite) 2021 (Додаток Б).

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Antonevych M., Snytyuk V. Development and research of the intelligent technology for predicting the popularity of animals from the shelter / M. Antonevych, V. Snytyuk // Information technology and implementation (IT&I-2021)) / M. Antonevych, V. Snytyuk. – Kyiv: Taras Shevchenko National University of Kyiv, 2021. – С. 6-8. – [Електронний ресурс] – Режим доступу до ресурсу: http://iti.fit.univ.kiev.ua/wp-content/uploads/ITIS_2021.pdf
2. Смагіна А. Бездомні тварини: Україна у топ-10. Що робити, і чому притулки – не завжди вихід [Електронний ресурс] / Аліса Смагіна // Рубрика все по поличках. – 2021. – Режим доступу до ресурсу: <https://rubryka.com/article/bezdomni-tvaryny-ukrayina/>.
3. По всій Україні. Притулки для тварин, які гостро потребують допомоги під час війни [Електронний ресурс]. – 2022. – Режим доступу до ресурсу: <https://life.nv.ua/ukr/socium/pritulki-shcho-potrebuyut-dopomogi-yak-vryatuvati-tvarin-pid-chas-viyni-v-ukrajini-2022-50227450.html>.
4. Petfinder.my [Електронний ресурс] – Режим доступу до ресурсу: <https://www.petfinder.my/>.
5. PetFinder.my. PetFinder.my - Pawpularity Contest [Електронний ресурс] / PetFinder.my // Kaggle – Режим доступу до ресурсу: <https://www.kaggle.com/competitions/petfinder-pawpularity-score>.
6. Pytorch Image Models (timm) [Електронний ресурс] – Режим доступу до ресурсу: <https://fastai.github.io/timmdocs/#List-Models-with-Pretrained-Weights>.
7. Welcome to fastai [Електронний ресурс] – Режим доступу до ресурсу: <https://docs.fast.ai/>.
8. scikit-learn [Електронний ресурс] – Режим доступу до ресурсу: <https://scikit-learn.org/stable/>.
9. NumPy [Електронний ресурс] – Режим доступу до ресурсу: <https://numpy.org/>.
10. SciPy [Електронний ресурс] – Режим доступу до ресурсу: <https://scipy.org/>.

11. Matplotlib: Visualization with Python [Электронный ресурс] – Режим доступа до ресурсу: <https://matplotlib.org/>.
12. Henkel C. Efficient large-scale image retrieval with deep feature orthogonality and Hybrid-Swin-Transformers [Электронный ресурс] / Christof Henkel // arxiv. – 2021. – Режим доступа до ресурсу: <https://arxiv.org/abs/2110.03786>.
13. Deotte C. Hybrid Swin Transformer - Half CNN - Half Transformer [Электронный ресурс] / Chris Deotte. – 2021. – Режим доступа до ресурсу: <https://www.kaggle.com/c/petfinder-pawpularity-score/discussion/277917#1541782>.
14. Dieter. 1st place solution [Электронный ресурс] / Dieter. – 2021. – Режим доступа до ресурсу: <https://www.kaggle.com/c/landmark-recognition-2021/discussion/277098>.
15. Mingxing Tan. EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks [Электронный ресурс] / Mingxing Tan, Quoc V. Le. – 2020. – Режим доступа до ресурсу: <https://arxiv.org/pdf/1905.11946.pdf>.
16. Dev Khant. Swin Transformer + EfficientNet. [Электронный ресурс] / Dev Khant. – 2021. – Режим доступа до ресурсу: <https://www.kaggle.com/code/devkhant24/swin-transformer-efficientnet/>.
17. Abhishek Thakur. tez: pawpular swin-ference. [Электронный ресурс] / Abhishek Thakur. – 2021. – Режим доступа до ресурсу: <https://www.kaggle.com/code/abhishek/tez-pawpular-swin-ference>.
18. Manyu Li. RAPIDS SVR Boost - [17.8] - PRO. [Электронный ресурс] / Manyu Li. – 2021. – Режим доступа до ресурсу: <https://www.kaggle.com/code/manyuli/rapids-svr-boost-17-8-pro>.
19. Marky. RAPIDS SVR BoostX2-[18.0] 【JPN】 . [Электронный ресурс] / Marky. – 2021. – Режим доступа до ресурсу: <https://www.kaggle.com/code/masakitani/rapids-svr-boostx2-18-0-jpn>
20. Chris Deotte. RAPIDS SVR Boost - [17.8]. [Электронный ресурс] / Chris Deotte. – 2021. – Режим доступа до ресурсу: <https://www.kaggle.com/code/cdeotte/rapids-svr-boost-17-8/>

21. Zisserman A. Very Deep Convolutional Networks for Large-Scale Image Recognition [Електронний ресурс] / K. Simonyan, A. Zisserman // arxiv. – 2015. – Режим доступу до ресурсу: <https://arxiv.org/abs/1409.1556>.
22. Howard A. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications [Електронний ресурс] / Andrew G. Howard. – 2017. – Режим доступу до ресурсу: <https://arxiv.org/abs/1704.04861>.
23. Aditya Chakma. Starter-Keras| Meta Data| MobileNetV2 [Електронний ресурс] / Aditya Chakma. - 2021. – Режим доступу до ресурсу: <https://www.kaggle.com/code/adityachakma/starter-keras-meta-data-mobilenetv2>.
24. Alexander Scarlat MD. Pawpularity - Keras: image AND tabular features [Електронний ресурс] / Alexander Scarlat MD. – 2021. – Режим доступу до ресурсу: <https://www.kaggle.com/code/drscarlat/pawpularity-keras-image-and-tabular-features>.
25. K-Fold Cross Validation [Електронний ресурс] – Режим доступу до ресурсу: <https://medium.datadriveninvestor.com/k-fold-cross-validation-6b8518070833>.
26. Гайна Г. А. ОСНОВИ ПРОЕКТУВАННЯ БАЗ ДАНИХ / Георгій Анатолійович Гайна. – Київ: КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ БУДІВНИЦТВА І АРХІТЕКТУРИ, 2005. – 204 с.
27. Maneesh Kumar Singh. Levels in Data Flow Diagrams (DFD) [Електронний ресурс] / Maneesh Kumar Singh. – 2020. – Режим доступу до ресурсу: <https://www.geeksforgeeks.org/levels-in-data-flow-diagrams-dfd/>.
28. Gane C. Structured Systems Analysis: Tools and Techniques / C. Gane, T. Sarson., 1979.
29. Pattern recognition [Електронний ресурс] – Режим доступу до ресурсу: https://en.wikipedia.org/wiki/Pattern_recognition.
30. Тимощук П. В. Штучні нейронні мережі / П. В. Тимощук. – Львів: Видавництво Львівської Політехніки, 2011
31. Руденко О. Г. Штучні нейронні мережі / О. Г. Руденко, Є. В. Бодянський. – Харків: ТОВ "Компанія СМІТ", 2006. – 404 с.
32. Haykin S. Neural Networks: A Comprehensive Foundation, 2nd Edition / Simon

Haykin. – Ontario Canada: McMaster University, 1999.

33. Hochreiter S. Long short-term memory / S. Hochreiter, J. Schmidhuber // Neural computation / S. Hochreiter, J. Schmidhuber. – (9 (8)). – С. 1735–1780.

34. Hopfield J. Neural networks and physical systems with emergent collective computational abilities" / J. J. Hopfield // roceedings of the National Academy of Sciences / J. J. Hopfield., 1982. – С. 2554–2558.

35. Lippman R. An Introduction to Computing with Neural Nets / R. P. Lippman, 1987. – (№4). – С. 4–22.

36. Goodfellow I. Generative Adversarial Networks [Электронный ресурс] / Ian Goodfellow. – 2014. – Режим доступа до ресурсу: <https://arxiv.org/abs/1406.2661>.

37. Y. LeCun. Learning algorithms for classification: A comparison on handwritten digit recognition. / L. Jackel, L. Bottou, C. Cortes, J. S. Denker, H. Drucker, I. Guyon, U. Muller, E. Sackinger, P. Simard, et al. - Neural networks: the statistical mechanics perspective, 1995. – С. 261-276,

38. A. Krizhevsky. Imagenet classification with deep convolutional neural networks. / I. Sutskever, G. E. Hinton - In Advances in neural information processing systems, 2012 – С.1097–1105.

39. Chollet F. Xception: Deep Learning with Depthwise Separable Convolutions [Электронный ресурс] / François Chollet. – 2017. – Режим доступа до ресурсу: <https://arxiv.org/abs/1610.02357>.

40. Kelsey P. A poetry-writing AI has just been unveiled. It's ... pretty good. [Электронный ресурс] / Piper Kelsey. – 2019. – Режим доступа до ресурсу: <https://www.vox.com/2019/5/15/18623134/openai-language-ai-gpt2-poetry-try-it>.

41. Why everyone is talking about the A.I. text generator released by an Elon Musk-backed lab [Электронный ресурс]. – 2020. – Режим доступа до ресурсу: <https://www.cnbc.com/2020/07/23/openai-gpt3-explainer.html>.

42. Devlin J. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding [Электронный ресурс] / Jacob Devlin. – 2019. – Режим доступа до ресурсу: <https://arxiv.org/abs/1810.04805v2>.

43. What is CNN? [Электронный ресурс] // SSLA – Режим доступа до ресурсу: <https://www.ssla.co.uk/cnn>
44. Parmar R. Demystifying Convolutional Neural Networks [Электронный ресурс] / Ravindra Parmar. – 2019. – Режим доступа до ресурсу: <https://towardsdatascience.com/demystifying-convolutional-neural-networks-384785791596>.
45. Fukushima, Kunihiro (October 1979). "位置ずれに影響されないパターン認識機構の神経回路のモデル --- ネオコグニトロン ---" [Neural network model for a mechanism of pattern recognition unaffected by shift in position — Neocognitron —]. Trans. IECE (in Japanese). J62-A (10): 658–665.
46. Fukushima K. Neocognitron: A Self-organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position / Kunihiro Fukushima. // Biol. Cybernetics. – 1980. – №36. – С. 193–202.
47. LeCun Y. Gradient-based learning applied to document recognition / Y. LeCun, Y. Bengio, P. Haffner. // Proceedings of the IEEE. – 1998. – №86(11). – С. 2278–2324.
48. Krizhevsky A. ImageNet Classification with Deep Convolutional Neural Networks / A. Krizhevsky, I. Sutskever, G. Hinton. // Communications of the ACM.. – 2012. – №60(6). – С. 84–90.
49. ImageNet Large Scale Visual Recognition Challenge [Электронный ресурс] – Режим доступа до ресурсу: <http://www.image-net.org/challenges/LSVRC/>.
50. Szegedy C. Going deeper with convolutions / Christian Szegedy. – 2014.
51. Deep Residual Learning for Image Recognition / K. He, X. Zhang, S. Ren, J. Sun. – 2015.
52. Keras Applications [Электронный ресурс] // [keras.io](https://keras.io/api/applications/) – Режим доступа до ресурсу: <https://keras.io/api/applications/>.
53. Liu Z. Swin Transformer: Hierarchical Vision Transformer using Shifted Windows [Электронный ресурс] / Z. Liu. – 2021. – Режим доступа до ресурсу: <https://arxiv.org/abs/2103.14030>.

54. Vaswani A. Attention Is All You Need [Электронный ресурс] / Ashish Vaswani. – 2017. – Режим доступа до ресурсу: <https://arxiv.org/abs/1706.03762>.
55. He C. Transformer in CV [Электронный ресурс] / Cheng He. – 2020. – Режим доступа до ресурсу: <https://towardsdatascience.com/transformer-in-cv-bbdb58bf335e>.
56. Wolf T. Transformers: State-of-the-Art Natural Language Processing / Thomas Wolf., 2020.
57. Common Crawl [Электронный ресурс] – Режим доступа до ресурсу: <https://commoncrawl.org/>.
58. Devlin J. Open Sourcing BERT: State-of-the-Art Pre-training for Natural Language Processing [Электронный ресурс] / Jacob Devlin. – 2018. – Режим доступа до ресурсу: <https://ai.googleblog.com/2018/11/open-sourcing-bert-state-of-art-pre.html>.
59. Better Language Models and Their Implications [Электронный ресурс]. – 2019. – Режим доступа до ресурсу: <https://openai.com/blog/better-language-models/>.
60. Wang A. GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding [Электронный ресурс] / Alex Wang – Режим доступа до ресурсу: <https://arxiv.org/abs/1804.07461>.
61. Sarkar A. Are Transformers better than CNN's at Image Recognition? [Электронный ресурс] / Arjun Sarkar. – 2021. – Режим доступа до ресурсу: <https://towardsdatascience.com/are-transformers-better-than-cnns-at-image-recognition-ced60ccc7c8>.
62. Facebook and UC Berkeley Boost CV Performance and Lower Compute Cost With Visual Transformers [Электронный ресурс]. – 2020. – Режим доступа до ресурсу: <https://medium.com/syncedreview/facebook-and-uc-berkeley-boost-cv-performance-and-lower-compute-cost-with-visual-transformers-c019823f0561>.
63. Lin T. Microsoft COCO: Common Objects in Context [Электронный ресурс] / Tsung-Yi Lin. – 2014. – Режим доступа до ресурсу: <https://arxiv.org/abs/1405.0312>.
64. Cocomini D. On Transformers, TimeSformers, and Attention [Электронный

- ресурс] / Davide Cocomini // 2021 – Режим доступа до ресурсу: <https://towardsdatascience.com/transformers-an-exciting-revolution-from-text-to-videos-dc70a15e617b>.
65. Tesla AI Day [Электронный ресурс] – Режим доступа до ресурсу: <https://www.youtube.com/watch?v=j0z4FweCy4M>.
66. Xiao T. Early Convolutions Help Transformers See Better [Электронный ресурс] / Tete Xiao. – 2021. – Режим доступа до ресурсу: <https://arxiv.org/abs/2106.14881>.
67. Raghu M. Do Vision Transformers See Like Convolutional Neural Networks? [Электронный ресурс] / Maithra Raghu. – 2021. – Режим доступа до ресурсу: <https://arxiv.org/abs/2108.08810>.
68. Naseer M. Intriguing Properties of Vision Transformers [Электронный ресурс] / Muzammal Naseer. – 2021. – Режим доступа до ресурсу: <https://arxiv.org/abs/2105.10497>.
69. Touvron H. Training data-efficient image transformers & distillation through attention [Электронный ресурс] / Hugo Touvron. – 2020. – Режим доступа до ресурсу: <https://arxiv.org/abs/2012.12877>.
70. ADE20K [Электронный ресурс] – Режим доступа до ресурсу: <http://groups.csail.mit.edu/vision/datasets/ADE20K/>.
71. Kaggle [Электронный ресурс] – Режим доступа до ресурсу: <https://www.kaggle.com/>.
72. PyCharm [Электронный ресурс] – Режим доступа до ресурсу: <https://www.jetbrains.com/pycharm/>.
73. Python [Электронный ресурс] – Режим доступа до ресурсу: <https://www.python.org/>.
74. Sharma R. 5 Key Advantages of Using Python for Artificial Intelligence Development [Электронный ресурс] / Rohit Sharma // upGrad blog – Режим доступа до ресурсу: <https://www.upgrad.com/blog/advantages-of-using-python-for-artificial-intelligence-development/>.
75. Matplotlib: Visualization with Python [Электронный ресурс] – Режим доступа

до ресурсу: <https://matplotlib.org/>.

76. NumPy [Электронный ресурс] – Режим доступа до ресурсу: <https://numpy.org/>.

77. Tensorflow [Электронный ресурс] – Режим доступа до ресурсу: <https://www.tensorflow.org/>.

78. Abadi M. TensorFlow: A system for large-scale machine learning [Электронный ресурс] / Martín Abadi – Режим доступа до ресурсу: <https://arxiv.org/abs/1605.08695>.

79. Keras: the Python deep learning API [Электронный ресурс] – Режим доступа до ресурсу: <https://keras.io/>.

80. PyTorch [Электронный ресурс] – Режим доступа до ресурсу: <https://pytorch.org/>.

81. Facebook brings GPU-powered machine learning to Python [Электронный ресурс]. – 2017. – Режим доступа до ресурсу: <https://www.infoworld.com/article/3159120/facebook-brings-gpu-powered-machine-learning-to-python.html>.

82. Natural Language Processing (NLP) with PyTorch [Электронный ресурс] – Режим доступа до ресурсу: <https://dl4nlp.info/en/latest/>.

83. When two trends fuse: PyTorch and recommender systems [Электронный ресурс]. – 2017. – Режим доступа до ресурсу: <https://www.oreilly.com/content/when-two-trends-fuse-pytorch-and-recommender-systems/>.

84. Welcome to fastai [Электронный ресурс] – Режим доступа до ресурсу: <https://docs.fast.ai/>

85. Django [Электронный ресурс] – Режим доступа до ресурсу: <https://www.djangoproject.com/>.

86. FAQ: General [Электронный ресурс] – Режим доступа до ресурсу: <https://docs.djangoproject.com/en/dev/faq/general/#django-appears-to-be-a-mvc-framework-but-you-call-the-controller-the-view-and-the-view-the-template-how-come-you-don-t-use-the-standard-names>.

87. Flask [Электронный ресурс] – Режим доступа до ресурсу:

<https://flask.palletsprojects.com/en/1.1.x/>.

88. Foreword [Електронний ресурс] – Режим доступу до ресурсу: <https://web.archive.org/web/20171117015927/http://flask.pocoo.org/docs/0.10/foreword>.

89. Flask Extensions [Електронний ресурс] – Режим доступу до ресурсу: <https://web.archive.org/web/20180517082208/http://flask.pocoo.org/extensions/>.

90. mysql-connector-python [Електронний ресурс] – Режим доступу до ресурсу: <https://pypi.org/project/mysql-connector-python/>.

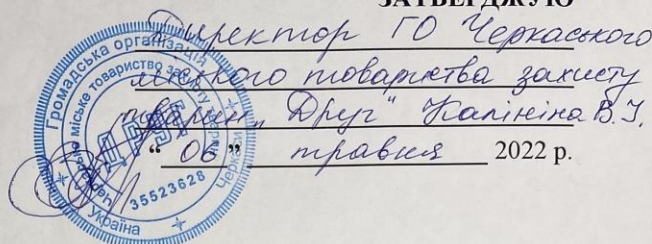
91. MySQL [Електронний ресурс] – Режим доступу до ресурсу: <https://www.mysql.com/>.

92. The Method of Deformed Stars as a Population Algorithm for Global Optimization / V. Snytyuk, M. Antonevych, A. Didyk, N. Tmienova // System Analysis & Intelligent Computing / V.Snytyuk, M. Antonevych, A. Didyk, N. Tmienova., 2022. – С. 229–247. – [Електронний ресурс] – Режим доступу до ресурсу: https://link.springer.com/chapter/10.1007/978-3-030-94910-5_13

93. Choosing the best parameters for method of deformed stars in n-dimensional space / M.Antonevych, A. Didyk, N. Tmienova, V. Snytyuk // Advanced Information Technology / M.Antonevych, A. Didyk, N. Tmienova, V. Snytyuk. – Kyiv, 2021. – С. 24–28. – [Електронний ресурс] – Режим доступу до ресурсу: <https://ait.knu.ua/choosing-the-best-parameters-for-method-of-deformed-stars-in-n-dimensional-space/>

ДОДАТОК А Довідка про використання результатів випускної кваліфікаційної роботи

ЗАТВЕРДЖУЮ



Довідка

про використання результатів випускної кваліфікаційної роботи

“Розробка та дослідження інтелектуальної технології визначення привабливості тварин із притулку”

Комісія у складі Калініна Валентина Ігорівна, Герболіньська Алла Миколаївна, Тарасенко Микола розглянула результати випускної кваліфікаційної роботи Антоневич Марини Миколаївни “Розробка та дослідження інтелектуальної технології визначення привабливості тварин із притулку” та констатує таке.

Наявність великої кількості безпритульних тварин у населених пунктах та у притулках для покинутих тварин вимагає розробки та використання технологій штучного інтелекту для прискорення процесу пошуку нових власників тварин. Одним із факторів можна назвати такий, що приваблива фотографія викликає набагато більший інтерес і тварини, як правило, отримують свою затишну домівку швидше.

Отримані автором результати, а саме використання штучних нейронних мереж для визначення привабливості тварин та відповідний програмний модуль мають важливе значення для прискорення пошуку нових власників для тварин із притулку. За допомогою аналізу фотографії тварини можливо швидко визначити, чи дана фотографія є якісною, чи потрібно перефотографувати тварину. Випускна кваліфікаційна робота є актуальною, впровадження одержаних у ній результатів дозволяє прискорити процес пошуку нових власників для тварин із притулку.

КОМІСІЯ:

Калініна В. І.
Герболіньська А. М.
Тарасенко В. Ю.

ДОДАТОК Б Матеріали конференції IT&I-2021 (Satellite)



**INFORMATION
TECHNOLOGY &
IMPLEMENTATION
(SATELLITE)**

02 December, 2021
Kyiv, Ukraine

ISBN 978-966-96914-0-8



9 789669 691408

TARAS SHEVCHENKO NATIONAL UNIVERSITY OF KYIV
(FACULTY OF INFORMATION TECHNOLOGY, FACULTY OF COMPUTER SCIENCE AND
CYBERNETICS)
NATIONAL TECHNICAL UNIVERSITY OF UKRAINE "IGOR SIKORSKY KYIV
POLYTECHNIC INSTITUTE"
VIKTOR GLUSHKOV INSTITUTE OF CYBERNETICS OF THE NAS OF UKRAINE
INSTITUTE OF INFORMATION TECHNOLOGY AND LEARNING TOOLS OF THE NAES OF
UKRAINE
INSTITUTE FOR INFORMATION RECORDING OF THE NAS OF UKRAINE
INSTITUTE OF SOFTWARE SYSTEMS OF THE NAS OF UKRAINE
THE COUNCIL OF YOUNG SCIENTISTS OF THE FACULTY OF COMPUTER SCIENCE AND
CYBERNETICS AND THE FACULTY OF INFORMATION TECHNOLOGY OF TARAS
SHEVCHENKO NATIONAL UNIVERSITY OF KYIV

VIII INTERNATIONAL CONFERENCE

Information Technology and Implementation (Satellite)

02 December, 2021

Conference Proceedings

Kyiv

2021

UDC 004.9

I-74

Volume editor: Vitaliy Snytyuk, Dr.Sc., Prof.

Program Committee: Aldrich Chris, Andreas Pester, Frederic Mallet, Hiroshi Tanaka, Iurii Krak, Karsten Henke, Mykola Nikitchenko, Oleg Chertov, Oleksandr Marchenko, Sándor Bozóki, Vitaliy Tsyganok, Vladimir Vovk

Organizing Committee: Anatoly Anisimov, Vitaliy Snytyuk, Anatolii Pashko, Kateryna Kolesnikova, Mykola Nikitchenko, Nataliia Lukova-Chuiko, Oleh Ilarionov, Oleksandr Marchenko, Oleksii Bychkov, Serhii Toliupa, Taras Panchenko, Valentyna Pleskach, Viktor Morozov, Volodymyr Zaslavsky, Iurii Krak, Yurii Samokhvalov, Yurii Kravchenko

Conference Secretary: Hryhorii Hnatiienko.

I-74 Information Technology and Implementation (Satellite): Conference Proceedings, December 02, 2021, Kyiv, Ukraine / Taras Shevchenko National University of Kyiv and [etc]; Vitaliy Snytyuk (Editor). – Kyiv: Publisher Individual entrepreneur Picha Y.V., 2021. 185 p.

ISBN 978-966-96914-0-8

This book includes abstracts of the 8th International Conference "Information Technology and Implementation (Satellite) – 2021". Philosophical, theoretical and applied aspects which describe the results, problems and prospects of the creation and use of intelligent computing methods and creating of information systems and technology on their basis are reviewing.

Main tracks of the conference are: Artificial Intelligence Technologies, Cyberspace Protection Technologies, Data Analytics, Digital Project Management Technologies, E-commerce, E-government and E-learning Technologies, Mathematical Foundations of Information Technology, Network and Internet Technologies.

До цієї збірки увійшли тези 8-ї Міжнародної конференції «Інформаційні технології та впровадження (сателітної) – 2021». Розглядаються філософські, теоретичні та прикладні аспекти, які описують результати, проблеми та перспективи створення і використання інтелектуальних обчислювальних методів та створення на їх основі інформаційних систем і технологій.

Основні напрямки конференції: Технології штучного інтелекту, Технології захисту кіберпростору, Аналітика даних, Технології цифрового управління проєктами, Електронна комерція, Електронне урядування та Технології електронного навчання, Математичні основи інформаційних технологій, Мережеві та Інтернет-технології.

UDC 004.9

© Authors of abstracts, 2021

ISBN 978-966-96914-0-8

© Publisher Individual entrepreneur Picha Y.V., 2021

Information Technology and Implementation (IT&Is-2021)	3
--	---

CONTENTS

<i>Antonevych M., Snytyuk V.</i> Development and Research of the Intelligent Technology for Predicting the Popularity of Animals From the Shelter	6
<i>Antoniuk V., Parkhomenko I.</i> Identification of Critical Information Infrastructure Objects	9
<i>Bakukha N., Khlevna Iu.</i> Classification Models in E-Commerce Projects	11
<i>Bidochka V., Paliy S.</i> Decentralized Network in IoT Systems	13
<i>Buchyk S., Kosse A.</i> BadUSB: Overview of the Possible Attacks With the Usage of Arduino Board	16
<i>Buchyk S., Piatashova E.</i> Phishing Detection Based on Unsupervised Learning Method	19
<i>Buchyk S., Piatyhor V.</i> Password Complexity Analysis of Diceware Password Generation Algorithm Modification Based on Pseudo-Random Sequences	21
<i>Chen J.</i> Competitiveness factors prognosis of international educational programs of Chinese Educational Establishments	24
<i>Chernetsky N., Pavlichenko A., Morozov V.</i> Comparison of Methods of Development of Computer Video Games in Project Management in the Game Industry	27
<i>Deinega V., Mochalova D.</i> Analysis of Forecasting Methods for Financial Markets	29
<i>Fedirko Yu., Yehorchenkov O.</i> Random Forest Algorithm in Predicting Diabetes (Type 2)	33
<i>Fediuk D., Khlevna Iu.</i> Collecting Feedback During Online Conferences as a Means of Increasing Team Productivity	37
<i>Gaina G.</i> Use of Ontologies and Graphs of Knowledge in the Educational Process	39
<i>He Yu.</i> Development of Methods for Environmental Pollution Monitoring	41
<i>Hnatiienko H., Izhevska Ye.</i> Mathematical Model for Determination of Prioritizing Services When Allocating Limited Resources	44
<i>Hnatiienko H., Hnatiienko V., Rimek A.</i> Mathematical Modeling of Functional Stability of Organizational System	47
<i>Hnatiienko H., Vlasenko O.</i> Mathematical Support of the Rating Voting Procedure	50
<i>Huang M.</i> Development of GIS Using for Monitoring Crop Yields	54
<i>Chunmei J.</i> Development of Methods for Decision-Making for Agriculture Under Uncertainty	56
<i>Khlevnyi A., Kashapova L.</i> The Conceptual Project Management Model in the Sphere of E-Commerce Based on a Personalized Analysis	59
<i>Khlevnyi A., Nesterenko Ye.</i> How Human Resources Processes Have Changed During the Pandemic	62
<i>Kontorer A., Khlevna Iu.</i> Peculiarities of Project Team Management in a Remote Work Conditions	65
<i>Kopp A., Orlovskiy D., Orekhov S.</i> An Approach to Build a Decentralized Collection of Business Process Models	68

¹ **Maryna Antonevych**

Master's student of the Faculty of Information Technology

² **Vitaliy Snytyuk**

Dr. of. Sci., Professor, Dean of the Faculty of Information Technology

^{1,2} *Taras Shevchenko National University of Kyiv*

DEVELOPMENT AND RESEARCH OF THE INTELLIGENT TECHNOLOGY FOR PREDICTING THE POPULARITY OF ANIMALS FROM THE SHELTER

Abstract. Millions of stray animals are injured on the streets every day around the world. So, what can be done, to help these animals to get new home? Scientists and researchers can suggest improvements that will help shelter animals to have a higher chance to be adopted with the help of data science. One of the solutions for this problem can be called one that will also use artificial intelligence. We intend to use a computer vision apparatus based on neural networks.

Keywords: Computer vision, prediction, intelligent technology, artificial intelligence, neural networks, deep learning, data science, shelter animals

Introduction. Millions of stray animals are injured on the streets every day around the world [1]. Such animals feel alone and they cannot live happy and healthy life. It is a great problem that so many animals cannot find their loving homes. In addition, these animals face many problems during their life in shelters. As example, for most dogs, being relinquished to an animal shelter is a drastic change and a very stressful experience [2]. Such environmental changes are very stressful for the most of dogs. Stress may also affect a shelter dog's physical health, causing increased susceptibility to diseases and a longer recovery time from illness [2].

And all of them need love and a cozy home. So, what can be done, to help these animals to get new home? One picture is worth a thousand words and the photography can save thousand lives. It is obvious that pets with charming photos can generate more interest and they can be adopted faster [1].

Scientists and researchers can determine the appeal of pet pictures. Moreover, they can suggest improvements that will help shelter animals to have a higher chance to be adopted with the help of data science. There are many animal shelters & organizations that help animals to get their new loving homes all over the world.

For example, "PetFinder.my" is Malaysia's leading animal welfare platform with over 180000 animals, 54000 of which have been safely adopted [1]. "PetFinder" works closely with animal lovers, the media, corporations and international organizations that help to improve animal welfare [1].

Moreover, they have launched Kaggle Competition on the 23rd of September, 2021. That's why it is new to the world. Their aim is to help shelter animals to get their new home with the help of data science.

Problem Formulation. As it Kaggle Competition, the given problem is posted on the site kaggle.com [1]. The task is to predict engagement with a pet's profile based on the photography for that profile. Therefore, our aim is to analyze raw images and

metadata and, after that, create intelligent technology that will be able to predict the “pawpularity” of pet photos.

Dataset. As it said in [1], the data can be used for academic research and education. In addition, the owners of the task have included optional Photo Metadata, manually labeling each photo for key visual quality and composition parameters. These labels are not used for deriving their “pawpularity” score, but it may be beneficial for better understanding the content and co-relating to a photo's attractiveness. The example of images is given in Fig.1.

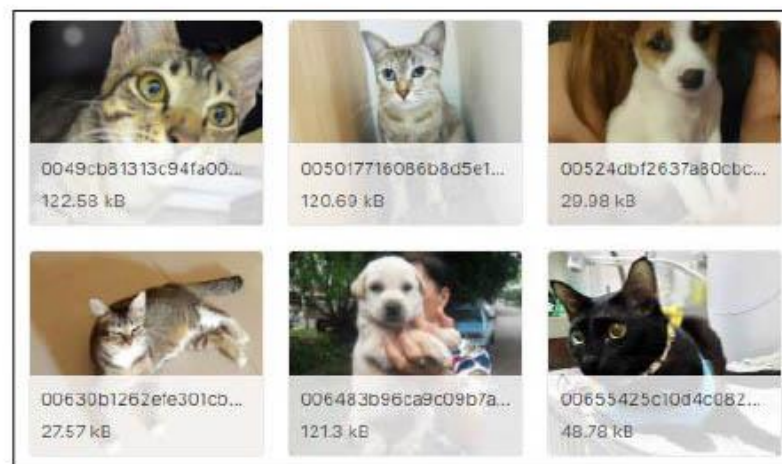


Figure 1 - Dataset

Previous Decisions Achieved. “PetFinder.my” currently uses a basic attraction counter for ranking pet photos. It analyzes image composition and other factors based on characteristics of thousands of pet profiles. The “pawpularity” score is derived from each pet profile's page view statistics at the listing pages, using the algorithm that normalizes the traffic data across different pages, platforms (web & mobile) and other various metrics. While this basic tool is useful, it is still in an experimental stage and the algorithm could be improved [1].

Offered Approach. One of the proposed solutions for this problem can be called the one that will use artificial intelligence (AI) technologies. As we have images in the dataset, so the solution belongs to the domain of the computer vision (CV) problems. CV is a field of AI that enables computers and systems to derive important information from digital images, videos and other visual inputs — and take actions or make recommendations based on that information. If AI enables computers to think, CV enables them to see, observe and understand [3].

For example, CV consists of such tasks as image processing, object detection, image recognition, image segmentation and so on.

Image processing is a method to perform some operations on an image, in order to get an enhanced image or to extract some useful information from it [4].

Object detection is a CV technique that allows to locate objects in an image or video. With this kind of identification and localization, object detection can be used to count objects in a scene and determine and track their precise locations, all while accurately labeling them [5].

Image recognition, in the context of machine vision, is the ability of software to identify objects in images [6].

Image segmentation is a method in which a digital image is broken down into various subgroups called “image segments” which helps in reducing the complexity of the image to make further processing or analysis of the image simpler. Segmentation in easy words is assigning labels to pixels. All picture elements or pixels belonging to the same category have a common label assigned to them [7].

All these tasks can be solved using such methods as: basic morphology operations (erosion, dilation, opening, closing etc.); machine learning methods (e.g. image segmentation can be done with the help of clustering methods [8], such as K-Means or evolutionary methods like genetic algorithms, MODS etc.); deep learning methods (e.g. Convolutional Neural Networks, Recurrent Neural Networks).

Therefore, we intend to use a computer vision apparatus based on neural networks for predicting the floating value of the “pawpularity” score.

References:

1. PetFinder.my - Pawpularity Contest, 2021. URL: <https://www.kaggle.com/c/petfinder-pawpularity-score>
2. Center for Shelter Dogs, Transition and Stress. URL: <https://centerforshelterdogs.tufts.edu/dog-welfare/transition-and-stress/>
3. What is Computer Vision?, IBM, URL: <https://www.ibm.com/topics/computer-vision>
4. Digital Image Processing, University of Tartu, URL: <https://sisu.ut.ee/imageprocessing/book/1>
5. Object Detection Guide, FRITZ AI, URL: <https://www.fritz.ai/object-detection/>
6. Image recognition, TechTarget, URL: <https://searchenterpriseai.techtarget.com/definition/image-recognition>
7. Image Segmentation: Part 1, Towards Data Science, URL: <https://towardsdatascience.com/image-segmentation-part-1-9f3db1ac1c50>
8. Maryna Antonevych, Nataliia Tmienova and Vitaliy Snytyuk, “Models and Evolutionary Methods for Objects and Systems Clustering”. Proceedings of the International Scientific Symposium “Intelligent Solutions” (IntSol-2021), Uzhorod/online, Ukraine, 28-30 September 2021, pp.47-56.

ДОДАТОК В Лістинг програми навчання Swin Transformer (Модель-34)

Libraries

```
In [1]: import gc
import math
import datetime
import sys
sys.path.append('../input/timpytorchimagemodels/pytorch-image-models-master') # add library
from timm import create_model
from fastai.vision.all import *
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import KFold
from sklearn.model_selection import StratifiedKFold

In [2]: set_seed(2999, reproducible=True)
```

Data

```
In [3]: # dataset_path
dataset_path = Path('../input/petfinder-pawpularity-score/')

In [4]: df_train = pd.read_csv(dataset_path/'train.csv')
df_train.head(10)
```

		Id	Subject	Focus	Eyes	Face	Near	Action	Accessory	Group	Collage	Human	Occlusion	Info	Blur	Pawpularity
0	0007de18844b0dbb5e1f607da0606e0			0	1	1	1	0	0	1	0	0	0	0	0	63
1	0009c66b9439883ba2750fb825e1d7db			0	1	1	0	0	0	0	0	0	0	0	0	42
2	0013fd999caf9a3efe1352ca1b0d937e			0	1	1	1	0	0	0	0	1	1	0	0	28
3	0018df346ac9c1d8413fcc888ca8246			0	1	1	1	0	0	0	0	0	0	0	0	15
4	001dc955e10590d3ca46731034feef2			0	0	0	1	0	0	1	0	0	0	0	0	72
5	001dd4f6fab890610b1635f967ea081			0	0	1	0	0	0	0	0	0	0	0	1	74
6	0023b8a3abc93c712edd6120867deb53			0	1	1	1	0	0	0	0	1	1	0	0	22
7	0031d6a9ef7340f898c3e05f92c7bb04			0	1	1	0	0	0	1	1	0	0	1	0	35
8	0042bc5bada6d1cf8951f8f9fd399fa			0	1	1	1	0	0	0	0	0	0	0	0	53
9	0049cb81313c94fa007286e9039af910			0	1	1	1	0	0	0	0	0	0	0	0	21

```
In [5]: df_train['path'] = df_train['Id'].map(lambda x:str(dataset_path/'train/'+x+'.jpg'))
df_train = df_train.drop(columns=['Id'])
# shuffle dataframe
df_train = df_train.sample(frac=1).reset_index(drop=True)
df_train.head(10)
```

	Subject	Focus	Eyes	Face	Near	Action	Accessory	Group	Collage	Human	Occlusion	Info	Blur	Pawpularity	path
0	0	1	1	1	0	1	0	0	0	0	0	0	0	27	../input/petfinder-pawpularity-score/train/fa123435b35c107a6c53f8d5a5c36f6c.jpg
1	0	1	1	1	0	0	0	0	0	0	0	0	0	30	../input/petfinder-pawpularity-score/train/d7d222be164177b8a1033fb51a5d762f.jpg
2	0	1	1	1	0	0	1	0	0	0	0	0	0	26	../input/petfinder-pawpularity-score/train/62caf65e5c6ae1d881d1bb986b609011.jpg
3	0	1	1	1	0	0	0	0	0	0	0	0	0	24	../input/petfinder-pawpularity-score/train/c2243c99e0a96d5e0dfbb9b0e922bed.jpg
4	0	1	1	1	0	0	0	0	0	0	0	0	0	75	../input/petfinder-pawpularity-score/train/ddcfffbbc1a816adbbe38339f19c6b5f9.jpg
5	0	1	1	0	0	0	1	0	0	0	0	0	0	35	../input/petfinder-pawpularity-score/train/2ccc26b6ecc9c8ff4153761f6a03f58e.jpg
6	0	1	1	1	0	0	0	0	0	0	0	0	0	26	../input/petfinder-pawpularity-score/train/92189fde724e45c04af7235e29970040.jpg
7	0	1	1	1	0	0	0	0	1	1	0	0	0	34	../input/petfinder-pawpularity-score/train/027a909fd3bc802f80646e92f8fddc50.jpg
8	0	1	0	1	0	0	0	0	0	0	0	0	0	29	../input/petfinder-pawpularity-score/train/56f43f28d69c193544627d7bed5c8f81.jpg
9	0	1	1	1	0	0	0	0	0	0	0	0	0	36	../input/petfinder-pawpularity-score/train/b9cf47f8b3b5a0855bb3293b138776e5.jpg

```
In [6]: df_train.shape

Out [6]: (9912, 14)
```

```
In [7]: df_train['Pawpularity'].hist(figsize = (10, 5), color='c')
print("Mean: ", df_train['Pawpularity'].mean())
print("Median: ", df_train['Pawpularity'].median())
print("Std: ", df_train['Pawpularity'].std())
```

Mean: 38.03904358353511
Median: 33.0
Std: 20.591990105774542

```
In [8]: len(df_train['Pawpularity'].unique())
```

Out[8]: 100

Note that the Pawpularity score is an integer, so in addition to being a regression problem, it could also be treated as a 100-class classification problem. Alternatively, it can be treated as a binary classification problem if the Pawpularity Score is normalized between 0 and 1:

In [9]: Pawpularity_max = 100

```
In [10]: # normalize data
df_train['norm_score'] = df_train['Pawpularity'] / Pawpularity_max
df_train['norm_score']
```

```
Out[10]: 0      0.27
1      0.30
2      0.26
3      0.24
4      0.75
...
9907   0.97
9908   0.24
9909   0.34
9910   0.56
9911   0.49
Name: norm_score, Length: 9912, dtype: float64
```

```
In [11]: im = Image.open(df_train["path"][1])
width, height = im.size
print(width, height)

467 960
```

In [12]: im

Out[12]:



Swin Transform

```
In [13]: if not os.path.exists('/root/.cache/torch/hub/checkpoints/'):
os.makedirs('/root/.cache/torch/hub/checkpoints/')

!cp './input/swin-transformer/swin_large_patch4_window7_224_22kto1k.pth' '/root/.cache/torch/hub/checkpoints/swin_large_patch4_window7_224_22kto1k.pth'
```

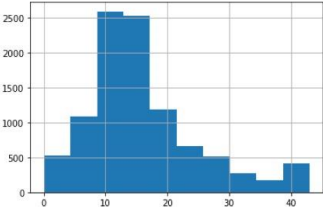
```
In [14]: # set seed
seed = 999
set_seed(seed, reproducible=True)
torch.manual_seed(seed)
torch.cuda.manual_seed(seed)
torch.backends.cudnn.deterministic = True
torch.use_deterministic_algorithms = True
```

Optimal Number of Bins

```
In [15]: #S turges' rule
num_bins = int(np.floor(1+(3.3)*(np.log2(len(df_train)))))
num_bins
```

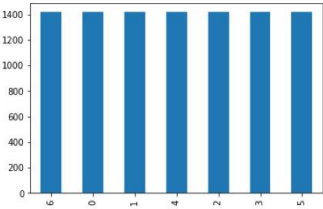
```
Out[15]: 44

In [16]: df_train['bins'] = pd.cut(df_train['norm_score'], bins=num_bins, labels=False)
df_train['bins'].hist()

Out[16]: <AxesSubplot:>


In [17]: N_FOLDS = 7
df_train['fold'] = -1
strat_kfold = StratifiedKFold(n_splits=N_FOLDS, random_state=seed, shuffle=True)
for i, (_, train_index) in enumerate(strat_kfold.split(df_train.index, df_train['bins'])):
    df_train.iloc[train_index, -1] = i
df_train['fold'] = df_train['fold'].astype('int')

In [18]: df_train.fold.value_counts().plot.bar()

Out[18]: <AxesSubplot:>


In [19]: df_train[df_train['fold']==0].head()

Out[19]:
  Subject  Eyes  Face  Near  Action  Accessory  Group  Collage  Human  Occlusion  Info  Blur  Pawpularity  path  norm_score  bins  fold
Focus
1      0      1      1      1      0          0      0      0      0      0      0      0      30  score/train/d7d222be164177b8a1033fb51a5d762f.jpg  0.30  12      0
4      0      1      1      1      0          0      0      0      0      0      0      0      75  score/train/ddcfffbc1a816adbbe38339f19c6b5f9.jpg  0.75  32      0
5      0      1      1      0      0          0      1      0      0      0      0      0      35  score/train/2ccc26b6ecc9c8ff4153761f6a03f58e.jpg  0.35  15      0
7      0      1      1      1      0          0      0      0      1      1      0      0      34  score/train/027a909fd3bc802f80646e92f8fcd50.jpg  0.34  14      0
19     0      1      1      0      0          0      1      0      1      0      0      0      40  score/train/b42907d9b673a42048bd2869b36bfe53.jpg  0.40  17      0

In [20]: df_train[df_train['fold']==0]['bins'].value_counts()
```

```
Out[20]:
```

11	134
15	100
12	89
13	87
10	77
14	70
9	70
7	60
17	54
20	53
8	51
16	51
18	49
43	44
19	36
23	35
21	30
27	25
0	24
6	24
22	23
4	22
5	20
25	18
24	18
31	16
26	14
28	13
3	11
29	10
1	10
30	9
35	9
32	8
33	8
2	7
34	7
38	6
36	6
40	6
37	4
42	3
41	3
39	2

Name: bins, dtype: int64

```
In [21]: df_train[df_train['fold']==1]['bins'].value_counts()
```

```
Out[21]:
```

11	134
15	100
12	89
13	86
10	77
14	70
9	70
7	60
20	53
17	53
16	52
8	51
18	49
43	43
19	36
23	35
21	30
27	25
0	24
6	23
4	22
22	22
5	21
25	19
24	18
31	16
26	15
28	14
3	11
29	10
1	10
30	9
2	8
35	8
32	8
33	8
34	7
38	6
40	6
36	6
37	4
41	3
39	3
42	2

Name: bins, dtype: int64

```
In [22]: def rmse(input_, target):
          res = torch.sqrt(F.mse_loss(Pawpularity_max*F.sigmoid(input_.flatten()), Pawpularity_max*target))
          return res
```

Load data

```
In [23]: fraction = 0.1 # val data fraction
          valid_col = 'is_valid'
          fn_col = 'path'
          label_col = 'norm_score'
          batch_size = 8
          num_workers = 8 # for GPU parallel computing
          item_tfms = 224
          batch_tfms = setup_aug_tfms([Brightness(), Contrast(), Hue(), Saturation()])
          # batch_tfms = setup_aug_tfms([Rotate(), Zoom(), Brightness(), Contrast()])
```

```
In [24]: def get_data(fold):
          df_train_f = df_train.copy()
          # add is_valid for validation fold
          df_train_f['is_valid'] = (df_train_f['fold'] == fold)
          dls = ImageDataLoaders.from_df(
              df_train_f, #pass in train DataFrame
```

```

        valid_pct=fraction, # train-validation random split
        valid_col=valid_col,
        seed=seed, #seed
        fn_col=fn_col, #filename/path is in the second column of the DataFrame
        label_col=label_col, #label is in the first column of the DataFrame
        y_block=RegressionBlock, #The type of target
        bs=batch_size, #pass in batch size
        num_workers=num_workers,
        item_tfms=Resize(224), #pass in item_tfms
        batch_tfms=batch_tfms #pass in batch_tfms
    )

    return dls

In [25]: # valid Kfolder size
the_data = get_data(0)
assert (len(the_data.train) + len(the_data.valid)) == (len(df_train)//batch_size)

```

Model

```

In [26]: def get_learner(fold_num):
data = get_data(fold_num)
model = create_model('swin_large_patch4_window7_224', pretrained=True, num_classes=data.c)
learn = Learner(data,
                model,
                loss_func=BCEWithLogitsLossFlat(),
                metrics=rmse).to_fp16()

    return learn

```

```

In [27]: df_test = pd.read_csv(dataset_path/'test.csv')
df_test.head()

```

```

Out[27]:

```

	Id	Subject	Focus	Eyes	Face	Near	Action	Accessory	Group	Collage	Human	Occlusion	Info	Blur
0	4128bae22183829d2b5fea10effdb0c3			1	0	1	0	0	1	1	0	0	1	0
1	43a2262d7738e3d420d453815151079e			0	1	0	0	0	0	1	1	0	0	0
2	4e429cead1848a298432a0acad014c9d			0	0	0	1	0	1	1	1	0	1	1
3	80bc3ccafcc51b66303c2c263aa38486			1	0	1	0	0	0	0	0	0	0	1
4	8f49844c38293144ae68dfbe20228f4			1	1	1	0	1	1	0	1	0	1	1

```

In [28]: df_test['Pawpularity'] = [1]*len(df_test)
df_test['path'] = df_test['Id'].map(lambda x: str(dataset_path/'test'/x)+'.jpg')
df_test = df_test.drop(columns=['Id'])
df_test['norm_score'] = df_test['Pawpularity']/100

```

```

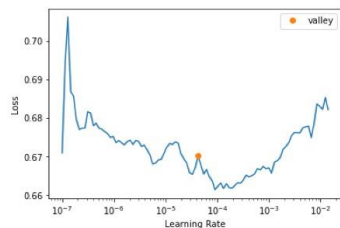
In [29]: get_learner(fold_num=0).lr_find(end_lr=3e-2)

```

```

Out[29]: SuggestedLRs(valley=4.256162355886772e-05)

```



```

In [30]: all_preds = []

for i in range(N_FOLDS):
    print(f'Fold {i} results')

    learn = get_learner(fold_num=i)
    learn.fit_one_cycle(6, 2e-5, cbs=[SaveModelCallback(),
                                     EarlyStoppingCallback(monitor='rmse',
                                                            comp=np.less,
                                                            patience=3)
                                     ])

    learn.recorder.plot_loss()

    dls = ImageDataLoaders.from_df(df_train, #pass in train DataFrame
                                  valid_pct=fraction,
                                  valid_col=valid_col,
                                  seed=seed, #seed
                                  fn_col=fn_col, #filename/path is in the second column of the DataFrame
                                  label_col=label_col, #label is in the first column of the DataFrame
                                  y_block=RegressionBlock, #The type of target
                                  bs=batch_size, #pass in batch size
                                  num_workers=num_workers,
                                  item_tfms=Resize(224), #pass in item_tfms
                                  batch_tfms=setup_aug_tfms(batch_tfms)
                                  )

    test_dl = dls.test_dl(df_test)

    preds, _ = learn.tta(dl=test_dl, n=5, beta=0)

    all_preds.append(preds)

    learn = learn.to_fp32()

    now = datetime.now()
    # learn.export(f'model_fold_{i}_.' + now.strftime("%Y-%m-%d--%H-%M-%S") + '.pkl')
    learn.save(f'model_fold_{i}_.' + now.strftime("%Y-%m-%d--%H-%M-%S") + '.pth')

    del learn, dls, test_dl
    torch.cuda.empty_cache()
    gc.collect()

Fold 0 results

```

epoch	train_loss	valid_loss	rmse	time
0	0.643291	0.645076	17.418671	08:17
1	0.642339	0.642979	16.912695	08:18
2	0.636257	0.642331	16.919899	08:18
3	0.623331	0.644252	16.943672	08:19
4	0.613556	0.645781	17.011297	08:18

```
/opt/conda/lib/python3.7/site-packages/torch/nn/functional.py:1805: UserWarning: nn.functional.sigmoid is deprecated. Use torch.sigmoid instead.
  warnings.warn("nn.functional.sigmoid is deprecated. Use torch.sigmoid instead.")
Better model found at epoch 0 with valid_loss value: 0.6450755000114441.
Better model found at epoch 1 with valid_loss value: 0.6429791450500488.
Better model found at epoch 2 with valid_loss value: 0.6423311233520508.
No improvement since epoch 1: early stopping
```

Fold 1 results

epoch	train_loss	valid_loss	rmse	time
0	0.641674	0.649030	17.585093	08:20
1	0.642229	0.645718	17.342482	08:18
2	0.631204	0.637766	16.337038	08:18
3	0.622376	0.640542	16.602400	08:18
4	0.602049	0.641795	16.639246	08:18
5	0.612993	0.642279	16.686745	08:18

```
/opt/conda/lib/python3.7/site-packages/torch/nn/functional.py:1805: UserWarning: nn.functional.sigmoid is deprecated. Use torch.sigmoid instead.
  warnings.warn("nn.functional.sigmoid is deprecated. Use torch.sigmoid instead.")
Better model found at epoch 0 with valid_loss value: 0.6490300893783569.
Better model found at epoch 1 with valid_loss value: 0.6457176208496094.
Better model found at epoch 2 with valid_loss value: 0.6377663016319275.
No improvement since epoch 2: early stopping
```

Fold 2 results

epoch	train_loss	valid_loss	rmse	time
0	0.646150	0.642604	16.966703	08:19
1	0.644309	0.638100	16.467430	08:19
2	0.639252	0.640168	16.665762	08:19
3	0.618775	0.641444	16.724682	08:19
4	0.613862	0.643182	17.015232	08:19

```
/opt/conda/lib/python3.7/site-packages/torch/nn/functional.py:1805: UserWarning: nn.functional.sigmoid is deprecated. Use torch.sigmoid instead.
  warnings.warn("nn.functional.sigmoid is deprecated. Use torch.sigmoid instead.")
Better model found at epoch 0 with valid_loss value: 0.6426041722297668.
Better model found at epoch 1 with valid_loss value: 0.638099730014801.
No improvement since epoch 1: early stopping
```

Fold 3 results

epoch	train_loss	valid_loss	rmse	time
0	0.644695	0.644044	17.040936	08:19
1	0.640707	0.642751	16.875189	08:19
2	0.631035	0.642375	16.827433	08:19
3	0.623144	0.647279	17.333691	08:18
4	0.606796	0.648815	17.398142	08:19
5	0.611297	0.650318	17.485939	08:20

```
/opt/conda/lib/python3.7/site-packages/torch/nn/functional.py:1805: UserWarning: nn.functional.sigmoid is deprecated. Use torch.sigmoid instead.
  warnings.warn("nn.functional.sigmoid is deprecated. Use torch.sigmoid instead.")
Better model found at epoch 0 with valid_loss value: 0.6440438032150269.
Better model found at epoch 1 with valid_loss value: 0.6427509188652039.
Better model found at epoch 2 with valid_loss value: 0.6423750519752502.
No improvement since epoch 2: early stopping
```

Fold 4 results

epoch	train_loss	valid_loss	rmse	time
0	0.648506	0.645819	17.371634	08:19
1	0.644327	0.650965	18.286100	08:19
2	0.628387	0.643451	17.083652	08:18
3	0.626162	0.641603	16.919794	08:18
4	0.612185	0.644337	17.183104	08:19
5	0.605473	0.643913	17.073513	08:18

```
/opt/conda/lib/python3.7/site-packages/torch/nn/functional.py:1805: UserWarning: nn.functional.sigmoid is deprecated. Use torch.sigmoid instead.
  warnings.warn("nn.functional.sigmoid is deprecated. Use torch.sigmoid instead.")
Better model found at epoch 0 with valid_loss value: 0.6458187103271484.
Better model found at epoch 2 with valid_loss value: 0.6434512734413147.
Better model found at epoch 3 with valid_loss value: 0.6416029930114746.
```

Fold 5 results

epoch	train_loss	valid_loss	rmse	time
0	0.640512	0.644980	17.113544	08:20
1	0.640024	0.641543	16.770023	08:20
2	0.636254	0.640009	16.530115	08:18
3	0.620218	0.646360	17.226076	08:18
4	0.620173	0.647418	17.255497	08:18
5	0.608401	0.648370	17.349083	08:18

```
/opt/conda/lib/python3.7/site-packages/torch/nn/functional.py:1805: UserWarning: nn.functional.sigmoid is deprecated. Use torch.sigmoid instead.
  warnings.warn("nn.functional.sigmoid is deprecated. Use torch.sigmoid instead.")
```

```

Better model found at epoch 0 with valid_loss value: 0.6449798345565796.
Better model found at epoch 1 with valid_loss value: 0.641543447971344.
Better model found at epoch 2 with valid_loss value: 0.6400085091590881.
No improvement since epoch 2: early stopping

```

```

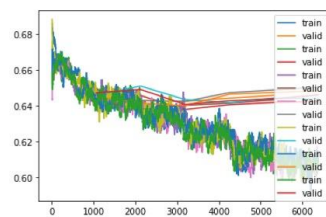
Fold 6 results
epoch  train_loss  valid_loss  rmse  time
0      0.641138    0.647199    17.501120  08:18
1      0.642871    0.649098    18.083582  08:18
2      0.624334    0.640699    16.737337  08:19
3      0.620310    0.642668    17.028461  08:18
4      0.620121    0.643868    17.034914  08:18
5      0.609383    0.645949    17.288437  08:18

```

```

/opt/conda/lib/python3.7/site-packages/torch/nn/functional.py:1805: UserWarning: nn.functional.sigmoid is deprecated. Use torch.sigmoid instead.
warnings.warn("nn.functional.sigmoid is deprecated. Use torch.sigmoid instead.")
Better model found at epoch 0 with valid_loss value: 0.6471994519233704.
Better model found at epoch 2 with valid_loss value: 0.6406985521316528.
No improvement since epoch 2: early stopping

```



```
In [31]: all_preds
```

```

Out[31]: [tensor([[0.4930],
          [0.5050],
          [0.4970],
          [0.5005],
          [0.5118],
          [0.5117],
          [0.4965],
          [0.5009]]),
          tensor([[0.3523],
          [0.3746],
          [0.3573],
          [0.3779],
          [0.3572],
          [0.3604],
          [0.3570],
          [0.3632]]),
          tensor([[0.4629],
          [0.4706],
          [0.4612],
          [0.4723],
          [0.4647],
          [0.4672],
          [0.4680],
          [0.4710]]),
          tensor([[0.3894],
          [0.4001],
          [0.3842],
          [0.4020],
          [0.3897],
          [0.4005],
          [0.3947],
          [0.3853]]),
          tensor([[0.3862],
          [0.3902],
          [0.3884],
          [0.3966],
          [0.3839],
          [0.3958],
          [0.3896],
          [0.3888]]),
          tensor([[0.4114],
          [0.4091],
          [0.4026],
          [0.4216],
          [0.3953],
          [0.4157],
          [0.4098],
          [0.4239]]),
          tensor([[0.4022],
          [0.4149],
          [0.4024],
          [0.4161],
          [0.3940],
          [0.4097],
          [0.4063],
          [0.4083]])]

```

```
In [32]: np.mean(np.stack(all_preds*100))
```

```
Out[32]: 0.41897792
```

```

In [33]: sample_df = pd.read_csv(dataset_path/'sample_submission.csv')
         preds = np.mean(np.stack(all_preds), axis=0)
         sample_df['Pawpularity'] = preds*100
         sample_df.to_csv('submission.csv', index=False)

```

```
In [34]: pd.read_csv('submission.csv').head()
```

Out[34]:

		Id	Pawpularity
0	4128bae22183829d2b5fea10effdb0c3		41.390846
1	43a2262d7738e3d420d453815151079e		42.349808
2	4e429cead1848a298432a0acad014c9d		41.329525
3	80bc3ccafcc51b66303c2c263aa38486		42.671608
4	8f49844c382931444e68dffbe20228f4		41.378574

ДОДАТОК Г Лістинг програми “app.py”

```

import os
from datetime import datetime

import pandas as pd
from flask import Flask, render_template, request, redirect, flash, url_for, send_file, Response

from database_func import connect_to_mysql_database
from neural_net_func import predict_attractiveness

""" ----- FLASK PARAMETERS ----- """
# створюємо додаток через Flask
app = Flask(__name__)
# секретний ключ для додатку
app.secret_key = "\x1e\x8e\xe3\xb7P\xfa\xce\xel`\xa9\xf4|TQS"
# глобальна змінна, що зберігатиме шлях до оброблюваного зображення
image_path = ""
# глобальна змінна для зберігання часу та дати,
# коли було класифіковане зображення для запису у історію БД
my_date_time = ""

# підключаємося до MySQL
my_db_connection = connect_to_mysql_database()

""" ----- FLASK FUNCTIONS ----- """

@app.route("/index.html")
def Home():
    # Якщо є з'єднання із БД - виводимо випадочний список можливих тварин
    if (my_db_connection):
        my_select = "SELECT * "
        my_from = "FROM PETS "
        my_query = my_select + my_from
        cursor = my_db_connection.cursor()
        cursor.execute(my_query)
        my_records = cursor.fetchall()
        # print("• Total number of rows in DB is: ", cursor.rowcount)
        # for row in my_records:
        #     print(row)
        return render_template('index.html', records=my_records)
    # Якщо БД не підключена - то і випадочного списку немає
    else:
        return render_template("index.html")

# UPLOAD AN IMAGE
@app.route("/index.html", methods=["POST"])
def upload_image():
    if request.method == "POST":
        # отримуємо завантажену фотографію
        file = request.files["file-image"]
        if (file.filename != ""):

            # Отримуємо результат вибору із випадочного списку (вибір тварини)
            get_selected_pet = str(request.form.get("select_1"))
            # Отримуємо результат опису тварини із вікна для введення тексту
            get_pet_description = str(request.form.get("text_area_1"))

            # Якщо всі ячейки заповнені (непророжні)
            if (get_selected_pet != "" and get_pet_description != ""):
                print("-----")
                # розбиваємо запис на окремі складові
                get_selected_pet = get_selected_pet.split("/")
                # отримуємо pet_id
                get_selected_pet_id = get_selected_pet[0]
                print("• get_selected_pet_id = ", get_selected_pet_id)

                # змінюємо назву зображення на таку, яка мене влаштовує, а не така, яка вона була з початку
                old_name = file.filename
                # розбиваємо назву зображення на власне назву і
                # розширення зображення з кінця і тільки один раз (тобто назва.розширення)
                name, ext = old_name.rsplit('.', 1)

                # назва зображення складається із дати і часу, коли воно було завантажено
                my_date = str(datetime.now().date())
                my_time = str(datetime.now().time())

                # також зберігаємо повне значення часу та дати, для подальшого запису у історію БД
                global my_date_time
                my_date_time = my_date + " " + my_time

                # робимо потрібне форматування для того, щоб із дати та часу створити нову назву завантаженого зображення
                my_time = my_time.replace(":", "-") # замінюємо непотрібні символи на бажані
                my_time = my_time.replace(".", "-") # замінюємо непотрібні символи на бажані
                result = my_date + "_" + my_time # створюємо бажану назву зображення

```

```

# утворюємо нову назву + додаємо розширення зображення
new_name = result + "." + ext

# зберігаємо нову назву зображення
file.filename = new_name

# шлях до моделей
my_path = './static/uploaded_images/'

global image_path
image_path = my_path + file.filename
print("• image_path = ", image_path)
print()
# зберігаємо зображення у цю папку
file.save(image_path)

# ПРОГНОЗУВАННЯ ПРИВАБЛИВОСТІ ТВАРИНИ ІЗ ПРИТУЛКУ
result = predict_attractiveness(image_path)
result = round(result, 2)

# видаляємо зображення, так як воно нам більше не потрібне
if os.path.exists(image_path):
    os.remove(image_path)
    print("image was deleted from the folder")

if (my_db_connection):
    my_insert = "INSERT INTO CASE_HISTORY (case_datetime, prediction_result, case_info,
PETS_pet_id) "
    my_values = "VALUES( '" + \
        + str(my_date_time) + "', '" + \
        + str(result) + "', '" + \
        + str(get_pet_description) + "', '" + \
        + str(get_selected_pet_id) + "');"

    my_query = my_insert + my_values
    print(my_query)
    cursor = my_db_connection.cursor()
    cursor.execute(my_query)
    my_db_connection.commit() # ОБОВ'ЯЗКОВО!! Інакше зміни не будуть виконані

    print("THE RECORD WAS SUCCESSFULLY ADDED TO CASE HISTORY. PREDICTION RESULT:", result)
    flash(str(result), "prediction-result")
    flash(str(my_date), "prediction-result")
    flash(str(my_time), "prediction-result")
    return redirect(url_for('Home', _anchor='anchor-image-window'))
else:
    flash("UNABLE TO CONNECT TO DATABASE. PLEASE, TRY LATER!", category="error")
    return redirect(url_for('Home', _anchor='anchor-image-window'))

# Якщо чогось не всі яєчки вибрані чи заповнені - виводимо повідомлення про помилку
else:
    flash('NOT ALL NOT ALL ITEMS ARE FILLED / SELECTED. TRY AGAIN!', category="error")
    return redirect(url_for('Home', _anchor='anchor-image-window'))

# Якщо не вибрана фотографія - повідомлення про помилку
else:
    print("MEOW-3")
    flash('NO IMAGE WAS SELECTED FOR UPLOADING. PLEASE, CHOOSE AN IMAGE!', category="error")
    return redirect(url_for('Home', _anchor='anchor-image-window'))

# GET DATA FROM DATABASE (CASE HISTORY)
@app.route("/getCaseHistoryCSV", methods=["GET"])
def get_case_history():
    if request.method == "GET":
        if (my_db_connection):
            my_select = "SELECT * "
            my_from = "FROM CASE_HISTORY "
            my_query = my_select + my_from
            cursor = my_db_connection.cursor()
            cursor.execute(my_query)
            my_records = cursor.fetchall()
            my_columns = [i[0] for i in cursor.description]
            print("• Total number of rows in DB is: ", cursor.rowcount)
            print("• Column names: ", my_columns)
            for row in my_records:
                print(row)

            caseHistory = pd.DataFrame(data=my_records, columns=my_columns)

            return Response(
                caseHistory.to_csv(),
                mimetype="text/csv",
                headers={"Content-disposition":
                    "attachment; filename=case-history.csv"})

```

```

# GET DATA FROM DATABASE (CASE HISTORY)
@app.route("/getPetsInfoCSV", methods=["GET"])
def get_pets_info():
    if request.method == "GET":
        if (my_db_connection):

            my_select = "SELECT * "
            my_from = "FROM PETS "
            my_query = my_select + my_from
            cursor = my_db_connection.cursor()
            cursor.execute(my_query)
            my_records = cursor.fetchall()
            my_columns = [i[0] for i in cursor.description]
            print("* Total number of rows in DB is: ", cursor.rowcount)
            print("* Column names: ", my_columns)
            for row in my_records:
                print(row)

            petsInfo = pd.DataFrame(data=my_records, columns=my_columns)

            return Response(
                petsInfo.to_csv(),
                mimetype="text/csv",
                headers={"Content-disposition":
                    "attachment; filename=pets-info.csv"})

if __name__ == '__main__':
    app.run(host='0.0.0.0')

```

ДОДАТОК Г Лістинг програми “database_func.py”

```
import mysql.connector
from mysql.connector import Error

# ПІДКЛЮЧЕННЯ ДО MYSQL
def connect_to_mysql_database():
    try:
        connection = mysql.connector.connect(host='localhost',
                                             database='mydb',
                                             user='root',
                                             password='Diploma2022',
                                             use_pure=True)

        db_Info = connection.get_server_info()

        print("• Connected to MySQL Server version ", db_Info)
        cursor = connection.cursor()
        cursor.execute("select database();")
        record = cursor.fetchone()
        print("• You're connected to database: ", record)
        print("• DB connection = ", connection)
        print()
        return connection

    except Error as e:
        print("ERROR WHILE CONNECTING TO MySQL: ", e)
        print()
        raise SystemExit()
```

ДОДАТОК Д Лістинг програми “neural_net_func.py”

```

import gc
import math
import datetime
import sys
from timm import create_model
from fastai.vision.all import *
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import KFold
from sklearn.model_selection import StratifiedKFold
import pandas as pd
import warnings

warnings.filterwarnings('ignore')

def predict_attractiveness(path_to_uploaded_image):
    # ДЛЯ ТОГО, ЩОБ ЗАПУСТИТИ ВСІ МОДЕЛІ (folds №i) ДЛЯ ПРОГНОЗУВАННЯ ПРИВАБЛИВОСТІ,
    # ПОТРІБНО СПОЧАТКУ ВІДТВОРИТИ ВЕСЬ КОД, ЩО БУВ НАПИСАНИЙ ДЛЯ НАВЧАННЯ МОДЕЛЕЙ.
    # ЦЕ РОБИТЬСЯ ДЛЯ ТОГО, ЩОБ НЕ БУЛО ПРОБЛЕМ ІЗ ЗАПУСКОМ НАВЧЕНОЇ МОДЕЛІ FASTAI

    set_seed(2999, reproducible=True)

    # dataset path
    dataset_path = Path('./static/dataset/petfinder-pawpularity-score/')

    df_train = pd.read_csv(dataset_path / 'train.csv')

    df_train['path'] = df_train['Id'].map(lambda x: str(dataset_path / 'train' / x) + '.jpg')
    df_train = df_train.drop(columns=['Id'])
    # shuffle dataframe
    df_train = df_train.sample(frac=1).reset_index(drop=True)

    Pawpularity_max = 100

    df_train['norm_score'] = df_train['Pawpularity'] / Pawpularity_max

    # set seed
    seed = 999
    set_seed(seed, reproducible=True)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.backends.cudnn.deterministic = True
    torch.use_deterministic_algorithms = True

    # Sturges' rule
    num_bins = int(np.floor(1 + (3.3) * (np.log2(len(df_train))))))

    df_train['bins'] = pd.cut(df_train['norm_score'], bins=num_bins, labels=False)

    N_FOLDS = 7

    df_train['fold'] = -1

    strat_kfold = StratifiedKFold(n_splits=N_FOLDS, random_state=seed, shuffle=True)

    for i, (_, train_index) in enumerate(strat_kfold.split(df_train.index, df_train['bins'])):
        df_train.iloc[train_index, -1] = i

    df_train['fold'] = df_train['fold'].astype('int')

    def rmse(input_, target):
        res = torch.sqrt(F.mse_loss(Pawpularity_max * F.sigmoid(input_.flatten()), Pawpularity_max * target))
        return res

    fraction = 0.1 # val data fraction
    valid_col = 'is_valid'
    fn_col = "path"
    label_col = 'norm_score'
    batch_size = 8
    num_workers = 8 # for GPU parallel computing
    item_tfms = 224
    batch_tfms = setup_aug_tfms([Brightness(), Contrast(), Hue(), Saturation()])

    # batch_tfms = setup_aug_tfms([Rotate(), Zoom(), Brightness(), Contrast()])

    def get_data(fold):
        df_train_f = df_train.copy()
        # add is_valid for validation fold
        df_train_f['is_valid'] = (df_train_f['fold'] == fold)
        dls = ImageDataLoaders.from_df(
            df_train_f, # pass in train DataFrame
            valid_pct=fraction, # train-validation random split
            valid_col=valid_col,
            seed=seed, # seed
            fn_col=fn_col, # filename/path is in the second column of the DataFrame
            label_col=label_col, # label is in the first column of the DataFrame

```



```

        item_tfms=Resize(224), # pass in item_tfms
        batch_tfms=setup_aug_tfms(batch_tfms)
    )

    test_dl = dls.test_dl(df_new_image)

    preds, _ = learn.tta(dl=test_dl, n=5, beta=0)
    print("preds = ", preds)

    all_preds.extend(preds)

    learn = learn.to_fp32()

    now = datetime.now()

    del learn, dls, test_dl
    torch.cuda.empty_cache()
    gc.collect()

    preds = np.mean(np.stack(all_preds), axis=0)
    preds = np.squeeze(preds)
    preds = float(preds) * 100 # множимо на 100, щоб було у межах [0, 100]

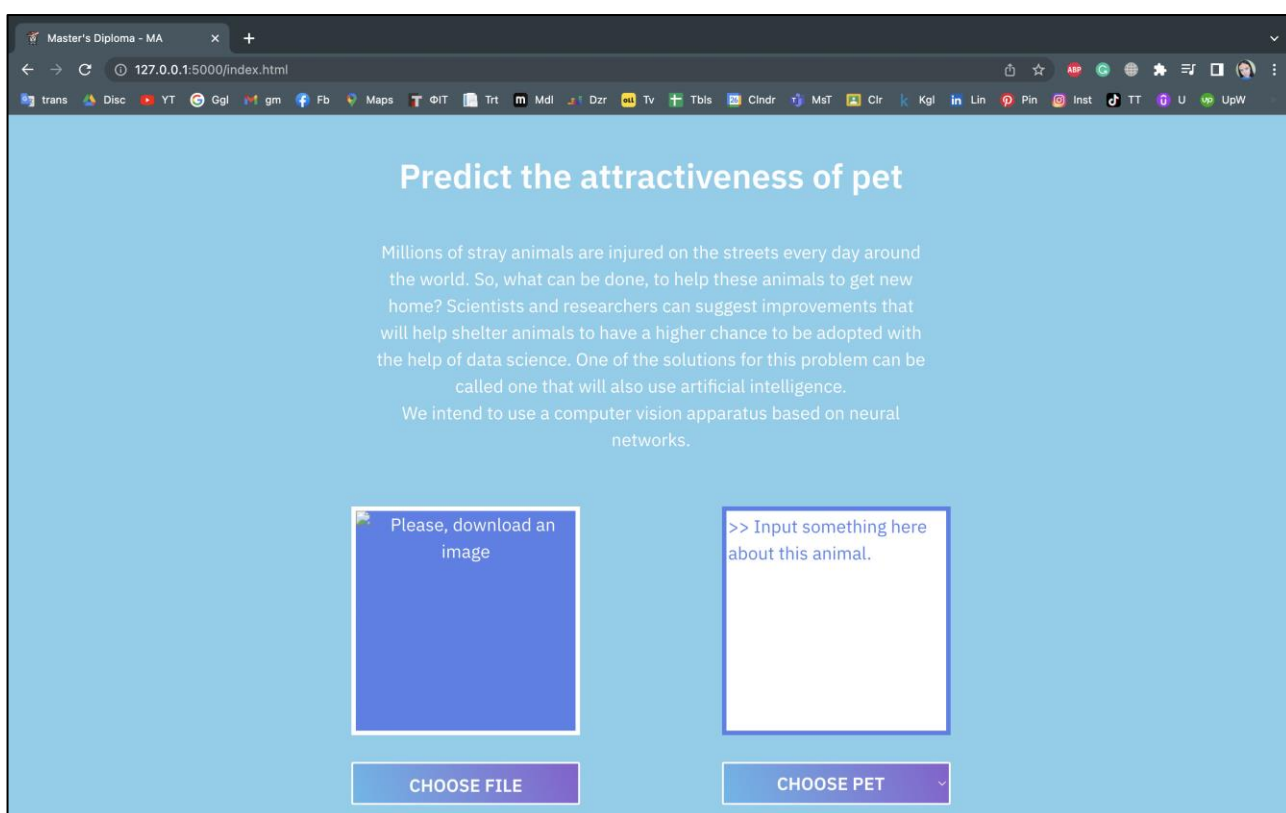
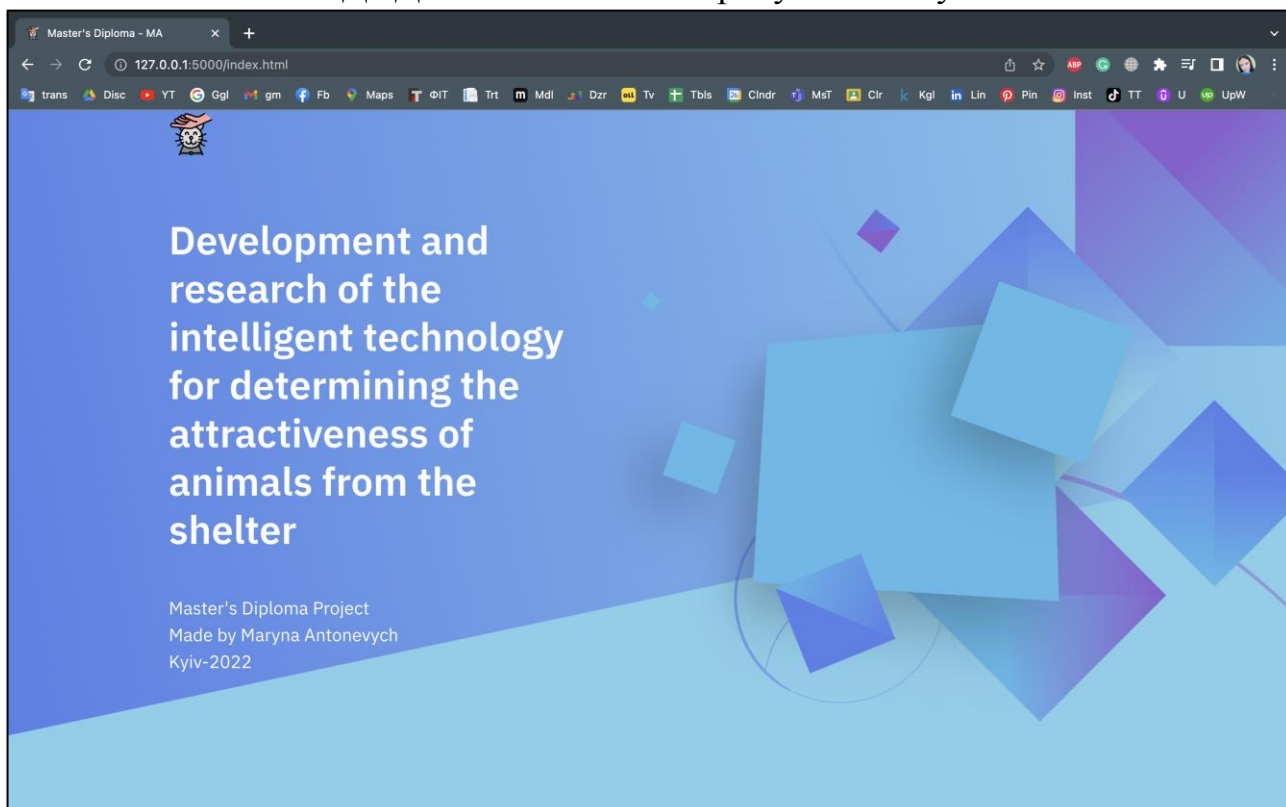
    del df_train, df_test, df_new_image

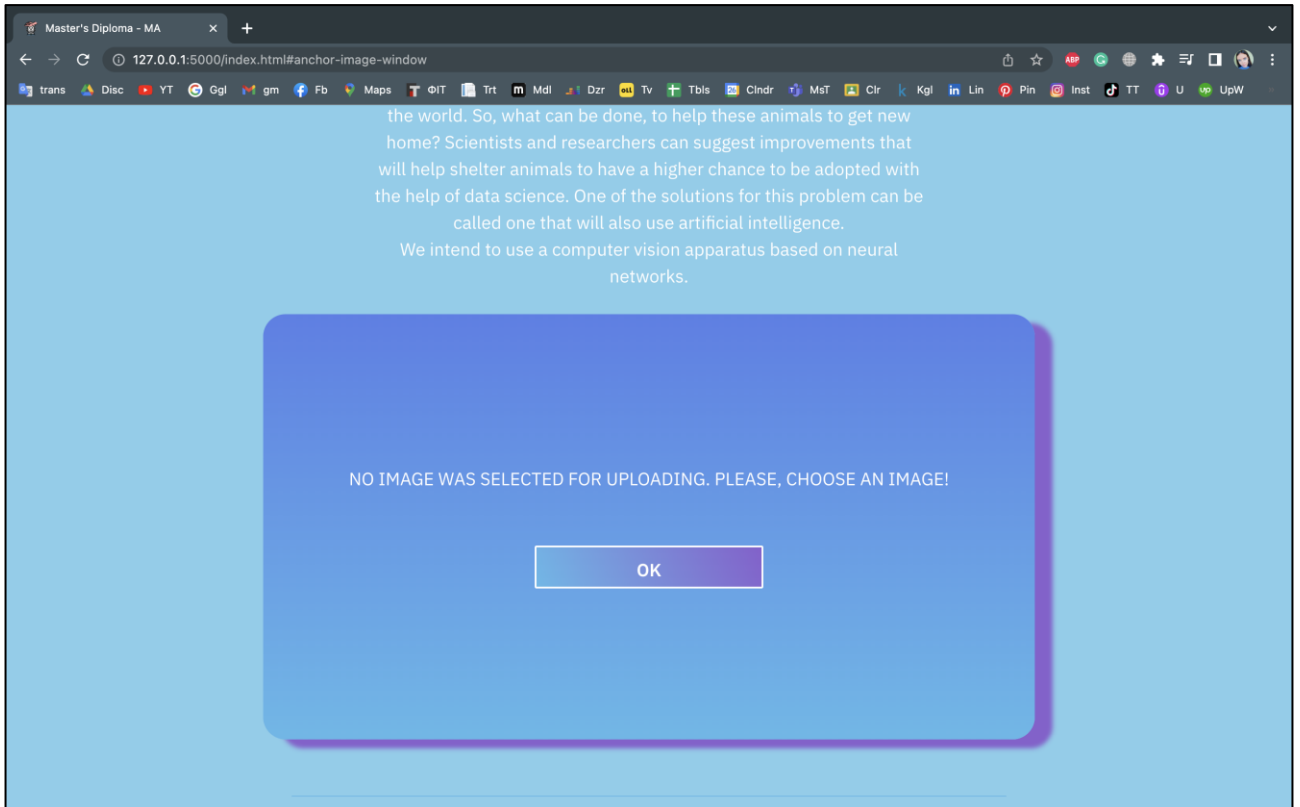
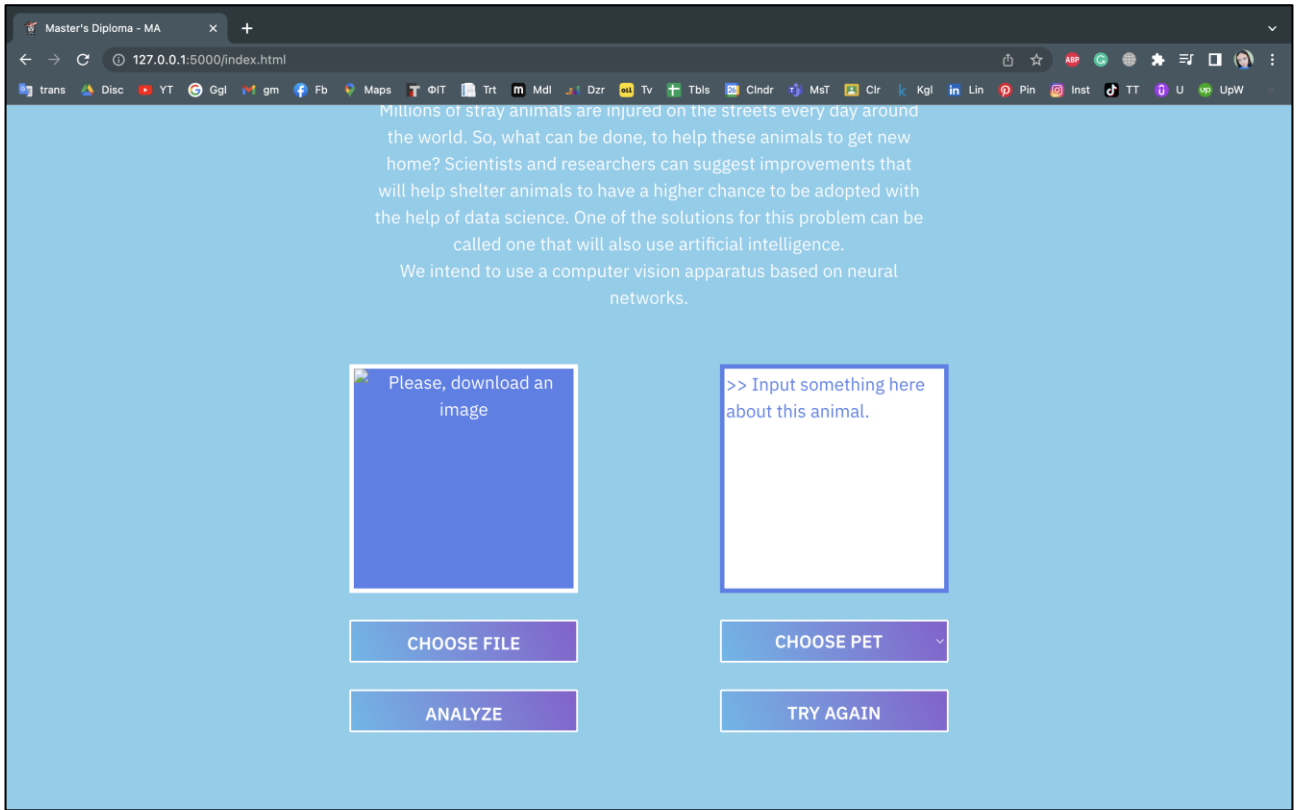
    return preds

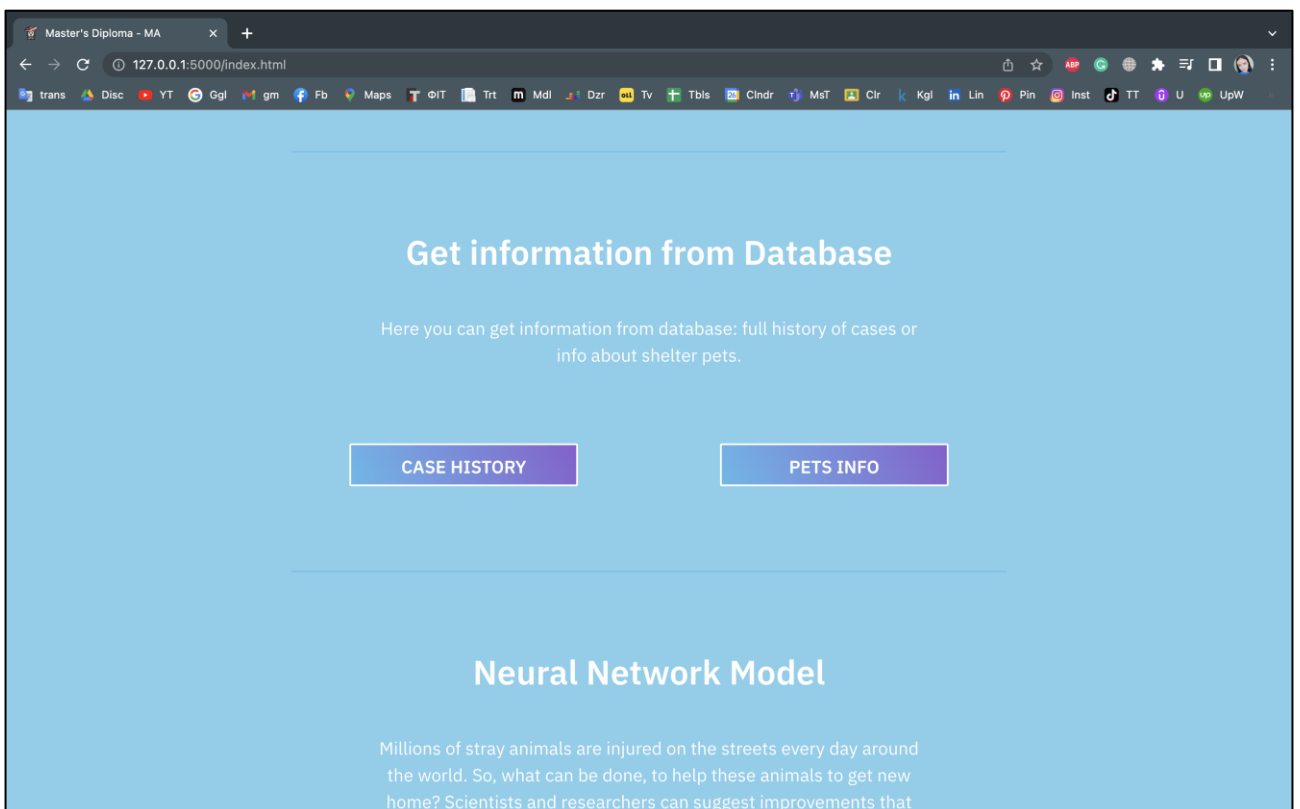
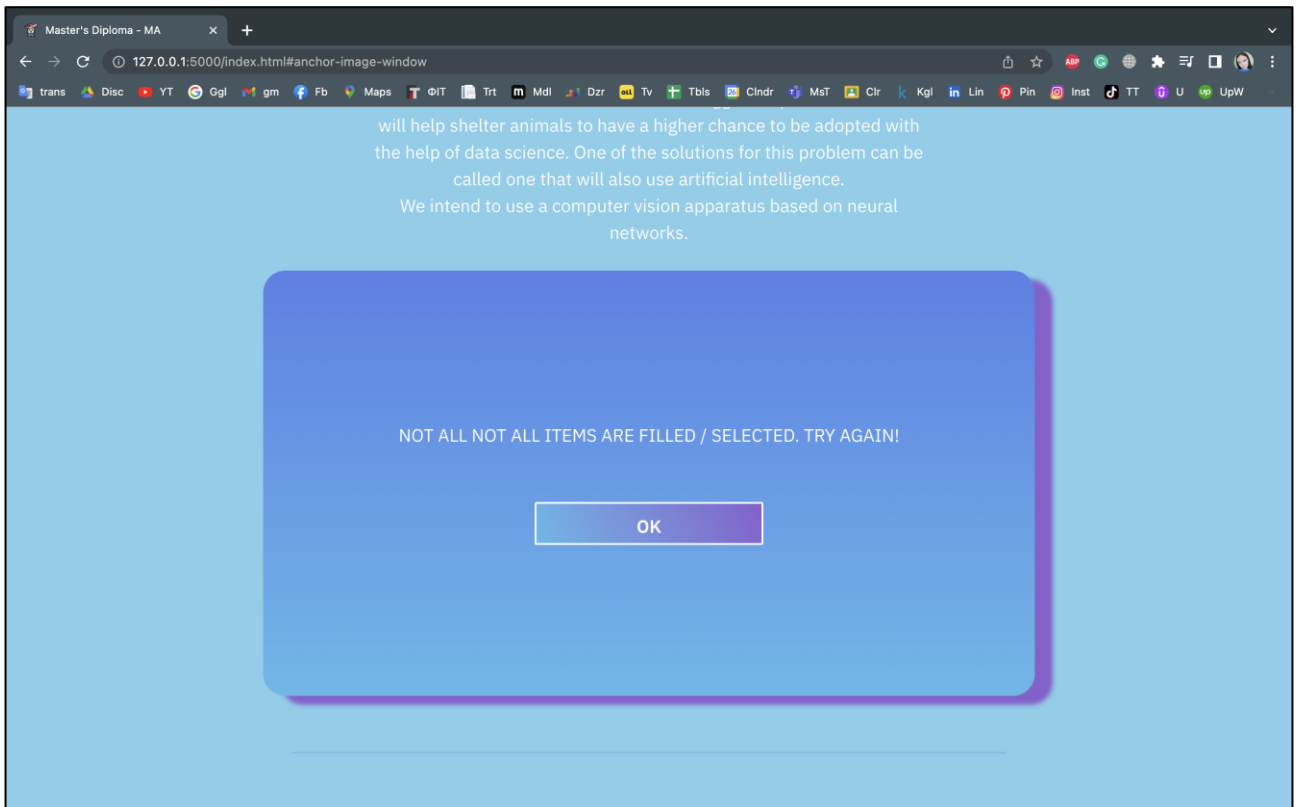
# # приклад запуску функції (для тестування)
# result = predict_pawpularity('./static/dataset/petfinder-pawpularity-
score/train/0a0da090aa9f0342444a7df4dc250c66.jpg')

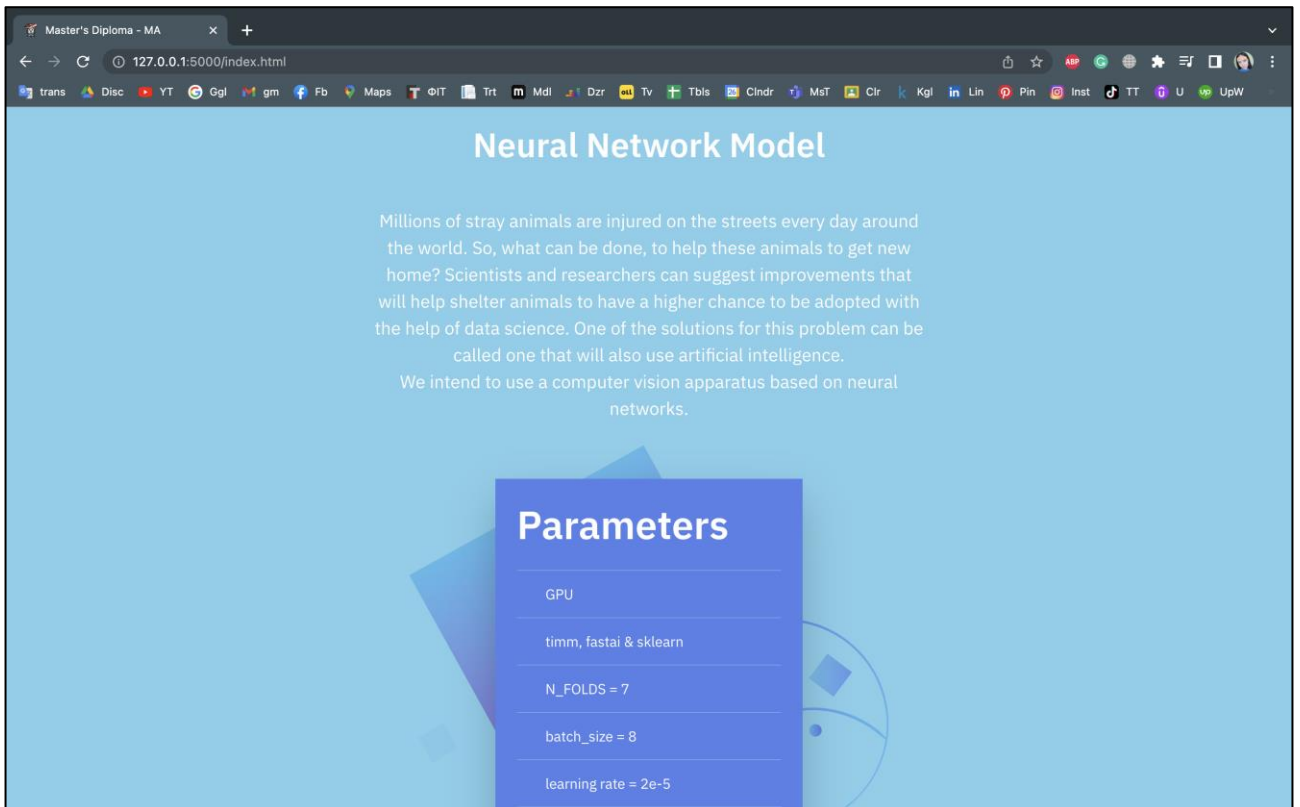
```

ДОДАТОК Е Знімки екрану веб-сайту









Master's Diploma - MA

127.0.0.1:5000/index.html

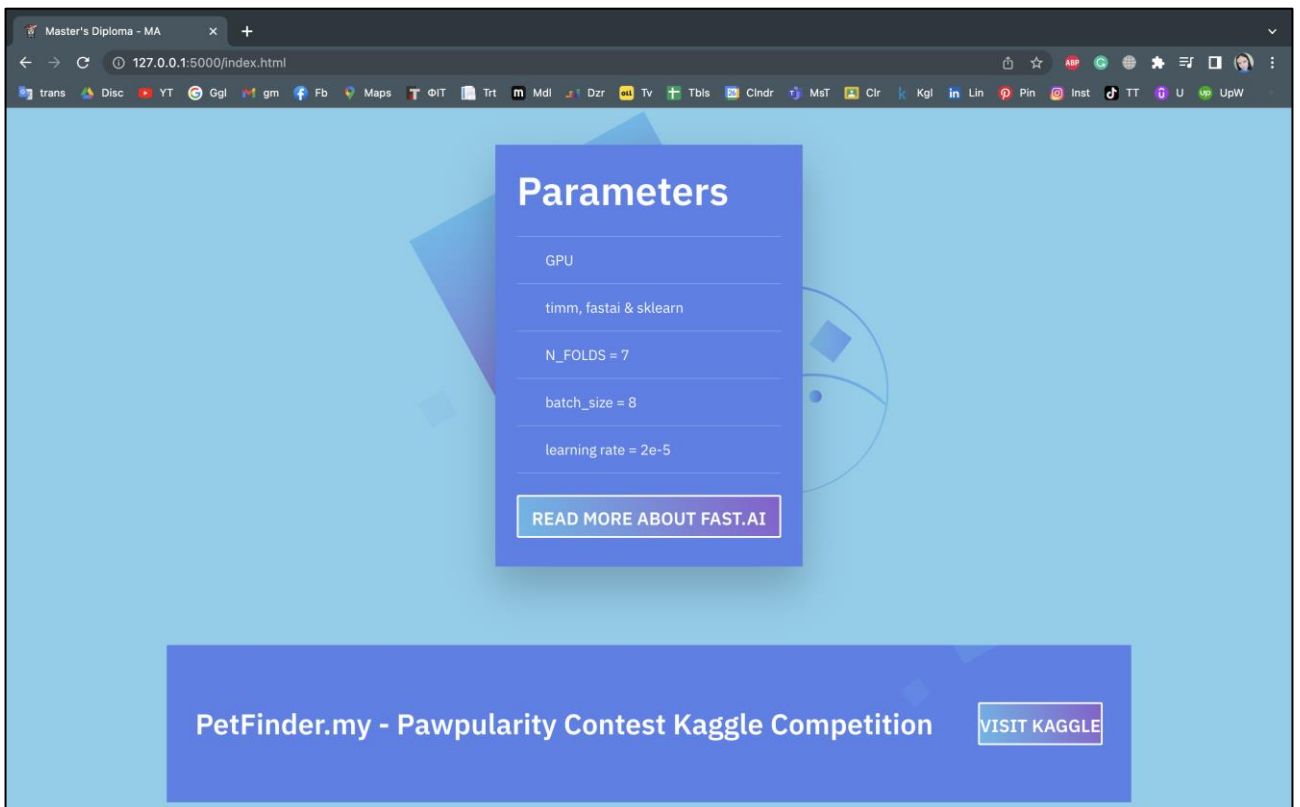
Neural Network Model

Millions of stray animals are injured on the streets every day around the world. So, what can be done, to help these animals to get new home? Scientists and researchers can suggest improvements that will help shelter animals to have a higher chance to be adopted with the help of data science. One of the solutions for this problem can be called one that will also use artificial intelligence.

We intend to use a computer vision apparatus based on neural networks.

Parameters

- GPU
- timm, fastai & sklearn
- N_FOLDS = 7
- batch_size = 8
- learning rate = 2e-5



Master's Diploma - MA

127.0.0.1:5000/index.html

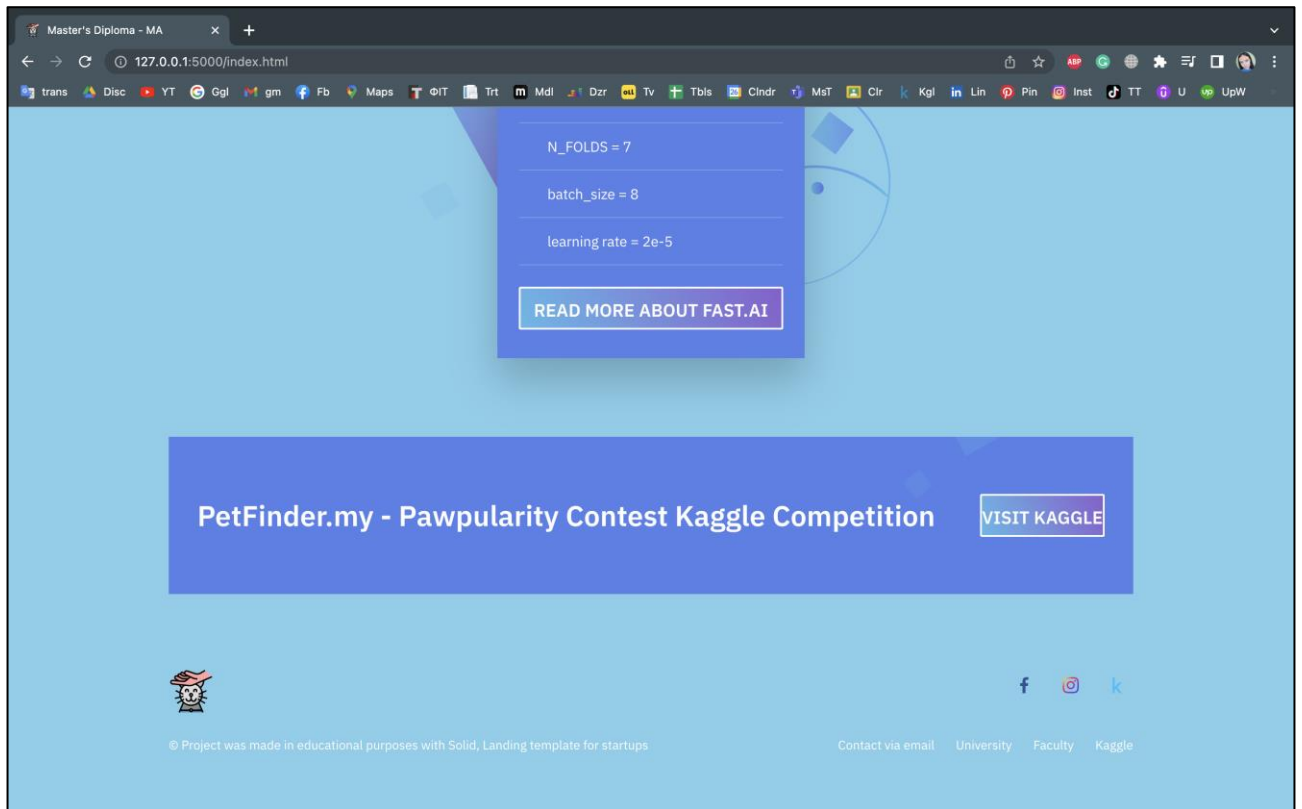
Parameters

- GPU
- timm, fastai & sklearn
- N_FOLDS = 7
- batch_size = 8
- learning rate = 2e-5

[READ MORE ABOUT FAST.AI](#)

PetFinder.my - Pawpularity Contest Kaggle Competition

[VISIT KAGGLE](#)



ДОДАТОК Є Знімки екрану таблиць бази даних MySQL

Limit to 1000 rows

```

1 SELECT *
2 FROM PETS

```

100% 10:2

Result Grid Filter Rows: Search Edit: Export/Import:

	pet_id	pet_category	pet_breed	pet_name	pet_age	pet_gender
▶ 1	1	cat	outbred	Rosie Meow	1	F
2	2	dog	outbred	Reks	2	M
3	3	cat	outbred	Tom	2	M
4	4	cat	outbred	Minnie	4	F
5	5	dog	outbred	Duke	4	M
6	6	dog	outbred	Leon	9	M
7	7	cat	outbred	Saun	2	M
8	8	dog	outbred	Mars	1	M
9	9	cat	outbred	Lili Luna	3	F
10	10	dog	outbred	Maks	4	M
	NULL	NULL	NULL	NULL	NULL	NULL

Limit to 1000 rows

```

1 SELECT *
2 FROM CASE_HISTORY

```

100% 18:2

Result Grid Filter Rows: Search Edit: Export/Import:

	case_id	case_datetime	prediction_res...	case_info	PETS_pet_id
▶ 1	1	2022-04-29 22:14:51	38.34	this is a white cat with blue eyes	1
2	2	2022-04-29 22:28:45	38.34	this is a white cat with blue eyes	1
	NULL	NULL	NULL	NULL	NULL